Sistemi bazirani na znanju

Predstavljanje znanja i rezonovanje

Sadržaj

- Motivacija i ciljevi
- Uvod i tema, istorija
  - Prikaz relevantnih koncepata
  - Pregled novih oblasti
  - terminologija
- Znanje i njegovo značenje
  - Epistemologija
  - Tipovi znanja
  - Piramida znanja
- SBZ i ES
  - ES bazirani na pravilima
  - Pravila
  - eksperti
- Metode reprezentacije znanja
  - Produkciona pravila
  - Semantičke mreže
  - Šeme i frejmovi
  - Logika
  - Semantički Web i RZ
- Ontologije
  - OWL
  - RDF
- Važni koncepti i termini
- Sažetak

Motivacija i ciljevi

- U SBZ se znanje mora predstaviti na neki način
  - SBZ su potpuno beskorisni ako u njima nemamo mogućnost predstave znanja ili nemamo znanja
- Postoje (kao što ćemo videti) različiti tipovi znanja
- Zbog toga za predstavljanje znanja mogu da budu prikladni različiti formati
  - Zavisno od zadatka i okolnosti
- Znanje se u SBZ koristi za zaključivanje
  - **Formati reprezentacije znanja moraju da budu usklađeni sa metodama zaključivanja**

Metode za reprezentaciju znanja (ZR metode)

- **Produkciona pravila**
- Stabla, grafovi (n.pr. Semantičke mreže, konceptne mape, konceptni grafovi, ontologija)
- Šeme i frejmovi
- Logika

=> Rezonovanje

Produkciona pravila

- Najčešće se koriste za predstavljanje znanja u ES
- Formalna varijacija produkcionih pravila je Backus-Naur forma (BNF)
  - Metajezik za definisanje sintakse jezika
  - Gramatika jezika je **kompletan, nedvosmislen skup produkcionih pravila za određeni jezik**
  - Stablo parsiranja je grafička reprezentacija rečenice u tom jeziku
  - Daju samo **sintaksni opis jezika**
    - Ne moraju sve rečenice da budu smislene

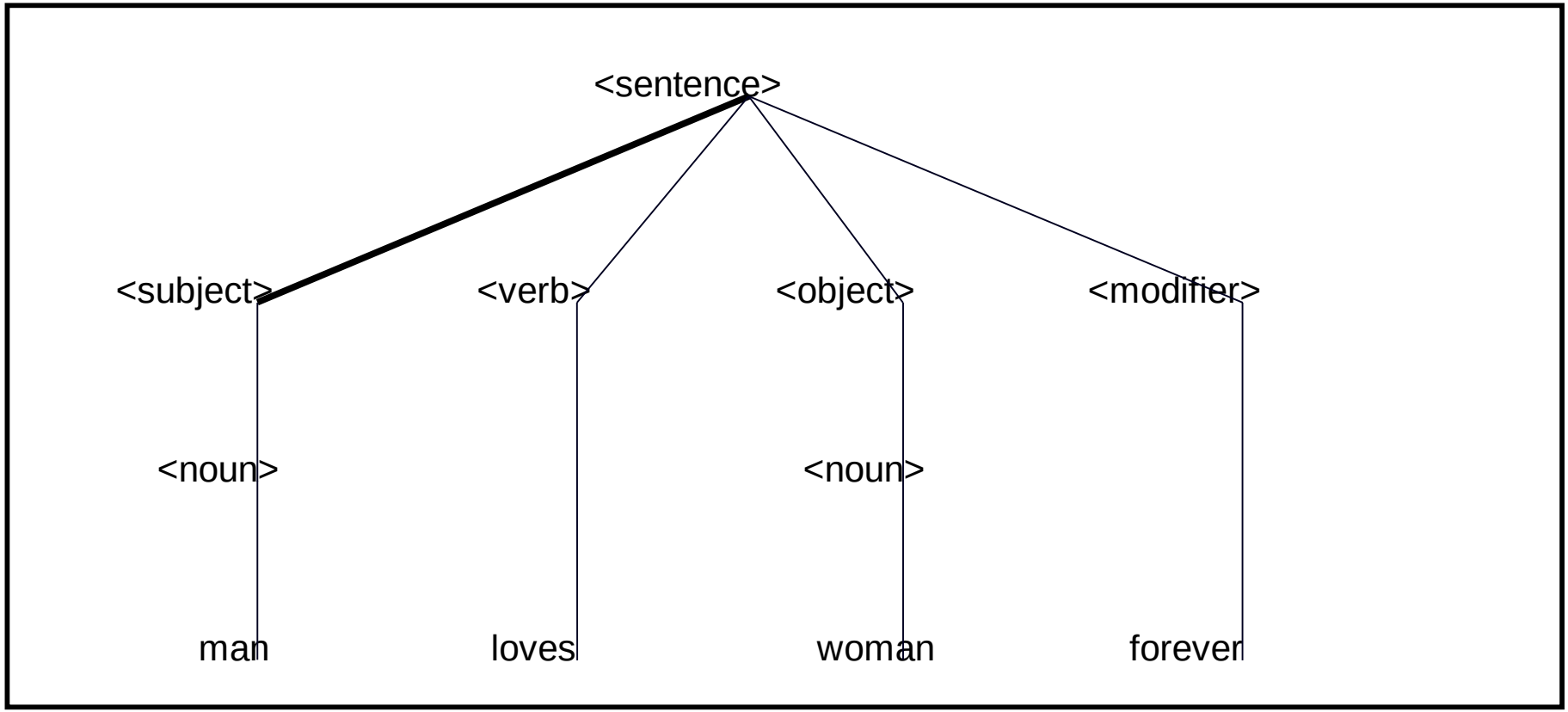
Primer 1: produkciona pravila

- Za podskup engleskog jezika

<sentence> -> <subject> <verb> <object> <modifier>
<subject> -> <noun>
<object> -> <noun>
<noun> -> man | woman
<verb> -> loves | hates | marries | divorces
<modifier> -> a little | a lot | forever | sometimes

Primer 1: Stablo parsiranja

- Primer rečenice: `man loves woman forever`



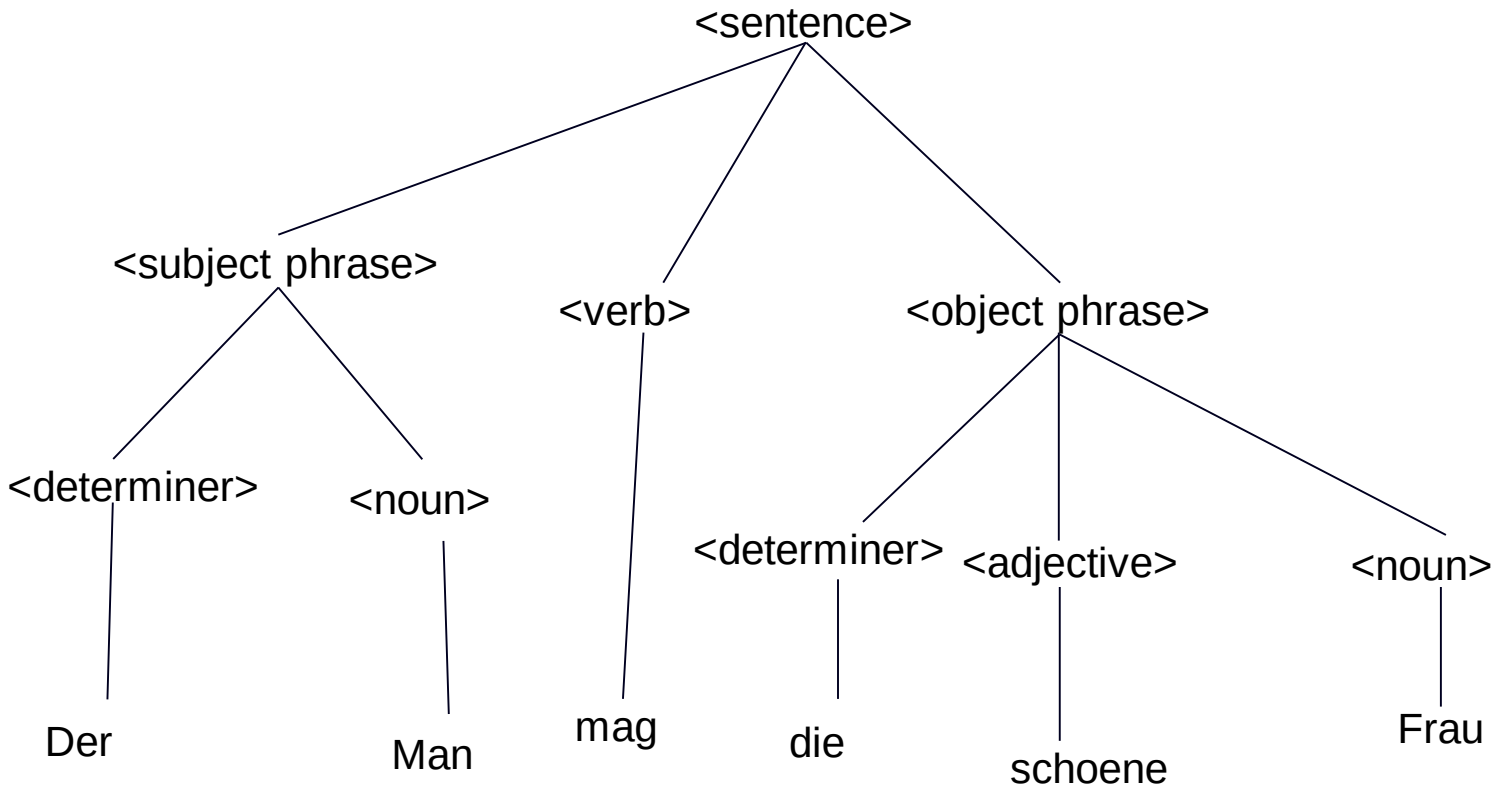
Primer 2 produkciona pravila

- Za podskup nemačkog jezika

```
<sentence>           -> <subject phrase> <verb> <object phrase>
<subject phrase>     -> <determiner> <adjective> <noun>
<object phrase>      -> <determiner> <adjective> <noun>
<determiner>  -> der | die | das | den
<noun>               -> Mann | Frau | Kind | Hund | Katze
<verb>               -> mag | schimpft | vergisst|
                                 verehrt | verzehrt
<adjective> -> schoene | stärke | laute | duenne
```

Primer 2 Stablo parsiranja

- Primer rečenice: Der Mann mag die schoene Frau



Primer 2 Stablo parsiranja

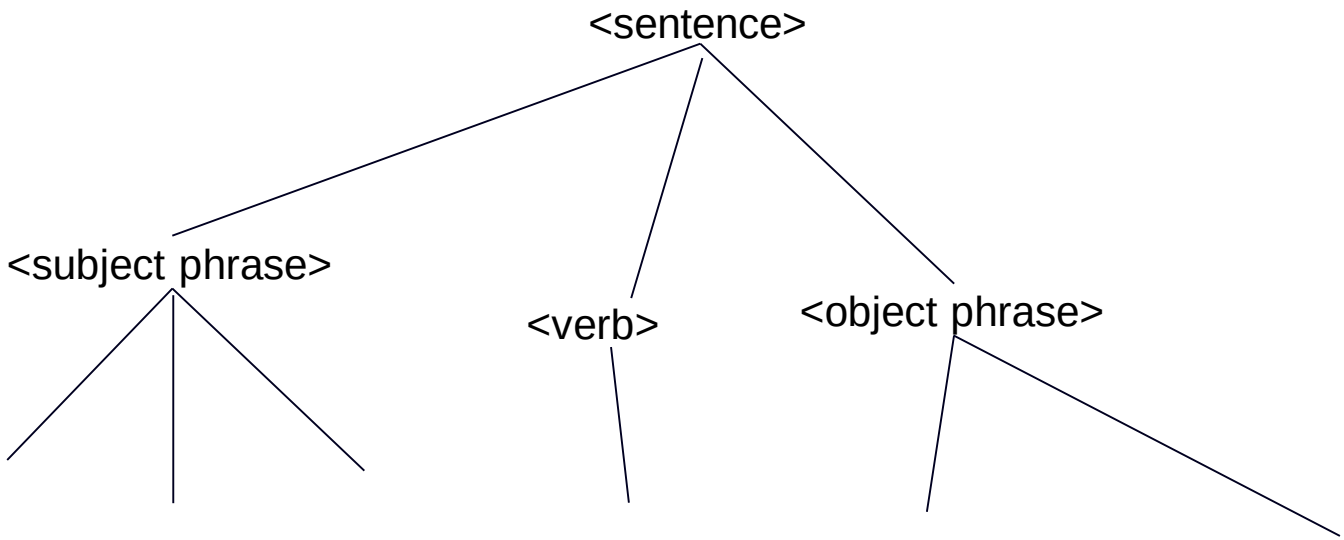
- Primer rečenice: Die laute Frau schimpft das Kind
- Za konstruisanu rečenicu nacrtati odgovarajuće stablo parsiranja

<sentence>

```
<sentence>           -> <subject phrase> <verb> <object phrase>
<subject phrase>     -> <determiner> <adjective> <noun>
<object phrase>      -> <determiner> <adjective> <noun>
<determiner>  -> der | die | das | den
<noun>               -> Mann | Frau | Kind | Hund | Katze
<verb>               -> mag | schimpft | vergisst|
                                 verehrt | verzehrt
<adjective> -> schoene | starke | laute | duenne
```

Primer 2 Stablo parsiranja

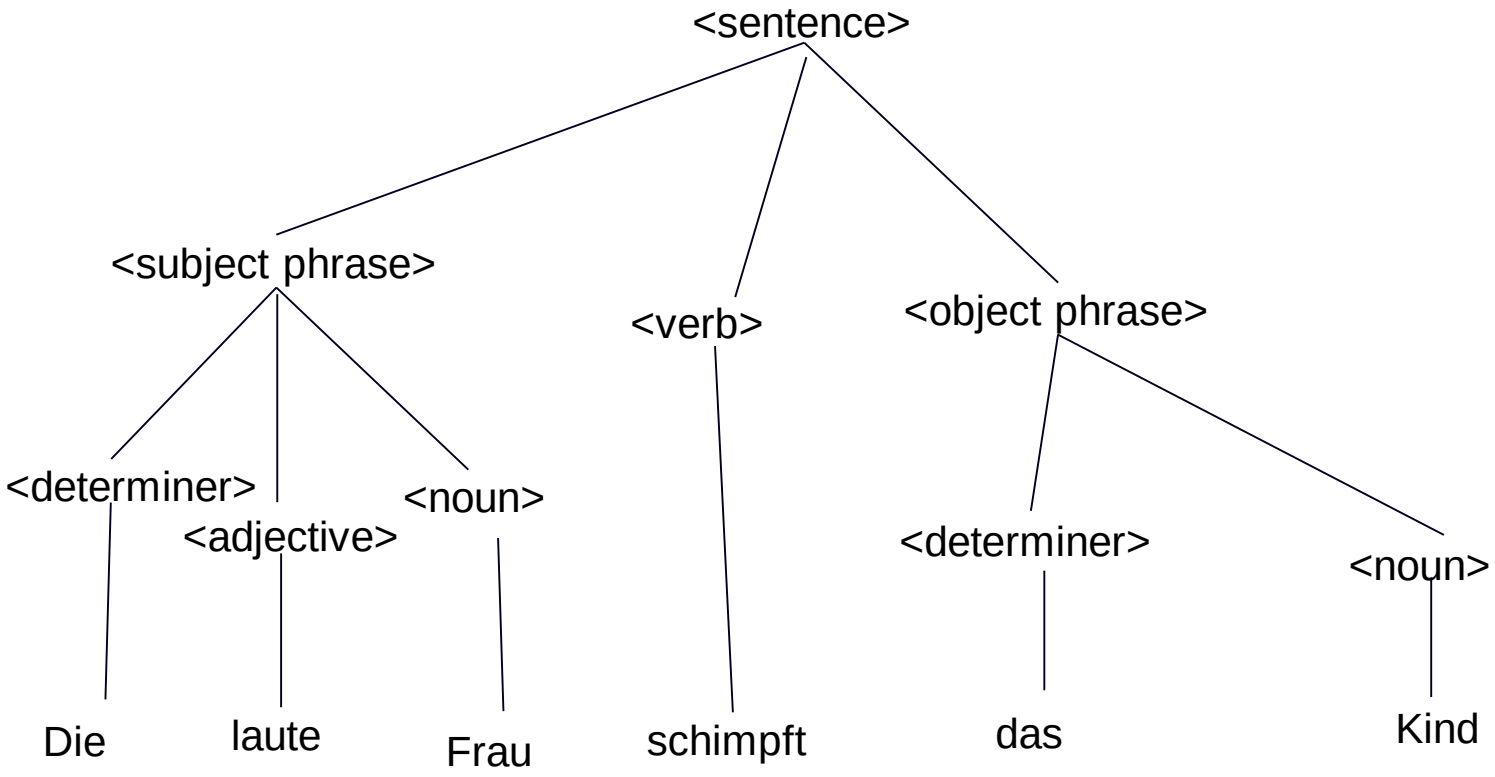
- Primer rečenice: Die laute Frau schimpft das Kind




```
<sentence>        -> <subject phrase> <verb> <object phrase>
<subject phrase>  -> <determiner> <adjective> <noun>
<object phrase>   -> <determiner> <adjective> <noun>
<determiner>      -> der | die | das | den
<noun>            -> Mann | Frau | Kind | Hund | Katze
<verb>            -> mag | schimpft | vergisst| verehrt | verzehrt
<adjective>       -> schoene | starke | laute | duenne
```

Primer 2 Stablo parsiranja

- Primer rečenice: Die laute Frau schimpft das Kind



Zaključivanje sa produkcionim pravilima

- Način korišćenja baze znanja određen je modulom za zaključivanje
  - Osnovni princip produkcionih sistema je da bi **svako pravilo trebalo da bude nezavisan deo znanja** i da **ne bi trebalo da bude svesno drugih pravila**
  - U tom slučaju modul za zaključivanje može jednostavno da aktivira pravilo svaki put kada su premise pravila zadovoljene.

Zaključivanje sa produkcionim pravilima

- Može se koristiti pristup ulančavanje unapred i ulančavanje unazad kroz pravila
- Svaki od sistema (pristupa) ima svoje prednosti i nedostatke i, u suštini, odgovara na različite tipove pitanja
- Primer Mycin:
  - Ulančavanje unapred može da odgovori na pitanje „na šta ovi simptomi ukazuju?"
  - Ulančavanje unazad moglo bi da odgovori na pitanje „da li ovaj pacijent boluje od amnezije?"

Zaključivanje sa produkcionim pravilima

- Postoje dva osnovna pitanja pri implementaciji sistema baziranih na pravilima
  - Kako se implementira **razrešavanje konflikta** (kada više pravila može da se aktivira)?
    - Postoje različite strategije
    - Jedna je da se pravilima dodeli prioritet
  - Kako se činjenice uparuju sa pravilima?
    - Potrebno je uparivanje paterna
  - Ima i puno drugih pitanja koja treba razrešiti

Primer reprezentacije pravila

- R0:
  - IF mortgage is due

    AND checking account has enough money to pay mortgage

    THEN pay mortgage

    AND reduce checking balance by amount of mortgage

Primer reprezentacije pravila

- Prethodno pravilo iskazano je na prirodnom jeziku (engleskom)
- Nama treba mašinski čitljiva reprezentacija
- **Nema standardne sintakse za predstavljanje pravila**
  - Ali postoji W3C standard za razmenu pravila (RIF – Rule Interchange Format) i drugi standardi kao R2ML, URML, RuleML.
- Sintaksa konkretnog pisanja pravila zavisi od shell-a koji se koristi
- Na primer:
  - Leva strana pravila (**uslov**) može da bude **lista** koja sadrži **ime objekta** iza koga slede **parovi atributa i vrednosti** pridruženih tom objektu
  - npr. **(osoba ∧ ime pera ∧ godine 25)**

Primer pravila

- R0:
  - IF (bill $\wedge$ name mortgage $\wedge$ status due $\wedge$ amount 500)

    AND (account $\wedge$ name checking $\wedge$ balance 500)

    THEN (assert (pay $\wedge$ item mortgage))

    AND (remove (bill $\wedge$ name mortgage $\wedge$ status due $\wedge$ amount 500))

    AND (remove (account $\wedge$ name checking $\wedge$ balance 500))

    AND (assert (account $\wedge$ name checking $\wedge$ balance 0))

Primer pravila

- Pretpostavimo da je u radnoj memoriji sledeći sadržaj
- WM:
  - (**bill** $\wedge$ ***name*** **mortgage** $\wedge$ ***status*** **due** $\wedge$ ***amount*** **500)** $\wedge$ (account $\wedge$ *name* checking $\wedge$ *balance* 500)
- Pravilo će biti aktivirano

Primer pravila

- *bill* je objekat i *account* je objekat
- *name*, *status*, *amount* su atributi kojima slede njihove vrednosti
- *remove* i *assert* su primitive koje uklanjaju ili dodaju činjenice u radnu memoriju
- Problem:
  - **Pravilo bi se moralo kreirati za svaki iznos i svaki tip računa**

Primer pravila

- *bill* je objekat i *account* je objekat
- *name*, *status*, *amount* su atributi kojima slede njihove vrednosti
- *remove* i *assert* su primitive koje uklanjaju ili dodaju činjenice u radnu memoriju
- Problem:
  - Pravilo bi se moralo kreirati za svaki iznos i svaki tip računa
- **Rešenje**
  - **Uvode se varijable, operacije i opeatori**

Primer pravila

- R0:
  - IF (bill $\wedge$ name <BILL> $\wedge$ status due $\wedge$ amount <AMOUNT>)

    AND (account $\wedge$ name checking $\wedge$ balance <BALANCE> $\wedge$ (<BALANCE> $\geq$ <AMOUNT>))

    THEN (assert (pay $\wedge$ item <BILL>))

    AND (remove (bill $\wedge$ name <BILL> $\wedge$ status due $\wedge$ amount <AMOUNT>))

    AND (remove (account $\wedge$ name checking $\wedge$ balance <BALANCE>))

    AND (assert (account $\wedge$ name checking $\wedge$ balance (<BALANCE> - <AMOUNT>)))

Primer pravila

- R0:
  - IF (bill $\wedge$ name <BILL> $\wedge$ status due $\wedge$ amount <AMOUNT>)

    AND (account $\wedge$ name checking $\wedge$ balance <BALANCE> $\wedge$ (<BALANCE> $\geq$ <AMOUNT>))

    THEN (assert (pay $\wedge$ item <BILL>))

    AND (remove (bill $\wedge$ name <BILL> $\wedge$ status due $\wedge$ amount <BALANCE>))

    AND (modify(account $\wedge$ name checking $\wedge$ balance (<BALANCE> - <AMOUNT>)))

Primer pravila

- Pretpostavimo da je u radnoj memoriji sledeći sadržaj
- WM:
  - (bill $\wedge$ name electric $\wedge$ status due $\wedge$ amount 100) $\wedge$ (account $\wedge$ name checking $\wedge$ balance 400)
- Pravilo će biti aktivirano
  - Činjenica račun tekući sa stanjem 300 biće dodata u radnu memoriju

Primer pravila

- Pretpostavimo da je u radnoj memoriji sledeći sadržaj
- WM:
  - (bill $\wedge$ name electric $\wedge$ status over_due $\wedge$ amount 100) $\wedge$ (account $\wedge$ name checking $\wedge$ balance 400)
- Šta bi bilo ako bi imali **račun za naplatu** čiji je **status zakasnio za naplatom**
  - Onda prethodno pravilo ne bi bilo zadovoljeno tj. ne bi se dodalo u Agendu
- Problem:
  - Svi uslovi moraju da budu zadovoljeni
- **Rešenje:**
  - **Uvodimo disjunkciju u pravila**

Primer pravila (disjunkcija)

- R0:
  - IF (bill $\wedge$ name <BILL> $\wedge$ status [due over_due] $\wedge$ amount <AMOUNT>)

    AND (account $\wedge$ name checking $\wedge$ balance <BALANCE> $\wedge$ (<BALANCE> $\geq$ <AMOUNT>))

    THEN (assert (pay $\wedge$ item <BILL>))

    AND (remove (bill $\wedge$ name <BILL> $\wedge$ status due $\wedge$ amount <AMOUNT>))

    AND (remove (account $\wedge$ name checking $\wedge$ balance <AMOUNT>))

    AND (assert (account $\wedge$ name checking $\wedge$ balance (<BALANCE> - <AMOUNT>)))

Primer pravila (disjunkcija)

- Pretpostavimo da je u radnoj memoriji sadržaj
  - (bill $\wedge$ name mortgage $\wedge$ status due $\wedge$ amount 700)

    (bill $\wedge$ name electric $\wedge$ status over_due $\wedge$ amount 200)

    (bill $\wedge$ name water $\wedge$ status not_due $\wedge$ amount 50)
    (account $\wedge$ name checking $\wedge$ balance 900)
- Koliko puta će se pravila dodati u agendu?
- Koliko puta će se pravila aktivirati?

Primer pravila (disjunkcija)

- Pretpostavimo da je u radnoj memoriji sadržaj
  - (bill $\wedge$ name mortgage $\wedge$ status due $\wedge$ amount 700)

    (bill $\wedge$ name electric $\wedge$ status over_due $\wedge$ amount 200)

    (bill $\wedge$ name water $\wedge$ status not_due $\wedge$ amount 50)
    (account $\wedge$ name checking $\wedge$ balance 800)
- Koliko puta će se pravila dodati u agendu?
- Koliko puta će se pravila aktivirati?

Uparivanje činjenica sa pravilima

- Kada je definisana sintaksa, mora se specificirati kako će se uraditi uparivanje činjenica sa pravilima
  - t.j. na koji način će modul za zaključivanje **utvrdi da li je uslov pravila zadovoljen činjenicama iz radne memorije**
- Procena je da SBZ bazirani na pravilima **90% vremena** troše na ovaj posao

Rete algoritam uparivanja

- Šta je rešenje za taj problem uparivanja? loše
  - Iscrpno uparivanje činjenica i pravila jedan po jedan
  - Korišćenje tehnika indeksiranja
- Rete algoritam je bio prvo uspešno rešenje
  - On skladišti informacije o uparenostima u mrežnu strukturu

Zaključivanje sa produkcionim pravilima

- Ima još pitanja koja treba razmotriti pri implementaciji mehanizama zaključivanja
  - Kojim redosledom se proveravaju pravila?
  - Kojim redosledom se proveravaju činjenice?
  - Algoritmi mogu da proizvedu i činjenice koje su nevažne za rešavanje postavljenog cilja. Kako to izbeći?
    - Ulančavanje unazad ne pati od ovog problema

Prikladnost produkcionih pravila

Kada je prikladno koristiti produkciona pravila, u kojim situacijama ih koristiti.

- **Izražajnost** pravila
  - Da li se relevantni aspekti domenskog znanja mogu predstaviti (međusobno nezavisnim) pravilima?
- **Računska efikasnost** za rešavanje problema putem pravila
  - Da li je potrebno sračunavanje prihvatljivo?
  - Da li su potrebni proračuni izvodljivi?
- **Lakoća razumevanja** pravila
  - Mogu li ljudi da interpretiraju pravila?
- **Lakoća generisanja** pravila
  - Koliko je teško čoveku da konstruiše pravila koja predstavljaju domensko znanje (koliko je napor potreban za pravljenje pravila) ?

Slučajevi primene produkcionih pravila

- jednostavni domeni
  - dokazivanje teorema, klasifikacija i razlikovanje objekata (tipova voća, drveća i telefonskih stubova, životinjskih vrsta,…), određivanje prostih brojeva
- različita kompleksnost produkcionih pravila
  - Elementarna produkciona pravila
    - Nema hijerarhije, prioriteta izvršavanja, faktora sigurnosti, aritmetike
  - Standardna pravila u ES/SBZ
    - Hijerarhija, prioriteta izvršavanja, faktori sigurnosti, aritmetika (n.pr. CLIPS, Jess)
  - Napredna pravila
    - Proceduralni konstrukti (npr. petlje)
    - Objekti (npr. COOL, Java objekti)
    - Fazi logika (npr. FuzzyCLIPS, FuzzyJ)

Prednosti produkcionih pravila

- Jednostavna i laka za razumevanje
- Moguća direktna računarska implementacija
- Formalne osnove za neke varijante složenijih sistema baziranih na znanju

Nedostaci produkcionih pravila

- Jednostavne implementacije su veoma neefikasne
- Neki tipovi znanja ne mogu se lako izraziti pravilima
- Veliki skupovi pravila su teški za razumevanje i održavanje
  - Kako ćemo pronaći bug u pravilima?

Metode za reprezentaciju znanja (ZR metode)

- Produkciona pravila
- **Stabla, grafovi (n.pr. Semantičke mreže, konceptne mape, konceptni grafovi, ontologija)**
- Šeme i frejmovi
- Logika

=> Rezonovanje

Strukturirani objekti

- Forma pogodna za reprezentaciju strukturiranog znanja
  - Objekti i veze
- Teorija grafova koju koriste:
  - VI
  - Računarska nauka

Graf - podsećanje

- Graf je apstraktni matematički objekat koji se sastoji od tačaka i linija, koje spajaju po dve tačka.
- Za formalnu definiciju grafa oslanjamo se na pojmove iz teorije skupova
  - Neka je X neprazan skup i ρ binarna relacija u X. Uređeni par $G = (X, \rho)$ naziva se graf.
  - Elementi skupa X su čvorovi grafa, a elementi skupa ρ grane grafa.

Graf - podsećanje

- Graf u kom je ***X* beskonačan skup** se zove ***beskonačan graf***.
- Graf u kom je ***X* konačan skup** zove se ***konačan graf***.
  - u predstavljanju znanja se uglavnom koriste konačni grafovi
- Graf u kome je **relacija p orijentisana** zove se **usmeren** *(antisimetričan, orijentisan)* ***graf***.
  - *strelicama na crtežu se označava smer*
  - u predstavljanju znanja se uglavnom koriste usmereni grafovi.
- Graf u kome je **relacija *ρ* simetrična** zove se ***neusmeren*** *(simetričan, neorjentisan)* ***graf***.
  - najčešće je moguće eksplicitno navesti da je relacija simetrična
  - nema strelica na crtežu

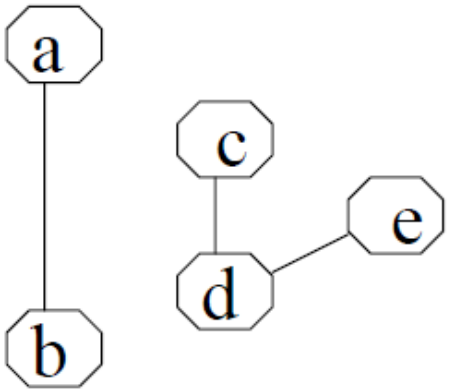
Graf - podsećanje

- Tipično, elementi skupa *X* su međusobno različiti, pa nije potrebno naglašavati da su čvorovi označeni
- Ako su u grafu oznečene i grane, a ne samo čvorovi, onda je u pitanju ***označen graf*** (labeled)
  - U predstavljanju znanja uglavnom se koriste označeni grafovi

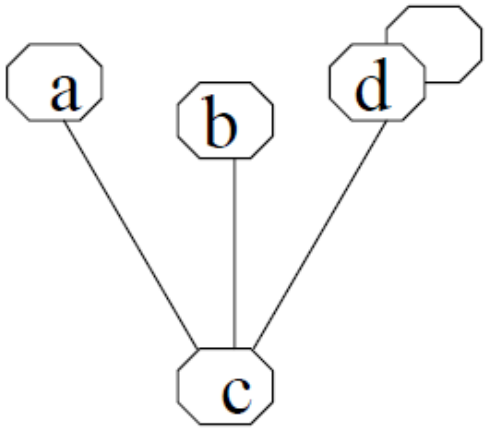
Grafovi, stabla i mreže

- Semantička mreža
- Konceptna mapa
  - nema nasleđivanja
- Ontologije
  - Deskriptivna logika

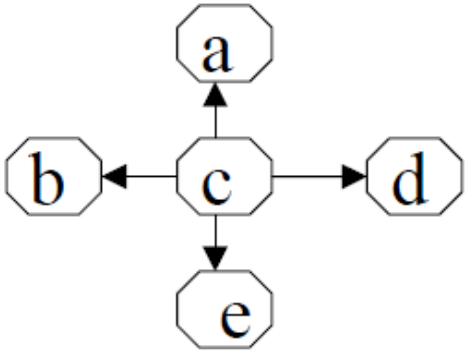
Grafovi, stabla i mreže (1)



(A)



(B)

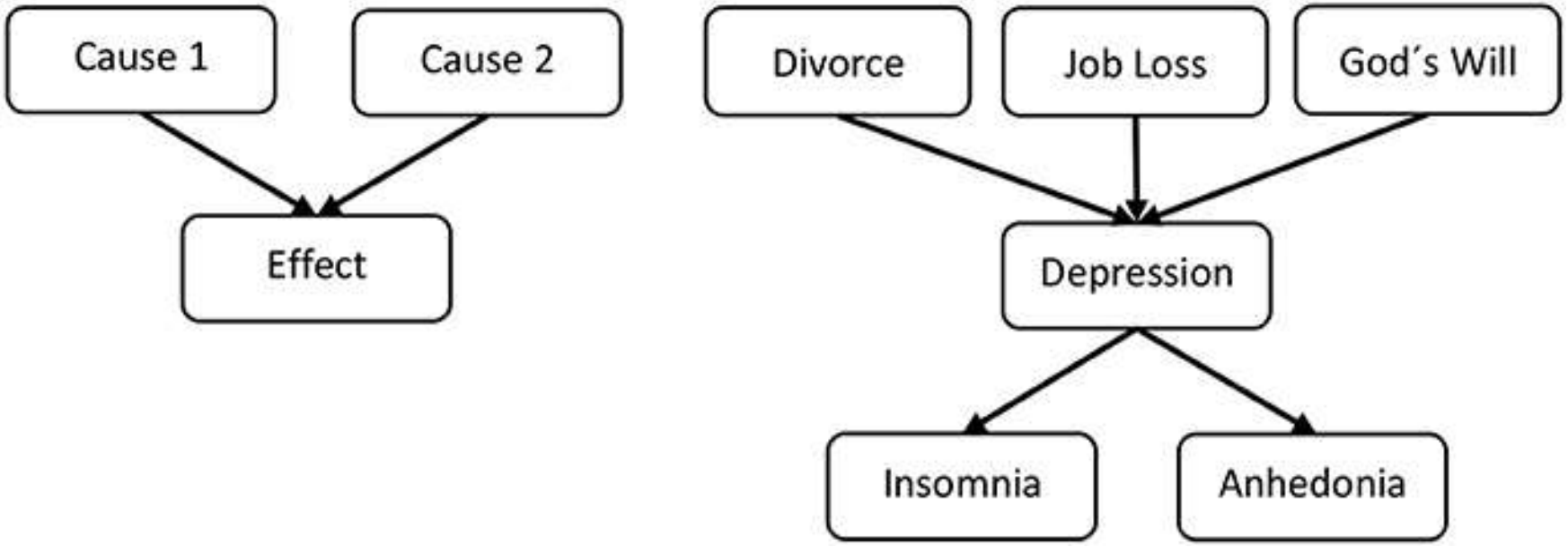


(C)

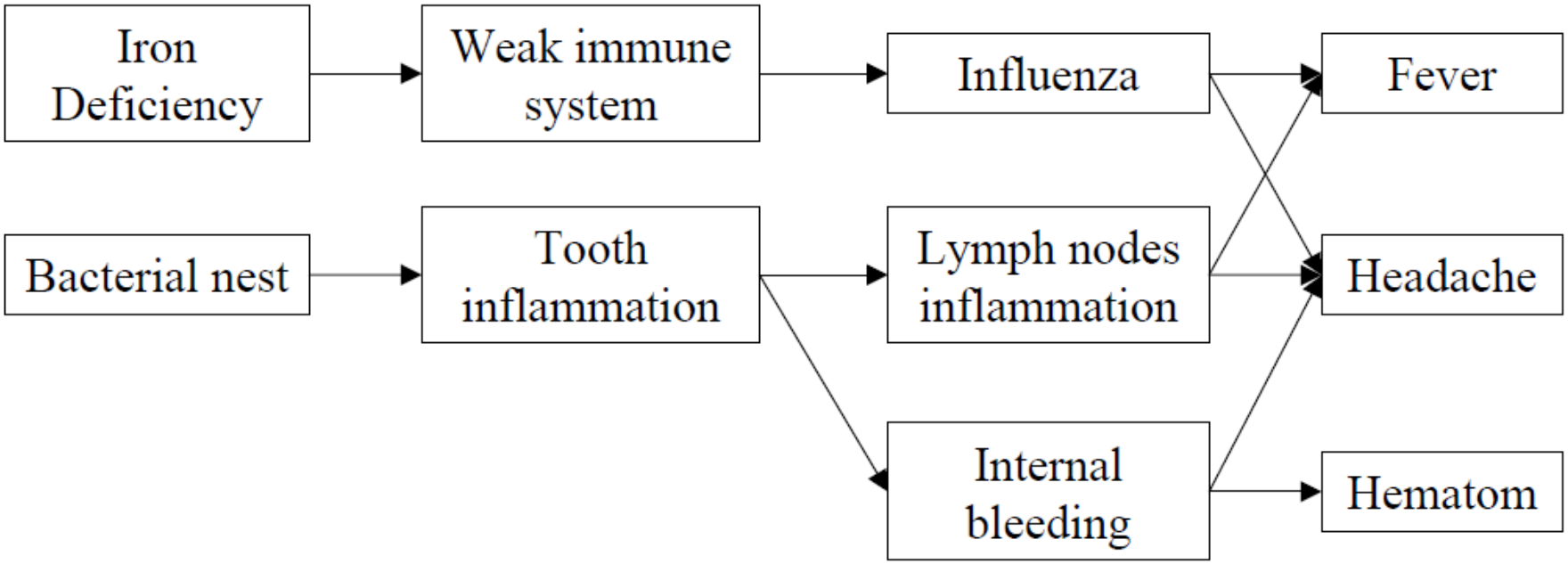
Grafovi, stabla i mreže (2)

- Relacije
  - prostorne, vremenske, kauzalne
- Stabla
  - hijerarhija, klasifikacija
- Semantička mreža
  - koncepti i relacije

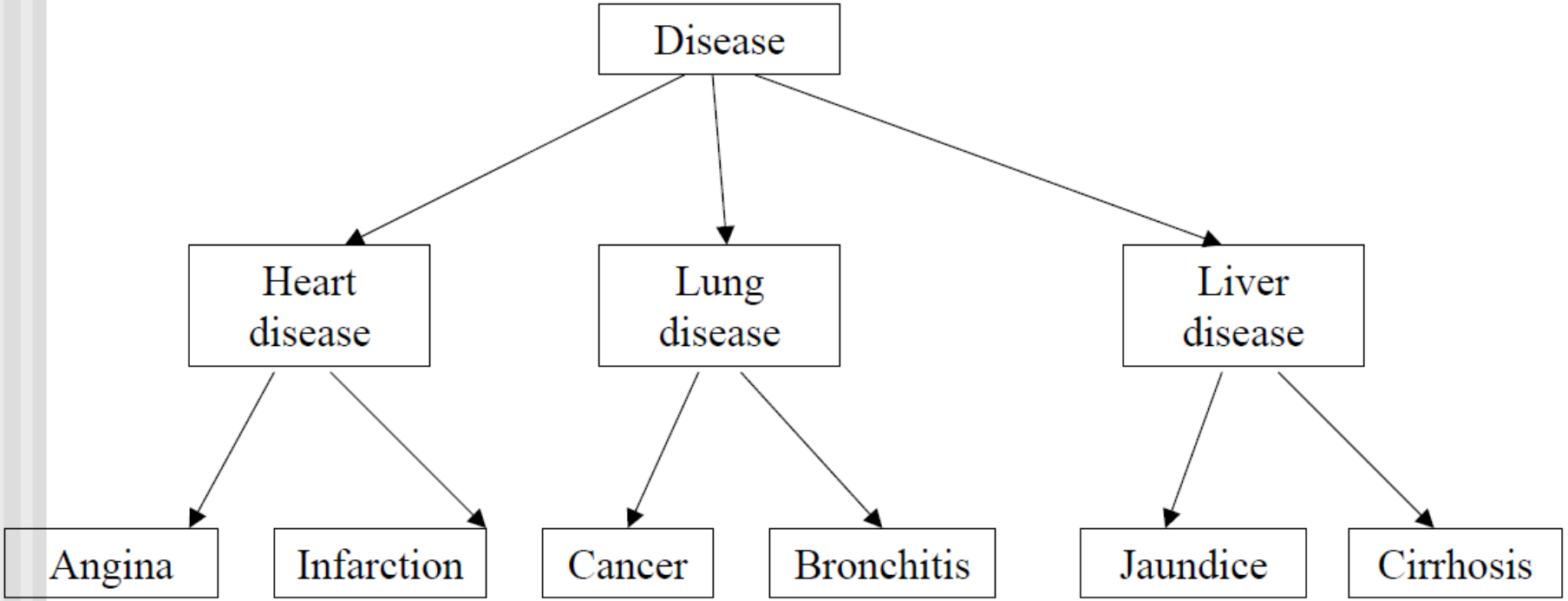
Kauzalna mreža na jednostavnom grafu



Kauzalna mreža na jednostavnom grafu



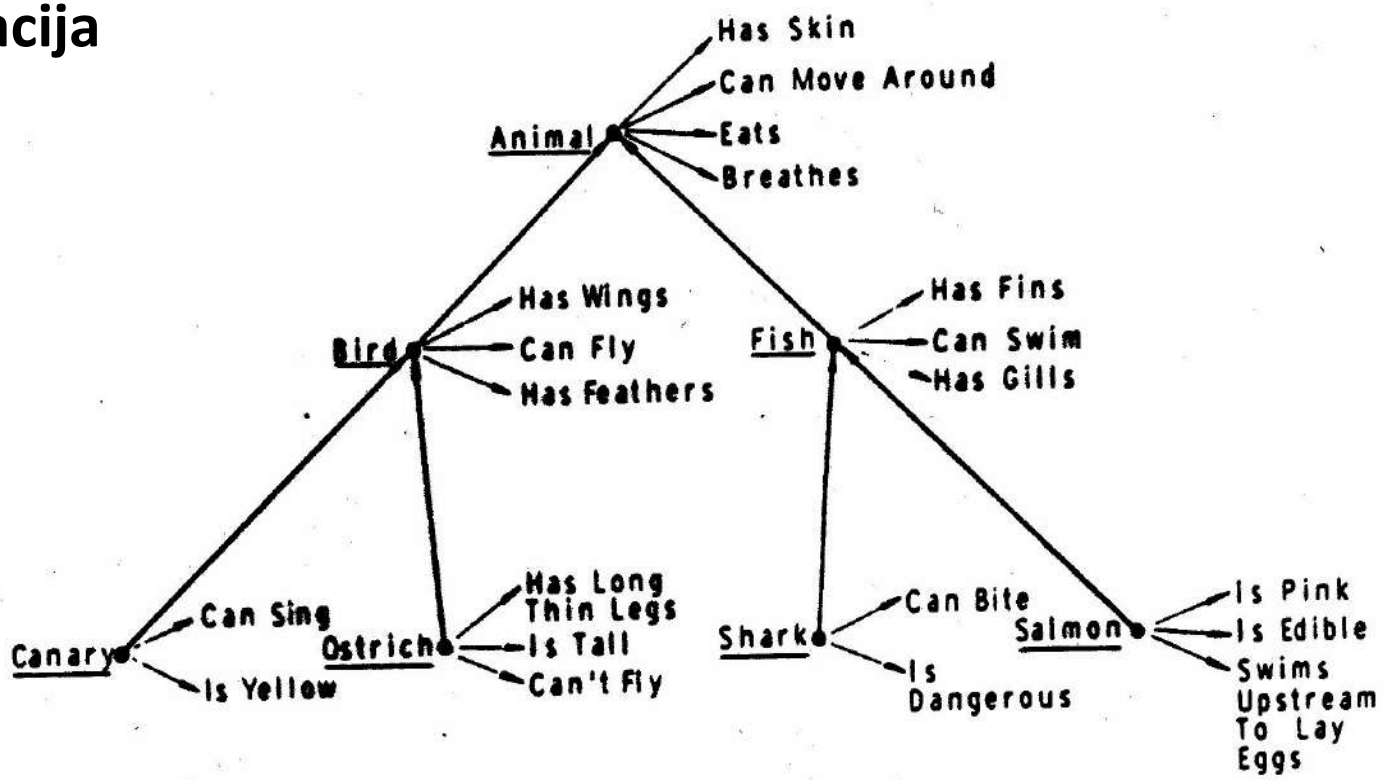
Hijerarhijska klasifikacija na stablu



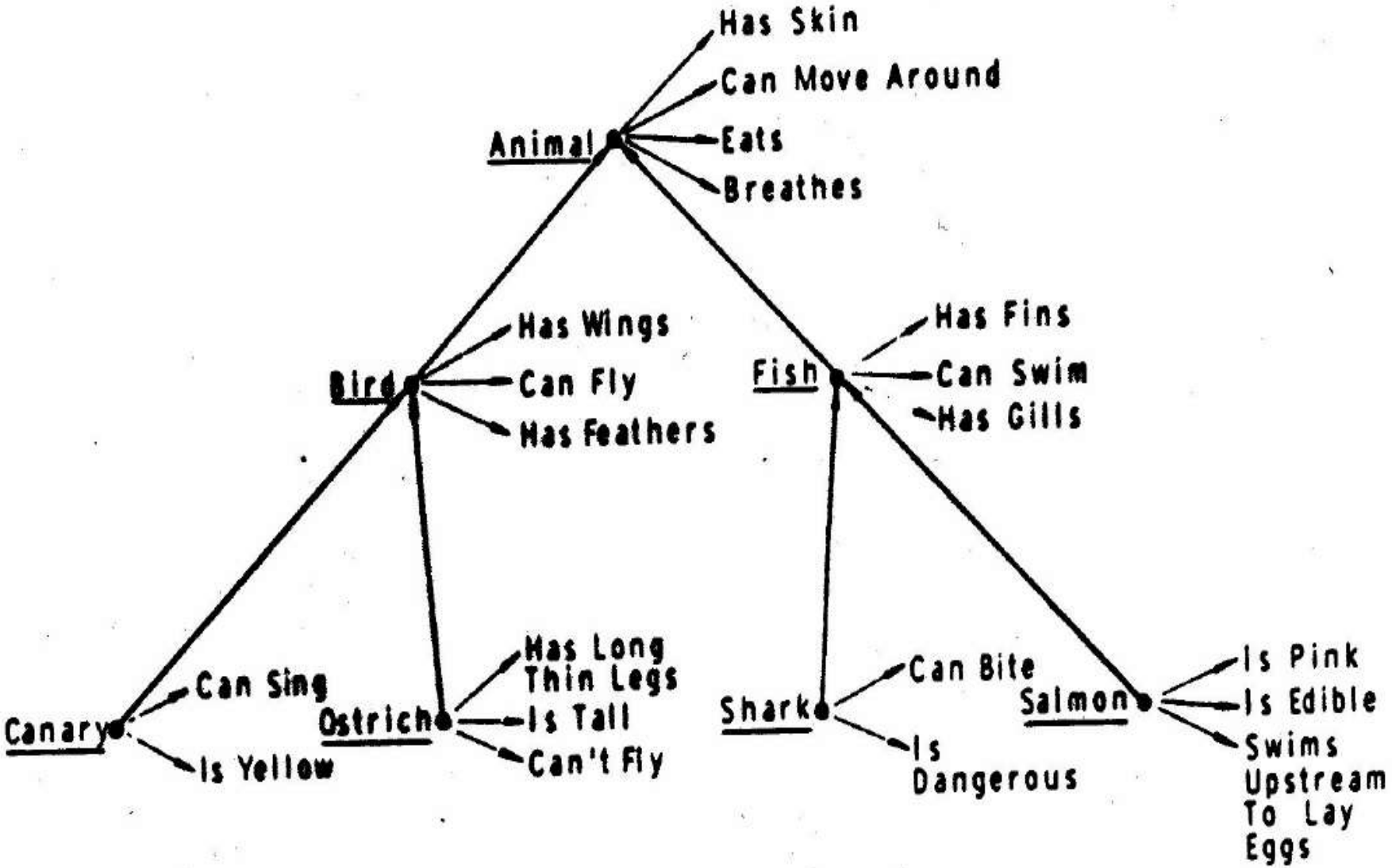
Semantička mreža (Quillian, 1968)

Koristi se za prikaz koncepta i relacija između koncepata
Semantička mreža predstavlja **označen usmeren graf**

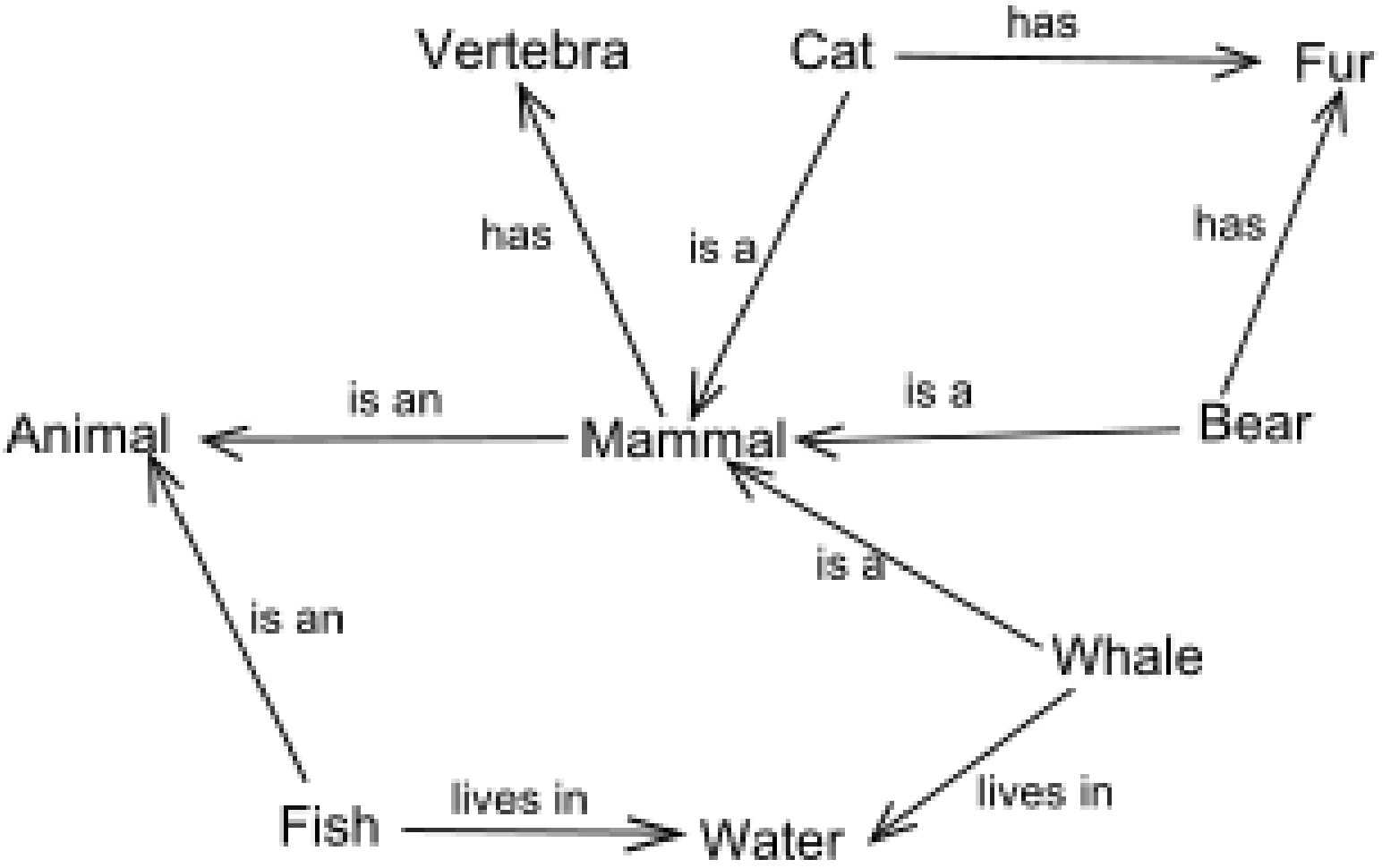
Predstavlja grafičku reprezentaciju **iskaznih (propozicionih) informacija**



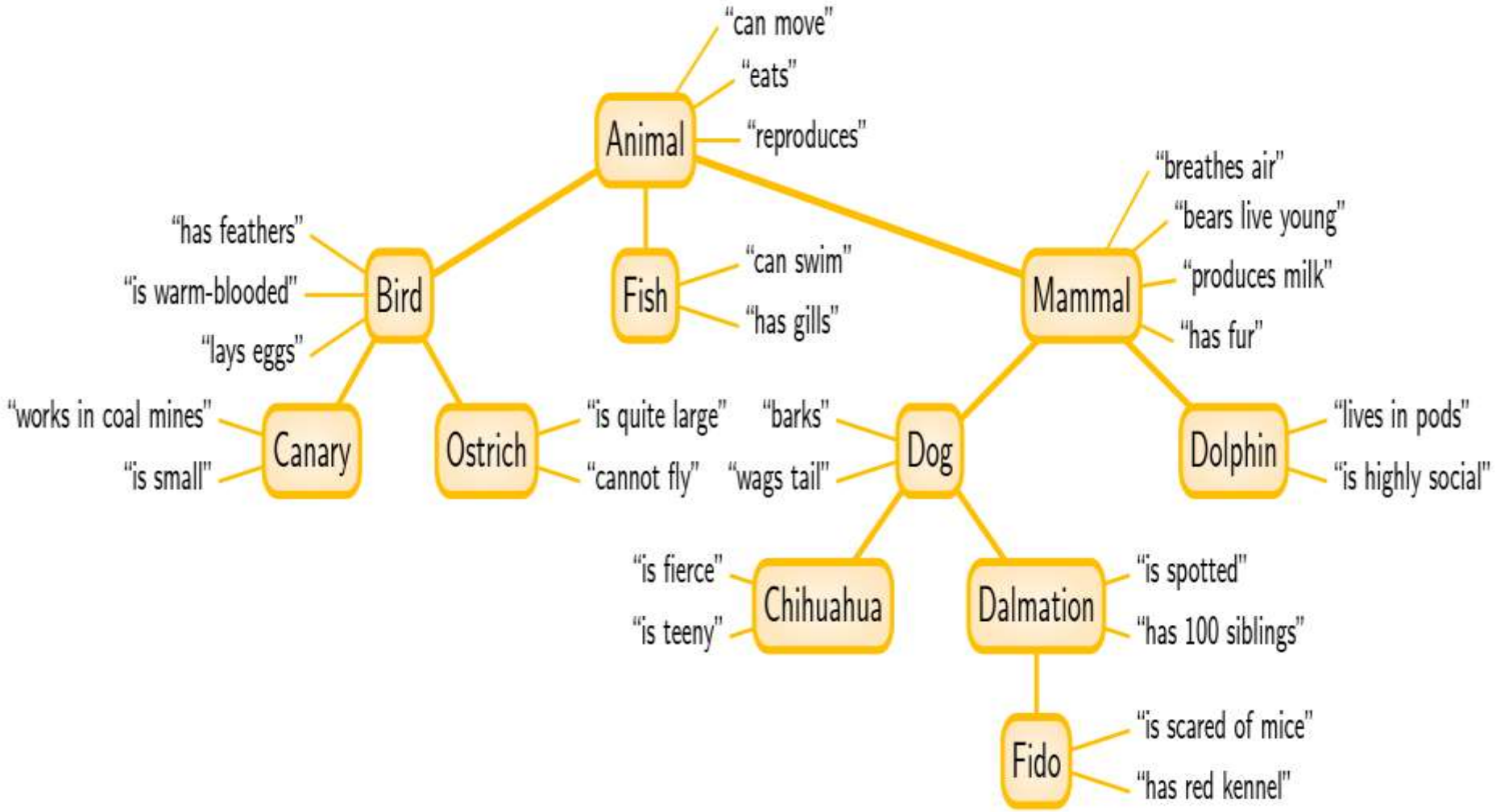
Semantička mreža (Quillian, 1968)



Semantička mreža



Hijerarhijska semantička mreža



Semantičke mreže

- **Grafička reprezentacija iskaznih (propozicionih) informacija**
- Razvio ih je M. R. Quillian – model **ljudske memorije** koji se sastoji od ***entiteta*** i ***relacija*** među tim entitetima.
- Semantička mreža je jednostavna reprezentaciona šema koja za predstavljanje znanja koristi **graf označenih čvorova** i **označenih usmerenih lukova**.
  - Obično se koristi za predstavljanje **statičkih**, **taksonomskih rečnika koncepata**
  - Ili drugih znanja koja se sastoje od **koncepata** i **relacija među konceptima**
- Bile su vrlo popularne ’60tih i ’70tih godina prošlog veka, danas se manje koriste.
  - Ali se na njima baziraju brojne savremene tehnike reprezentacije znanja
  - Često jeftinije od drugih formalizama reprezentacije znanja
- **Grafički prikaz** pridružen semantičkoj mreži je važan razlog njihove popularnosti.

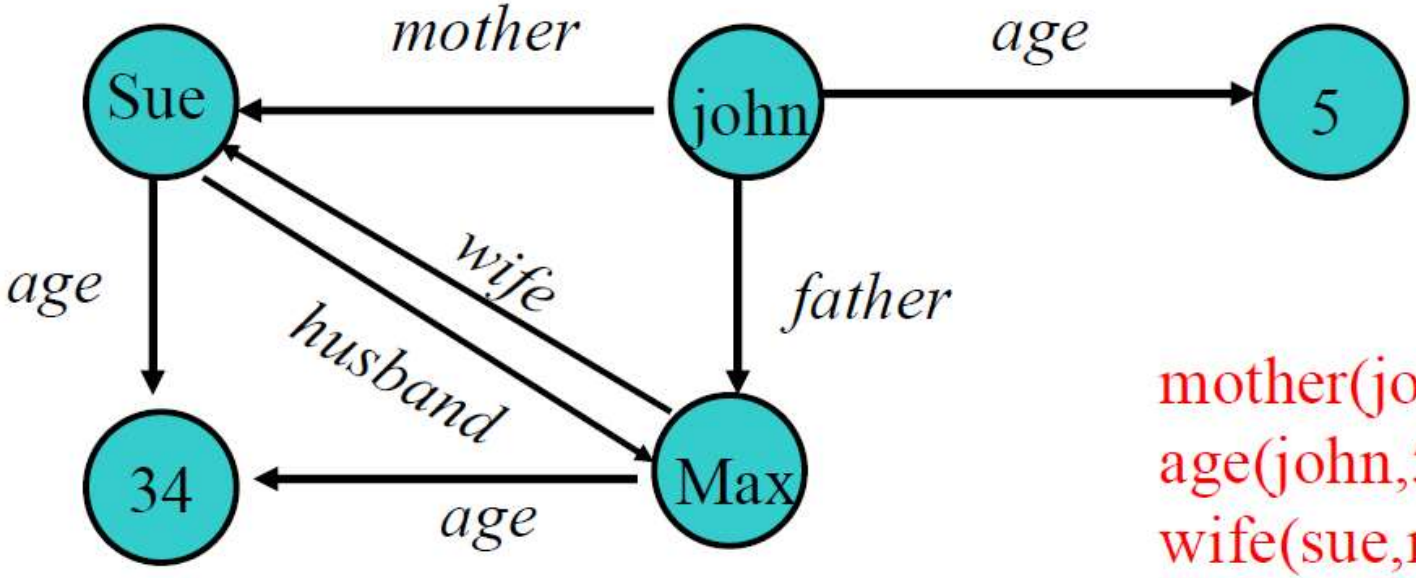
Semantičke mreže

- Semantičke mreže se obično koriste u sprezi sa **skupom pristupnih procedura koje obavljaju "rezonovanje"**
  - n.pr., nasleđivanje vrednosti i relacija
- Imale su veliki uticaj na razvoj **ontologija**, koje su danas **najrasprostanjeniji metod reprezenacije znanja**

Semantičke mreže

- Predstavlja se kao **označeni** (označuju se čvorovi i veze) **usmereni** (zna se šta je polazni a šta odredišni čvor) graf:
  - čvorovi = entiteti (čvor se označava imenom)
  - linkovi = relacije (link se označava tipom)
- **Čvorovima** se predstavljaju **koncepti, objekti** ili **situacije**
  - Labele su imena
  - Čvorovi mogu da budu **instance** (individualni objekti) ili **klase** (generički čvorovi)
- **Linkovi** predstavljaju **veze**
  - Veze sadrže **strukturne informacije** o znanju koje se predstavlja
  - **Labele** ukazuju na **tip relacije**
- **Tripleti objekt-atribut-vrednost** (OAV)
  - Za karakterizaciju znanja u semantičkoj mreži
  - Nastali od asocijativnih lista (iz LISPa)
  - imaju dominantni položaj u reprezentaciji znanja na Webu: asocijativne liste kroz JSON, OAV kroz RDF

Čvorovi i lukovi (veze)



mother(john,sue)
age(john,5)
wife(sue,max)
age(max,34)
...

OAV tripleti

- objekt-atribut-vrednost tripleti
  - Mogu se koristiti za karakterizaciju znanja u semantičkoj mreži
  - Brzo dvode do ogromnih tabela

Objekat	Atribut	Vrednost
Astérix	profession	warrior
Obélix	size	extra large
Idéfix	size	petite
Panoramix	wisdom	infinite

Objekti i atributi

- Atributima se izražavaju dodatne informacije o čvorovima u semantičkoj mreži
  - Često se izražavaju kao ***svojstva***
    - Kombinacija atributa i vrednosti
  - Atribut može biti izražen i kao relacija
    - n.pr. has-attribute

Primer: Semantix Net







Abraracourcix

Astérix

is-boss-of

is-boss-of

Cétautomatix

is-a

is-friend-of

is-a

is-a

buys-from

fights-with

Obélix

Gaul

is-a

Panoramix

is-a



AKO

Dog

sells-to

is-a

Human

lives-with

is-a

takes-care-of



barks-at

Idéfix



Ordralfabetix



[http://www.asterix.tm.fr]

Objašnjenje semantičke mreže

- **Boje čvorova** na grafiku predstavljaju **različite tipove individualnih objekata ili klasa**
- **Font labela relacija** implicira **različite tipove veze**
- Relacija **AKO** (a-kind-of) se koristi da predstavi vezu između klasa i predstavlja specijalnu relacija nasleđivanja (subclass to superclass)
  - Nepotrebno je povezati individualne objekte ljudi sa klasom Human
- Relacija **IS-A** se koristi da predstavi vezu između instance klase sa klasom (individual node to generic node)

Primer: Semantix Net







Abraracourcix

Astérix

is-boss-of

is-boss-of

Cétautomatix

is-a

is-friend-of

is-a

is-a

buys-from

fights-with

Obélix

Gaul

is-a

Panoramix

is-a



e.g.

Dog

sells-to

is-a

Human

lives-with

is-a

takes-care-of



barks-at

Idéfix

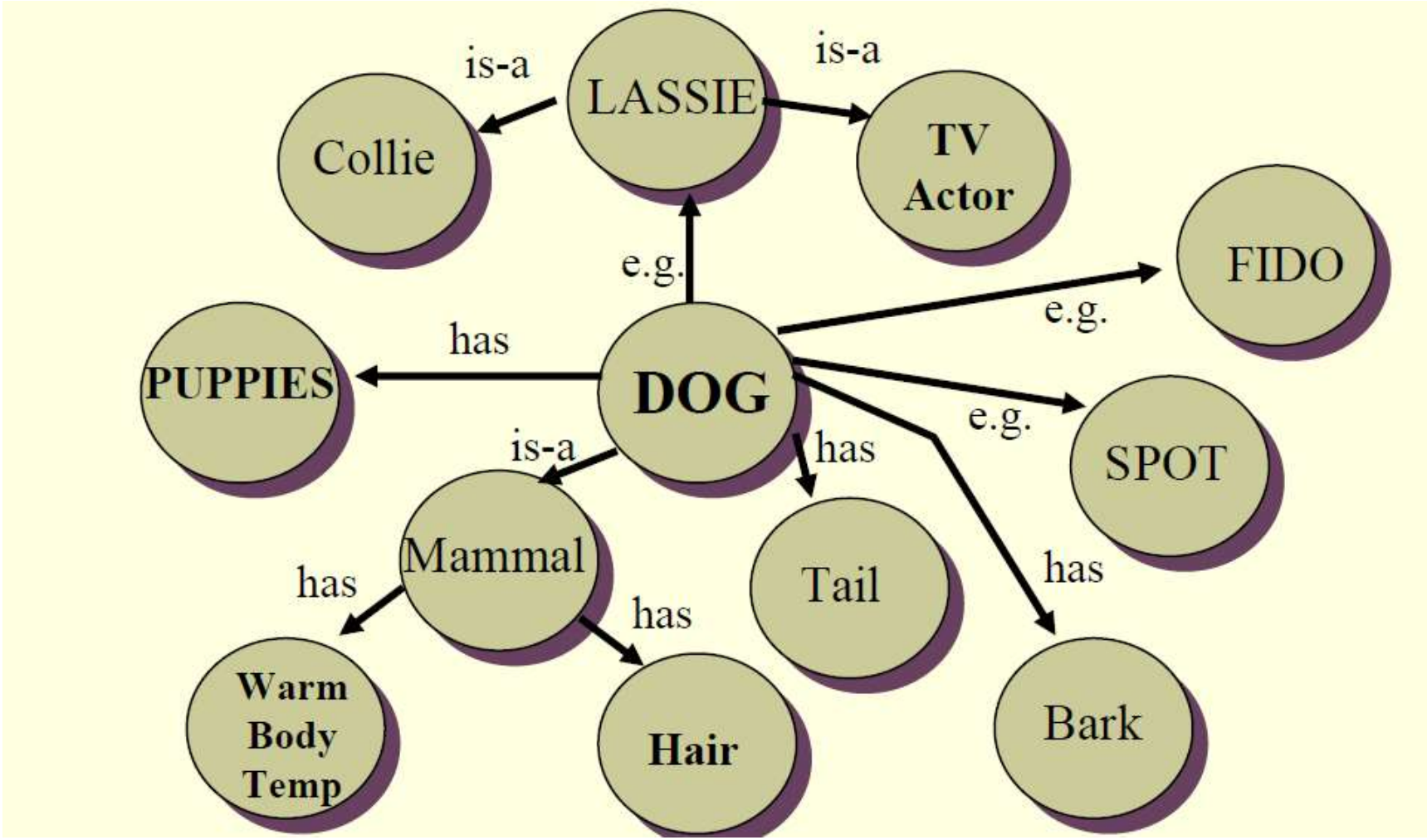


Ordralfabetix

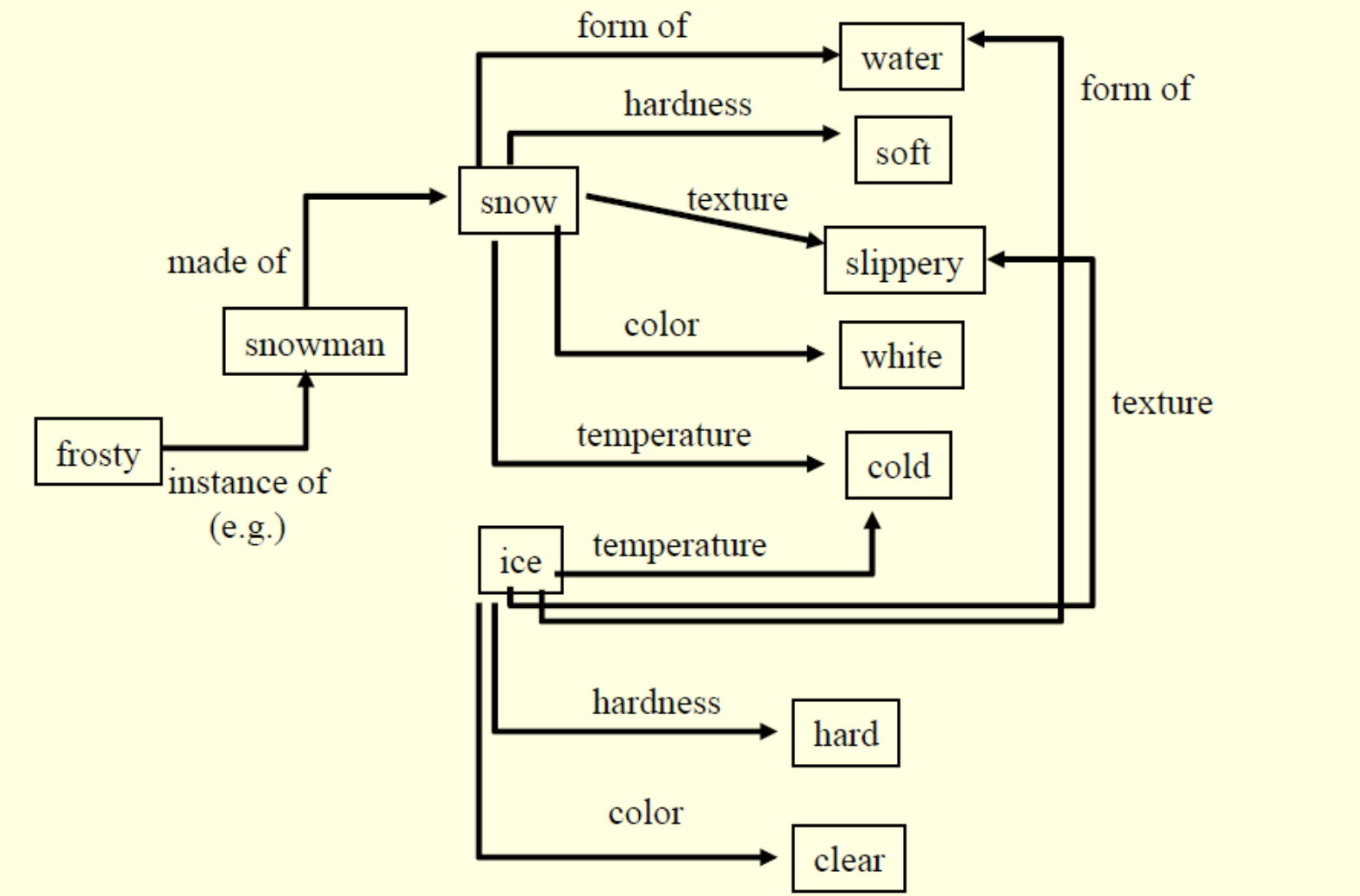


[http://www.asterix.tm.fr]

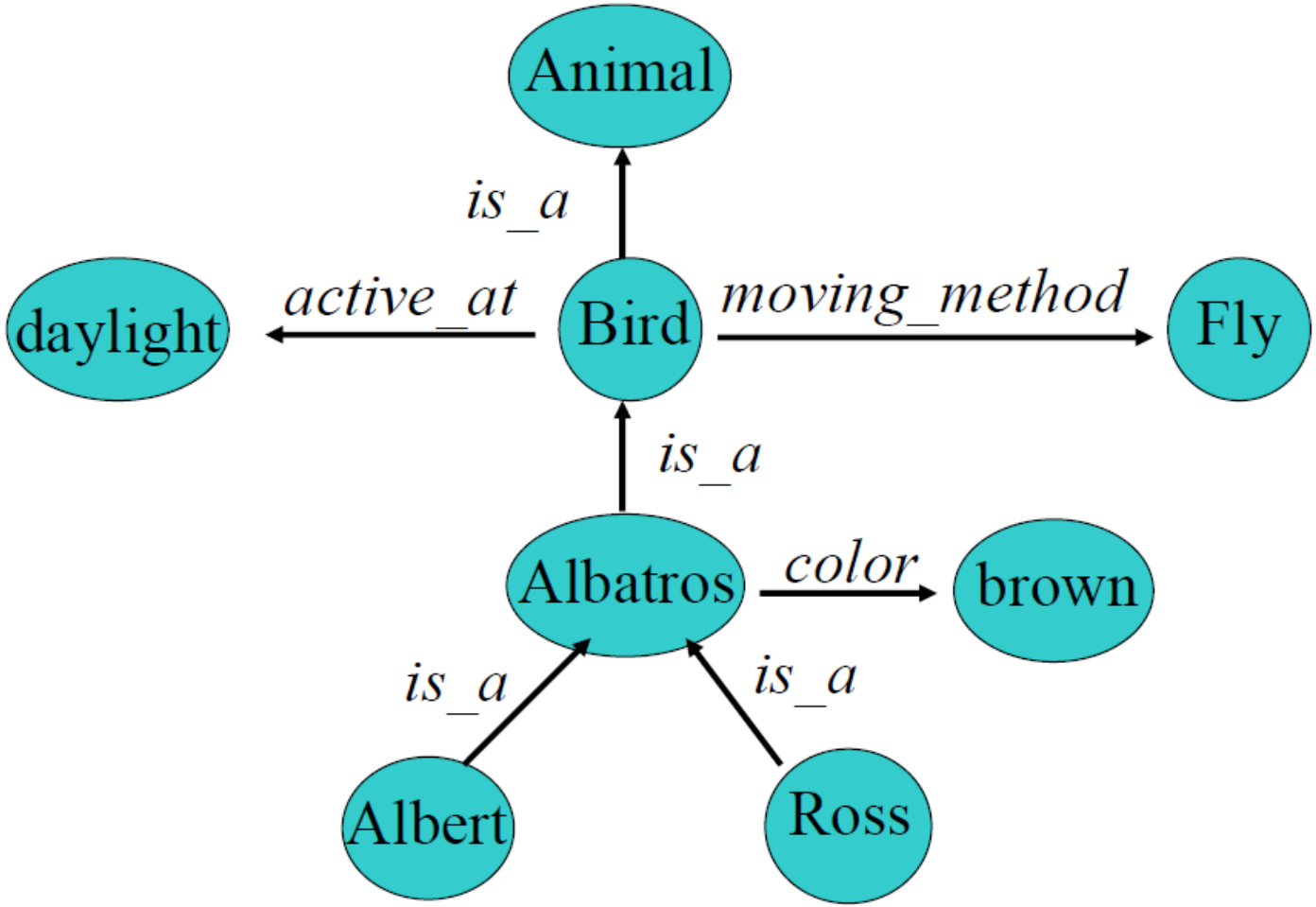
Semantička mreža o psima



Semantička mreža o snegu i ledu



Primer semantičke mreže (1)



Primer semantičke mreže (2)

⇒ ptica **is a** životinja

⇒ letenje **is a** način kretanja ptica

⇒ albatros **is a** ptica

⇒ Albert **is a** albatros

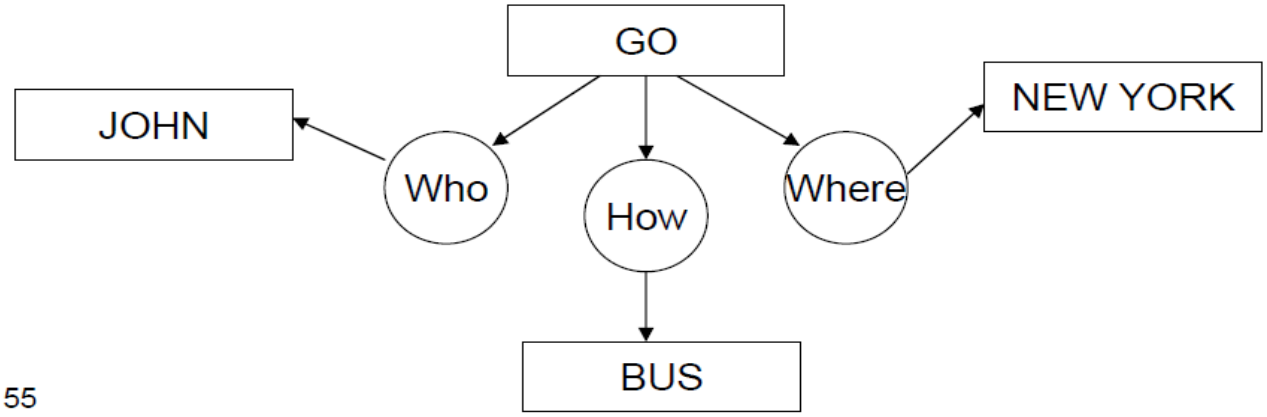
⇒ Ross **is a** albatros

- *'is_a'* se koristi da predstavi:
  - Relaciju *klase* objekata i *superklase* klase (animal je superklasa od bird);
  - Relaciju *instance* klase i *klase* (Ross je instanca klase albatros).

Natsavak naredni cas

Konceptni grafovi

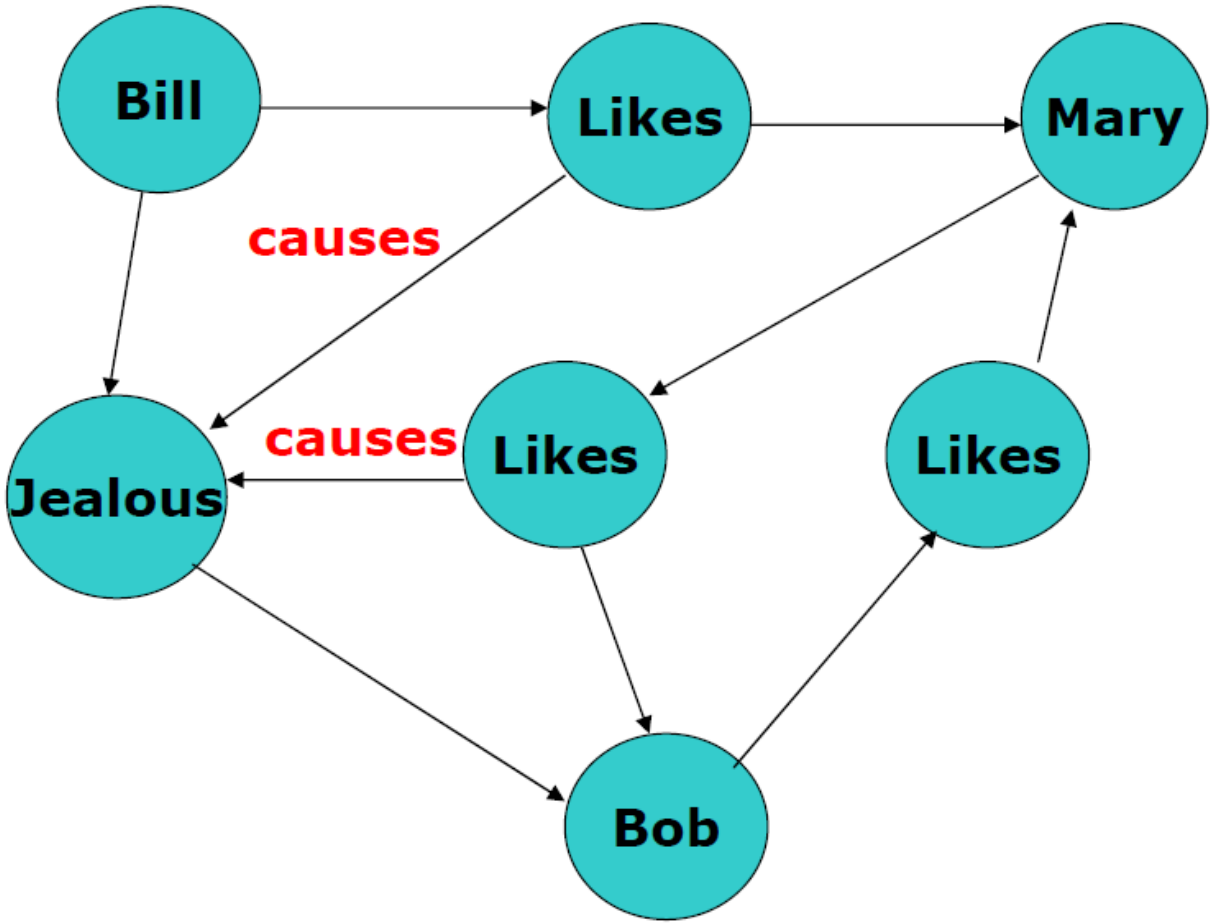
- *Konceptni grafovi* su vrsta **semantičkih mreža** za predstavljanje značenja (jednostavnih) rečenica prirodnog jezika
- Dva tipa čvorova:
  - *Konceptni čvorovi:* postoje dva tipa koncepata, individualni koncepti i generički koncepti
  - *Relacioni čvorovi:* **binarne** relacije među konceptima



Kauzalne relacije

- Kauzalna relacija “jealous” kao primer
  - *Bob* likes *Mary*
  - *Bill* likes *Mary*
  - *Mary* likes *Bob*
  - (causes (*Mary* likes *Bob*) (*Bill* to be jealous of *Bob*))
  - (causes (*Bill* likes *Mary*) (*Bill* to be jealous of *Bob*))

Kauzalna semantička mreža



Relacije

- Bez relacija, znanje je neuređena kolekcija činjenica
  - Rasuđivanje o tim činjenicama nije od velikog interesa
    - **Moguće je induktivno rasuđivanje** (od konkretnog ka opštem)
- **Relacije izražavaju strukturu kolekcije činjenica**
  - **To omogućuje generisanje novog smislenog znanja**
    - Generisanje novih činjenica
    - Generisanje novih relacija

Tipovi relacija

- Često korišćene relacije
  - IS-A (typeOf)
    - Povezuje instancu sa klasom
  - AKO (a-kind-of, subClassOf)
    - Povezuje klasu (podklasu) sa drugom klasom(superklasa)
  - HAS, has-a
    - Deo prema celini (inverzno od part-of)
    - Atribut objekta ili klase

Tipovi relacija i rezonovanje

- Relacije se mogu (formalno gledano) proizvoljno definisati
  - Omogućena je velika fleksibilnost ALI....
  - ALI za rezonovanje, mehanizam rezonovanja mora da zna kako se relacije mogu iskoristiti za generisanje novog znanja
    - metode zaključivanja se moraju specificirati za svaku relaciju
    - Osnovna ideja tehnologija za reprezentovanje znanja u **Semantičkom Webu** je da se **definiše skup relacija sa predefinisanom semantikom** (relacija za koje je hardkodovano kako će se rezonovati kada se koriste), a da se **semantika iskaza izvodi iz ovih predefinisanih konstrukata**. Time je omogućeno reprezentovanje implicitnog znanja.

OWL DL semantika

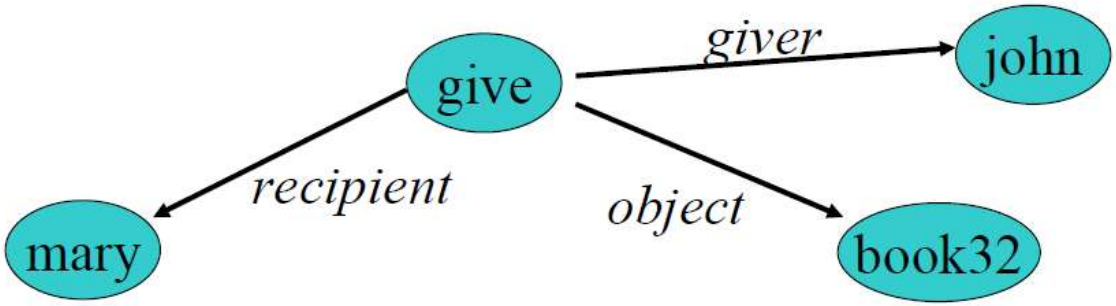
Abstract Syntax	DL Syntax	Semantics
Classes		
Class(A partial $C_1 \ldots C_n$)	$A \sqsubseteq C_1 \sqcap \ldots \sqcap C_n$	$A^{\mathcal{I}} \subseteq C_1^{\mathcal{I}} \cap \ldots \cap C_n^{\mathcal{I}}$
Class(A complete $C_1 \ldots C_n$)	$A \equiv C_1 \sqcap \ldots \sqcap C_n$	$A^{\mathcal{I}} = C_1^{\mathcal{I}} \cap \ldots \cap C_n^{\mathcal{I}}$
EnuneratedClass(A $o_1 \ldots o_n$)	$A \equiv \{o_1, \ldots, o_n\}$	$A^{\mathcal{I}} = \{o_1^{\mathcal{I}}, \ldots, o_n^{\mathcal{I}}\}$
SubClassOf(C_1 C_2)	$C_1 \sqsubseteq C_2$	$C_1^{\mathcal{I}} \subseteq C_2^{\mathcal{I}}$
EquivalentClasses($C_1 \ldots C_n$)	$C_1 \equiv \ldots \equiv C_n$	$C_1^{\mathcal{I}} = \ldots = C_n^{\mathcal{I}}$
DisjointClasses($C_1 \ldots C_n$)	$C_i \sqcap C_j = \bot, i \neq j$	$C_i^{\mathcal{I}} \cap C_j^{\mathcal{I}} = \emptyset, i \neq j$
Datatype(D)		$D \subset \Delta_{\mathcal{D}}^{\mathcal{I}}$
Datatype Properties		
DatatypeProperty(		
U super(U_1) ... super(U_n)	$U \sqsubseteq U_i$	$U^{\mathcal{I}} \subseteq U_i^{\mathcal{I}}$
domain(C_1) ... domain(C_m)	$\geqslant 1\, U \sqsubseteq C_i$	$U^{\mathcal{I}} \subseteq C_i^{\mathcal{I}} \times \Delta_{\mathcal{D}}^{\mathcal{I}}$
range(D_1) ... range(D_l)	$\top \sqsubseteq \forall U.D_i$	$U^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}} \times D_i^{\mathcal{I}}$
[Functional])	$\top \sqsubseteq \leqslant 1 U$	U_i is functional
SubPropertyOf(U_1 U_2)	$U_1 \sqsubseteq U_2$	$U_1^{\mathcal{I}} \subseteq U_2^{\mathcal{I}}$
EquivalentProperties($U_1 \ldots U_n$)	$U_1 \equiv \ldots \equiv U_n$	$U_1^{\mathcal{I}} = \ldots = U_n^{\mathcal{I}}$
Object Properties		
ObjectProperty(		
R super(R_1) ... super(R_n)	$R \sqsubseteq R_i$	$R^{\mathcal{I}} \subseteq R_i^{\mathcal{I}}$
domain(C_1) ... domain(C_m)	$\geqslant 1\, R \sqsubseteq C_i$	$R^{\mathcal{I}} \subseteq C_i^{\mathcal{I}} \times \Delta_{\mathcal{D}}^{\mathcal{I}}$
range(C_1) ... range(C_l)	$\top \sqsubseteq \forall R.C_i$	$R^{\mathcal{I}} \subseteq \Delta^{\mathcal{I}} \times C_i^{\mathcal{I}}$
[inverseOf(R_0)]	$R \equiv (R_0^-)$	$R^{\mathcal{I}} = (R_0^{\mathcal{I}})^-$
[Symmetric]	$R \equiv (R^-)$	$R^{\mathcal{I}} = (R^{\mathcal{I}})^-$
[Functional]	$\top \sqsubseteq \leqslant 1 R$	$R^{\mathcal{I}}$ is functional
[InverseFunctional]	$\top \sqsubseteq \leqslant 1 R^-$	$(R^{\mathcal{I}})^-$ is functional
[Transitive])	$Tr(R)$	$R^{\mathcal{I}} = (R^{\mathcal{I}})^+$
SubPropertyOf(R_1 R_2)	$R_1 \sqsubseteq R_2$	$R_1^{\mathcal{I}} \subseteq R_2^{\mathcal{I}}$
EquivalentProperties($R_1 \ldots R_n$)	$R_1 \equiv \ldots \equiv R_n$	$R_1^{\mathcal{I}} = \ldots = R_n^{\mathcal{I}}$
Annotation		
AnnotationProperty(S)		
Individuals		
Individual(		
o type(C_1) ... type(C_n)	$o \in C_i$	$o^{\mathcal{I}} \in C_i^{\mathcal{I}}$
value(R_1 o_1) ... value(R_n o_n)	$\{o, o_i\} \in R_i$	$\{o^{\mathcal{I}}, o_i^{\mathcal{I}}\} \in R_i^{\mathcal{I}}$
value(U_1 v_1) ... value(U_n v_n))	$\{o, v_i\} \in U_i$	$\{o^{\mathcal{I}}, v_i^{\mathcal{I}}\} \in U_i^{\mathcal{I}}$
SameIndividual($o_1 \ldots o_n$)	$o_1 = \ldots = o_n$	$o_1^{\mathcal{I}} = \ldots = o_n^{\mathcal{I}}$
DifferentIndividual($o_1 \ldots o_n$)	$o_i \neq o_j, i \neq j$	$o_i^{\mathcal{I}} \neq o_j^{\mathcal{I}}, i \neq j$

Predstavljanje nebinarnih relacija

- Lako se vidi kako se binarne relacije mogu predstaviti putem semantičke mreže
- Ali šta je sa nebinarnim relacijama?
  - "*give (john,mary,book32)*"
    - Realcija "give" je ternarna
    - Ona uključuje davaoca, primaoca i objekat
  - "*Mike is John's and Jim's* ***advisor****.* ***He*** *advised* ***John*** *to take a* ***statistics course*** *and* ***Jim*** *a* ***physics course***"
    - Relacija "advisor" ternarna
    - Ona uključuje savetnika, studenta i kurs

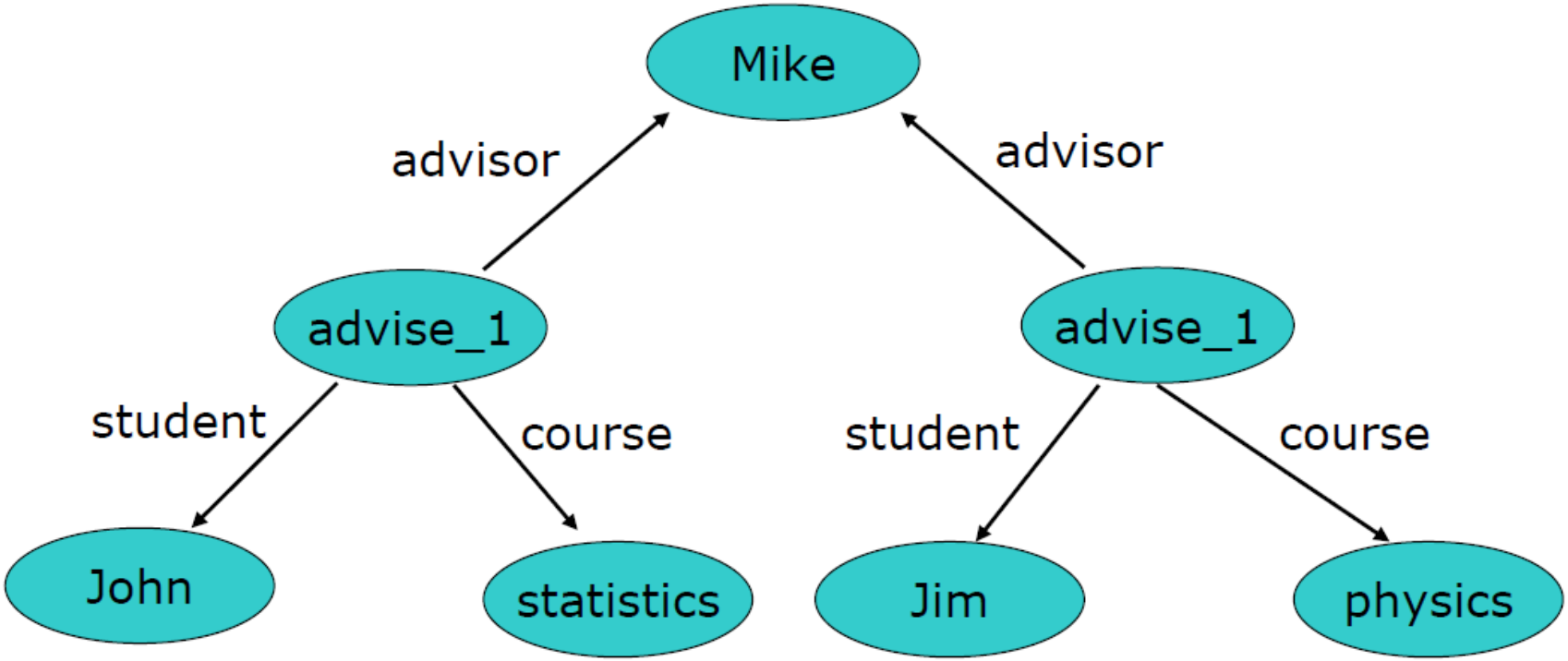
Predstavljanje nebinarnih relacija

- Problem se rešava tako što se “relacija pretvori u objekat”
- To je primer nečega što logičari zovu “reifikacija” (res, rei (lat.) – stvar)
  - reifikuj : smatraj apstraktan koncept za realan



- Posmatramo konkretnu instancu davanja knjige

Predstavljanje nebinarnih relacija



Posmatramo instance davanja saveta (advise_1 i advise_2)

Semantičke mreže u Prologu

- Znanje predstavljeno semantičkom mrežom može se lako konvertovati u Prolog program, na primer:

```
isa(hawk,bird)
has_property(bird,fly)
has_property(x,z) :- chain_isa(x,y),has_property(y,z)
chain_isa(x,y) :- isa(x,y),!.
chain_isa(x,y) :- isa(x,w),chain_isa(w,y)
```

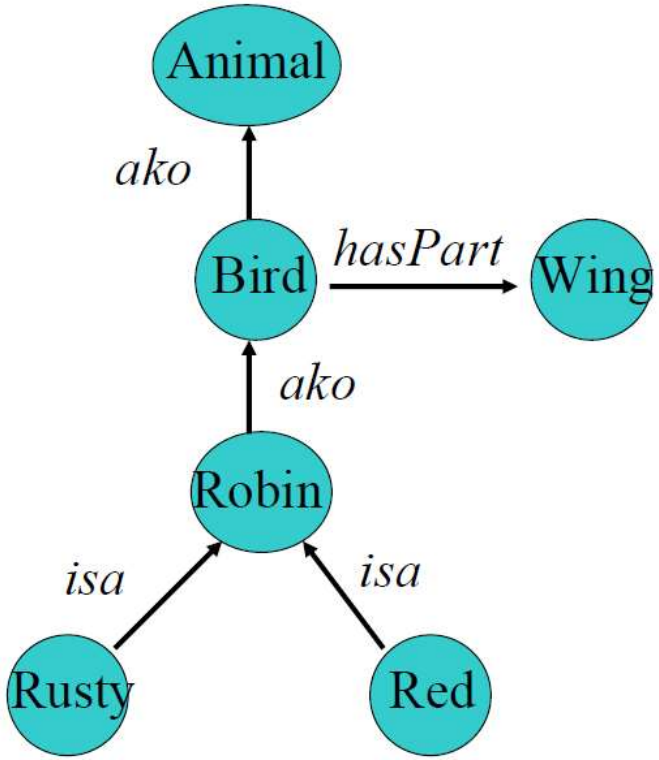
Semantičke mreže u LISPu

- … ali i u LISP

```
(defun *database* ()
'((canary (is-a bird)
   (color yellow)
   (size small))
(penguin (is-a bird)
   (movement swim))
(bird (is-a vertebrate)
   (has-part wings)
   (reproduction egg-laying))))
```

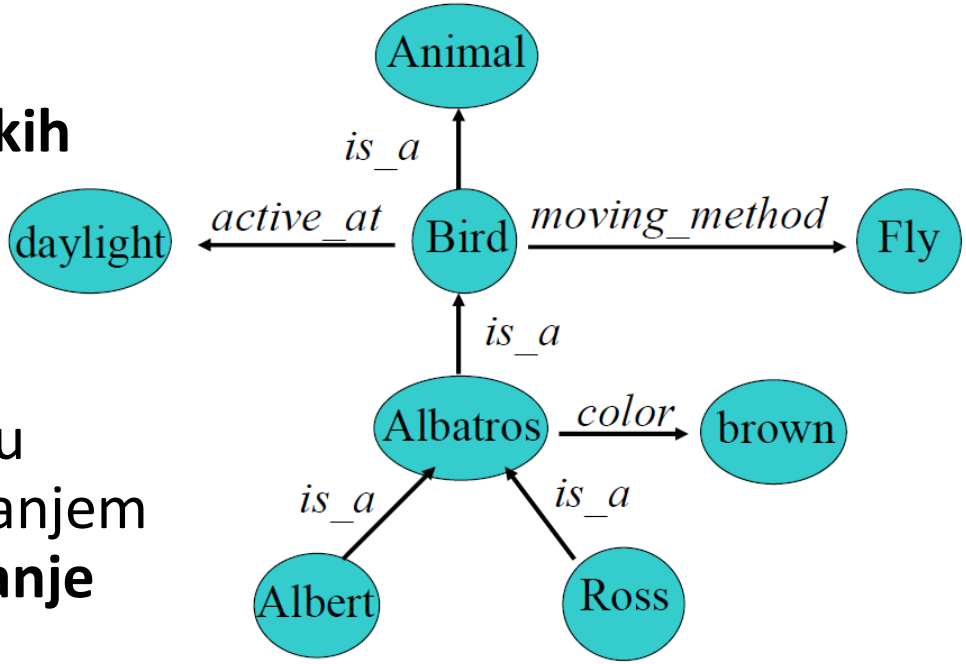
Individuali i klase

- Mnoge semantičke mreže razlikuju:
  - Čvorove koji predstavljaju individuale i one koji predstavljaju klase
  - Razlikuje se “ako” relacija od “is-a” relacije



Individuali i klase

- Druge koriste IS-A i za klase i za instance
  - To i jeste **problem semantičkih mreža**
  - **Nema standardne definicije imena linka**
  - Problem je delimično rešen u **Semanitčkom Webu** definisanjem **familije jezika za predstavljanje znanja** koji se baziraju na **semantičkim mrežama i uvedeni su standardi za predstavljanje ontologija**

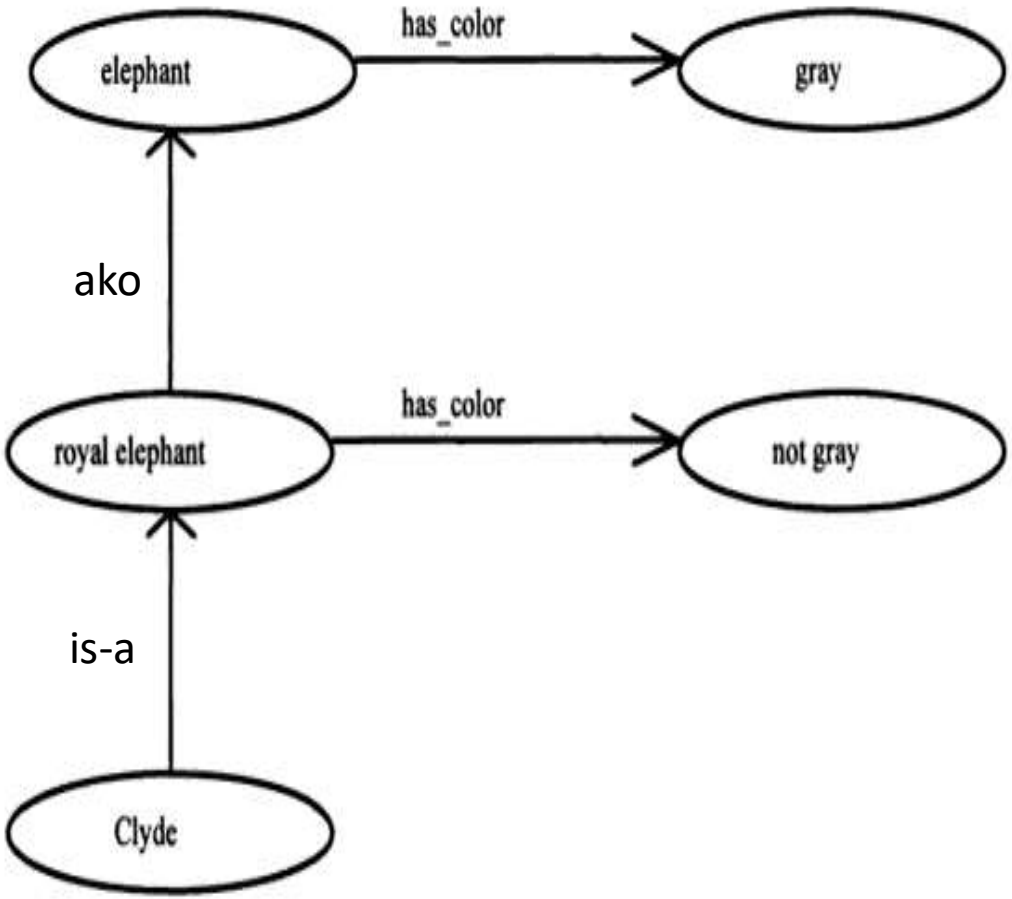


Zaključivanje nasleđivanjem

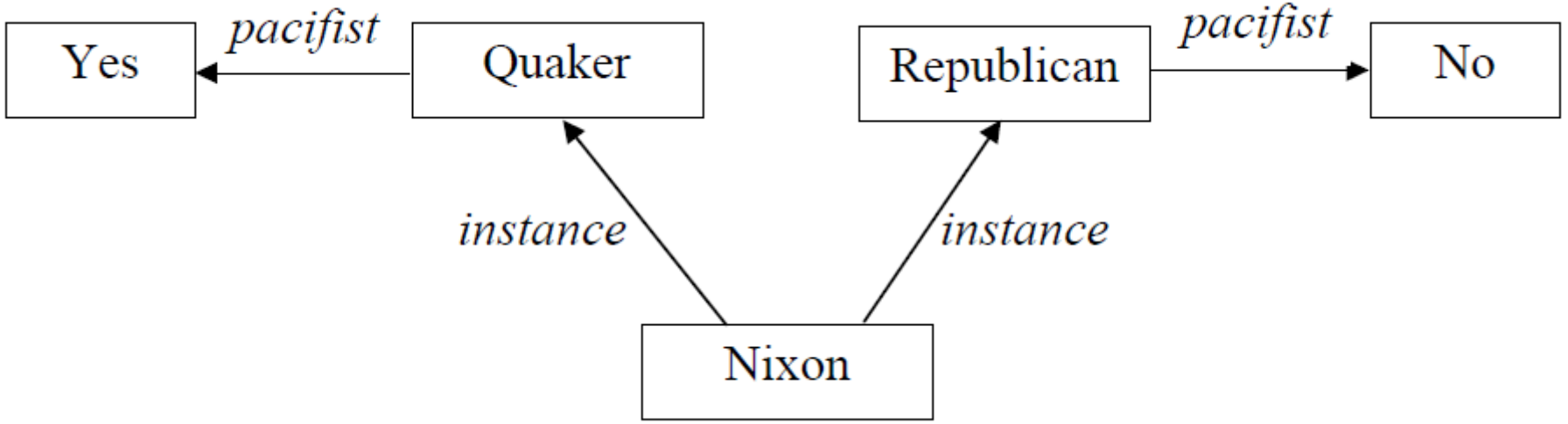
- Jedan od **glavnih načina rezonovanja** u semantičkim mrežama je **nasleđivanje vrednosti duž linkova podklasa i instanci**.
- Nasleđivanje nam omogućuje da predstavimo default vrednosti u semantičkoj mreži
- Semantičke mreže se **razlikuju po načinu nasleđivanja višestrukih različitih vrednosti**. Rešenja su da se:
  - Nasleđuju se **sve moguće vrednosti**, *ili*
  - Nasleđuje se **samo "najniža" vrednost** ili **vrednosti**
  - Ne nasledi **ništa** ili napiše **konkretno pravilo** za semantičku mrežu koja vrednost da se nasledi

nasleđivanje višestrukih različitih vrednosti – problem Izuzeci

- Većina slonova je siva. Kraljevski slonovi su slonovi. Većina kraljevskih slonova nije siva. Clyde je kraljevski slon.
- Ideja slična kao u OOP (i logici uopšte) – nasleđivanje nam omogućuje da novu klasu definišemo preko najbližeg roda (*genus proximum*) i vrsne razlike (*diferentia specifica*)

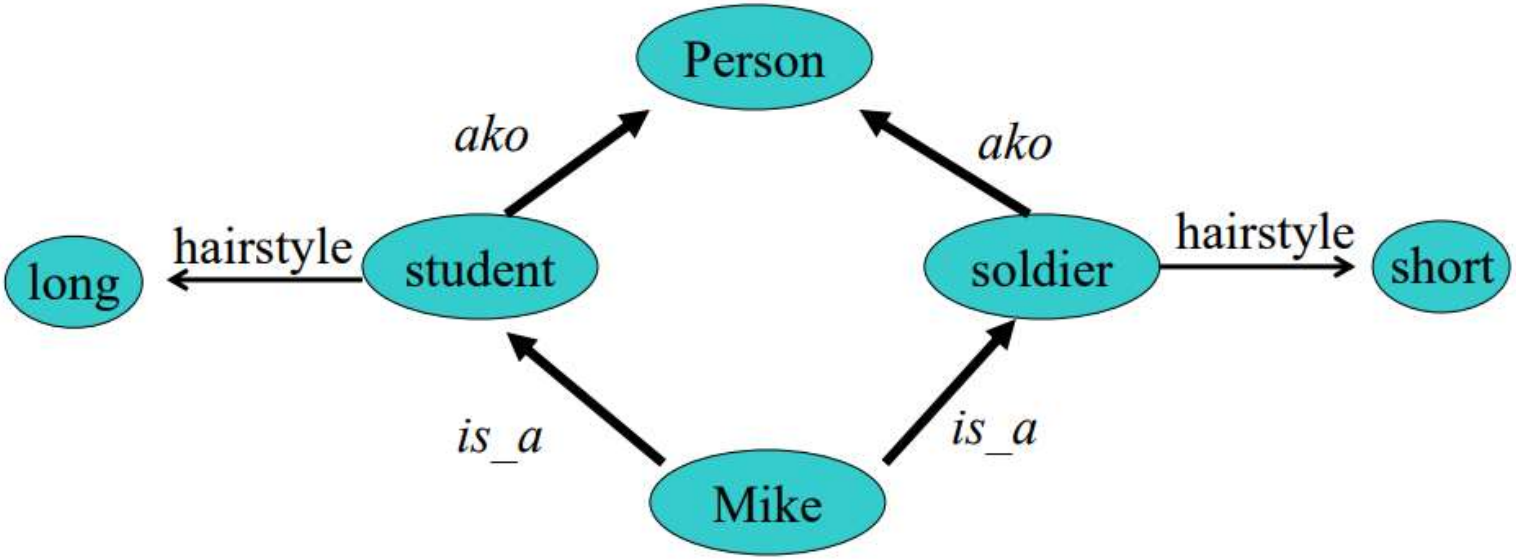


nasleđivanje višestrukih različitih vrednosti – problem Konfliktne nasleđene vrednosti



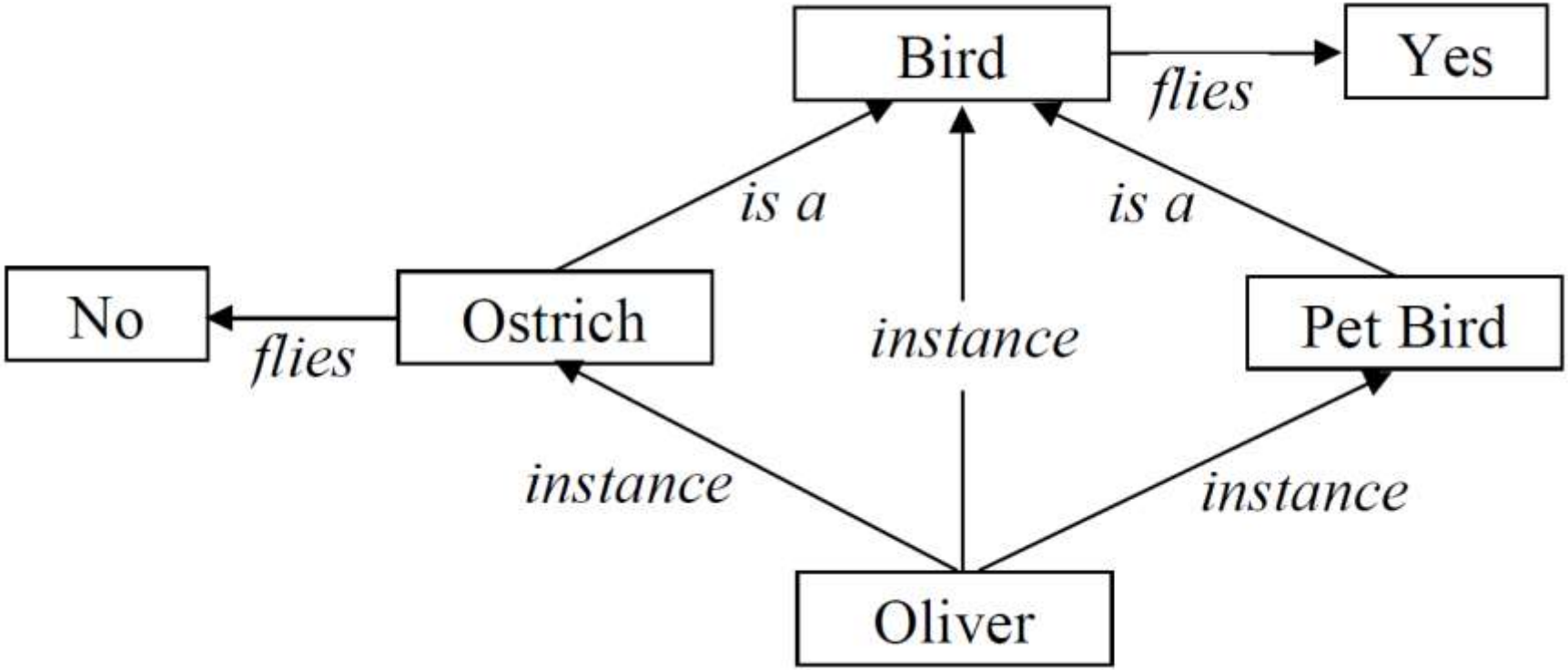
nasleđivanje višestrukih različitih vrednosti – problem Konfliktne nasleđene vrednosti

- Mike je student i vojnik. Većina studenata ima kratku dužu kosu, dok većina vojnika ima kratku kosu.



nasleđivanje višestrukih vrednosti – problem Tangled Hierarchies

- Nasleđivanja koja nisu prosta stabla:



Višestruko nasleđivanje

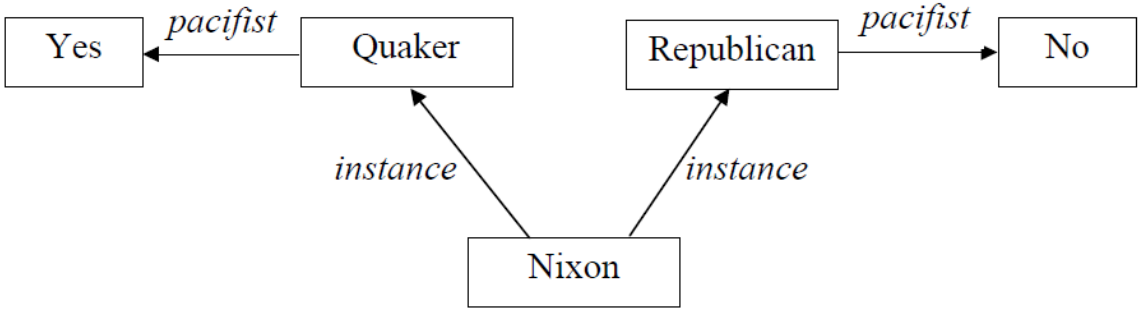
- Čvor može da ima proizvoljan (konačan) broj superklasa koje ga sadrže kao podklasu, što omogućuje čvoru da nasledi svojstva od više “roditeljskih” čvorova i njihovih predaka u mreži.
- Za određivanje nasleđivanja u takvim “zapetljanim” mrežama gde je dozvoljeno višestruko nasleđivanje se često koriste **pravila (najjednostavniji primer)**:
  - Ako je X<A<B i oba A i B imaju svojstvo P, tada X nasleđuje to svojstvo od A (rešava se problem izuzetka).
  - Ako je X<A i X<B ali nije A<B niti B<A, i A i B imaju svojstvo P sa različitim i nekonzistentnim vrednostima, tada X uopšte ne nasleđuje to svojstvo (rešava se problem konfliktnih vrednost).
    - U Semantičkom Webu ovo je razrešeno tako što je ontologija nekonzistentna – obzirom na veličinu ontologija može da bude veoma ozbiljan problem pronaći entitet koji je ontologiju preveo u nekonzistentno stanje

Moguća rešenja za probleme kod višestrukog nasleđivanja

- Ima više rešenja za **problem izuzetaka i konfliktnih vrednost kod višestrukog nasleđivanja**
  - **Pravila**
  - **Udaljenost u stablu nasleđivanja**
  - **Višestruka stabla**
    - Pretpostavljeno i alternativna
  - **Višestruki čvorovi za isti koncept**
  - **Evidencijalno rezonovanje**

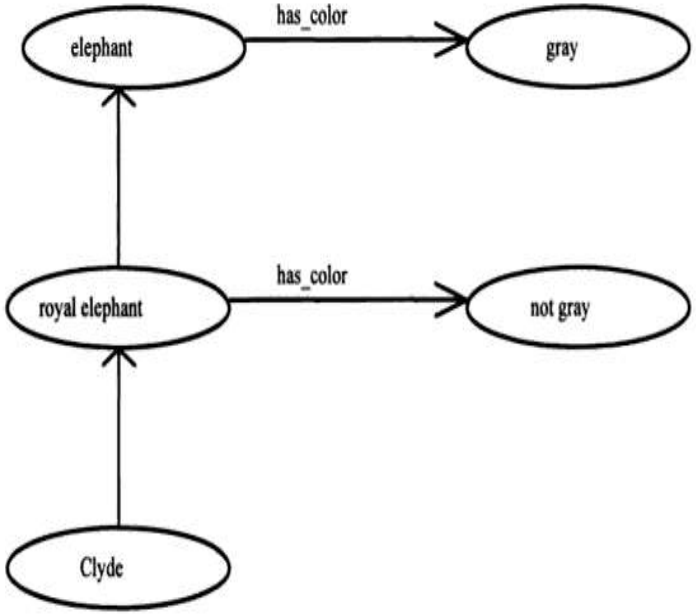
Pravila

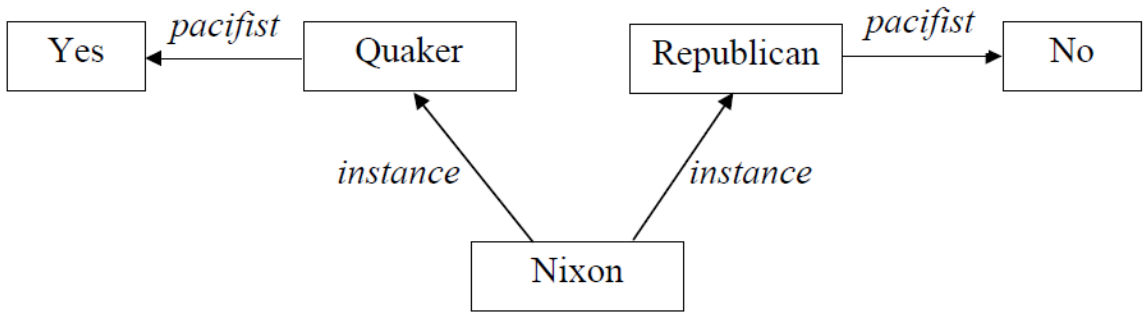
- Problem razrešimo tako što uvedemo pravila koja se odnose na konkretne instance
- Jednostavno rešenje
- Nije generičko
- Pravilo: Za Nixona je bitnije što je Republikanac



Udaljenost u stablu nasleđivanja

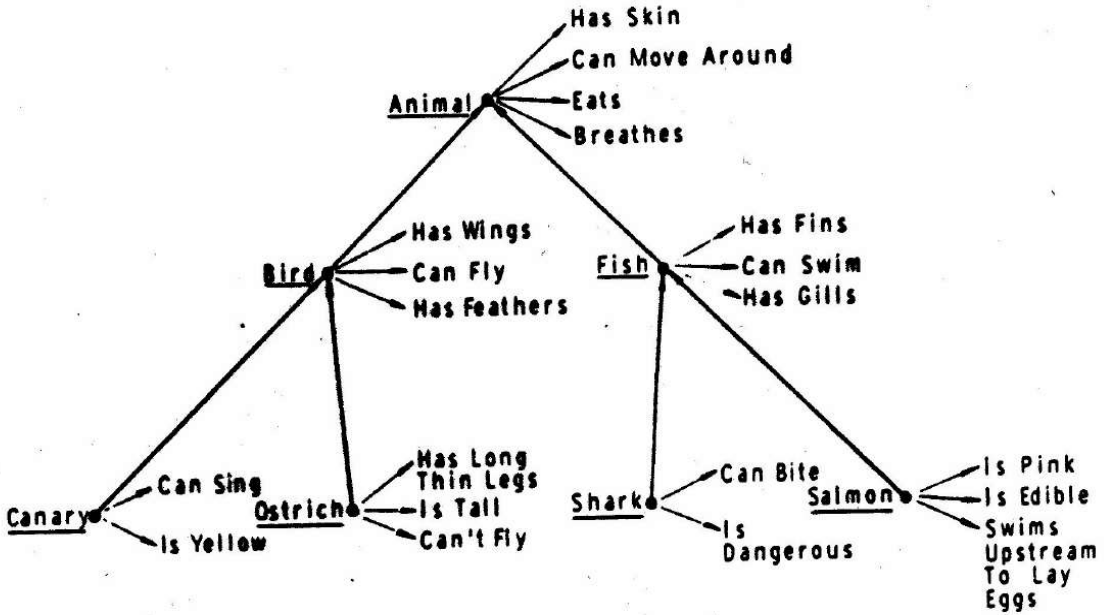
- Blizina: Čvor *A* je bliži čvoru *B* nego čvoru *C* ako A ima putanju zaključivanja koja prolazi kroz *B* na putu do *C*.
- Obratite pažnju da je ovo relacija parcijalnog poretka:
  - U slučaju Niksona nam neće vredeti



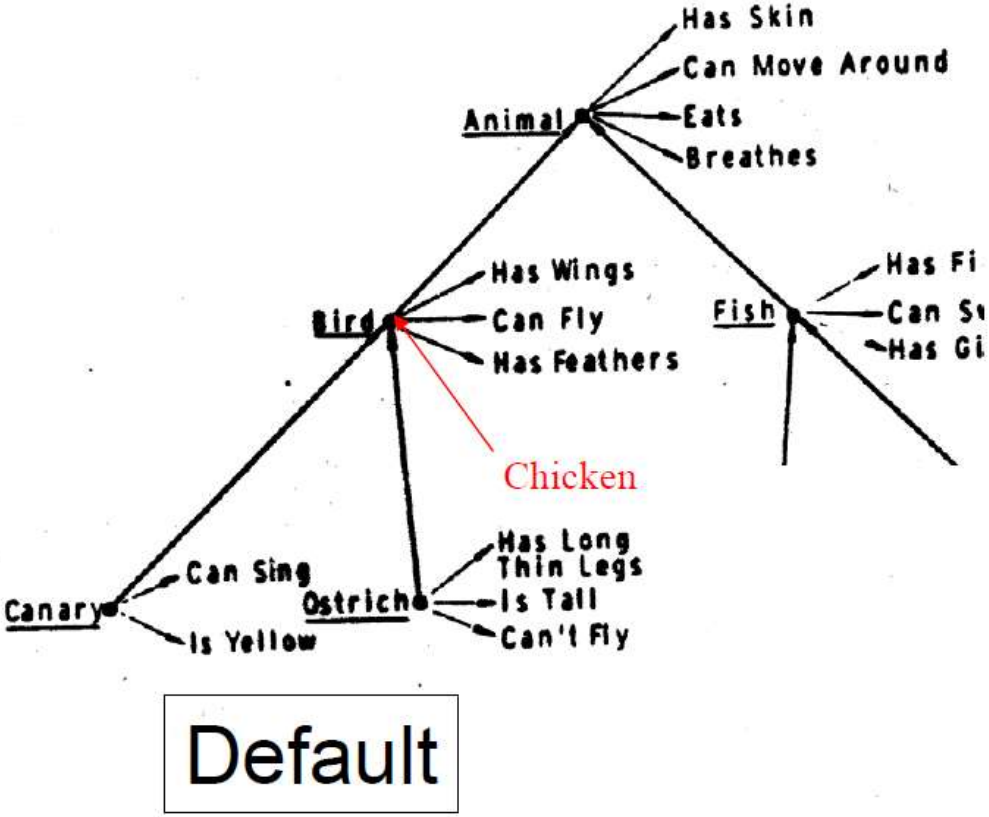


Višestruka stabla

- Ima više stabala: pretpostavljeno stablo i po jedno za svaku alternativnu hipotezu.
- Ako trebamo da ubacimo klasu kokošku u semantičku mrežu. Kokoška bi povezali sa kojim čvorom?
  - Pretpostavljeno stablo: kokošku spada u ptice.
  - Alternativno stablo: kokošku spada u životinje.

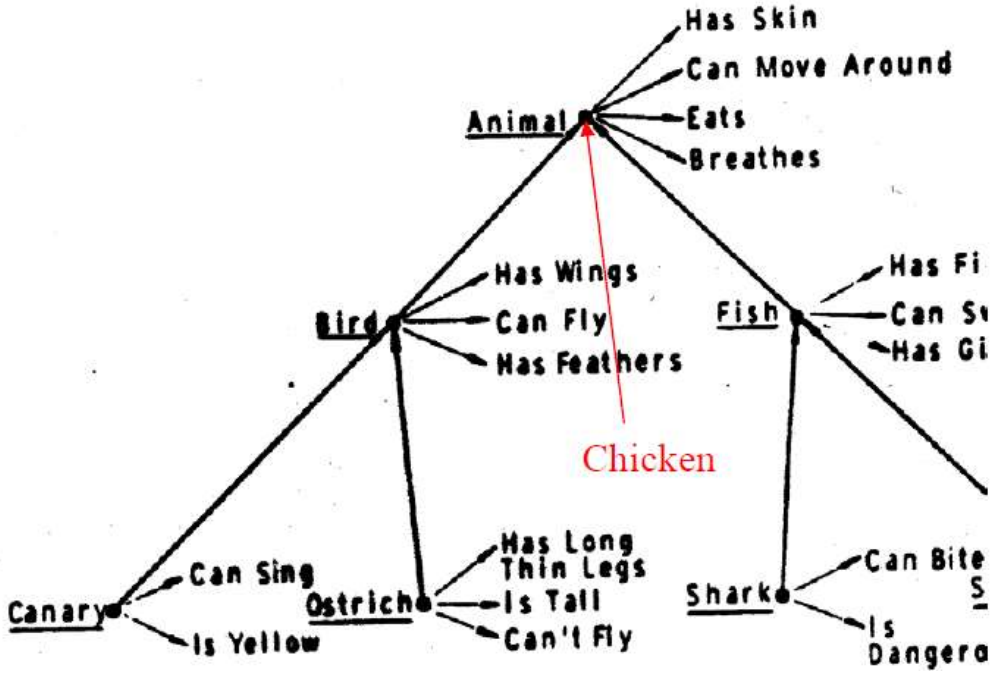


Višestruka stabla



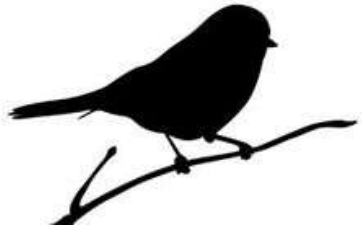
Default

Alternative

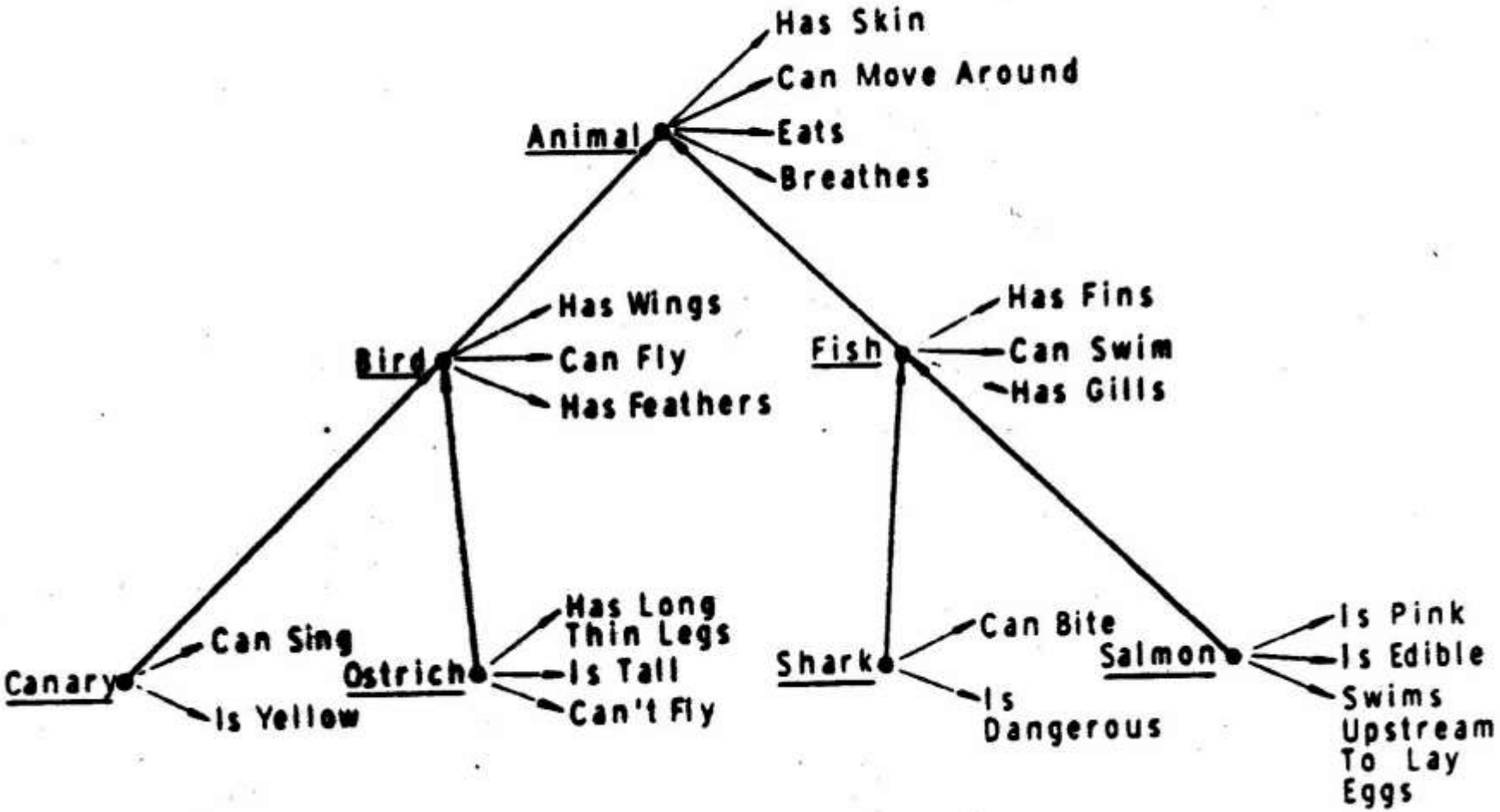


Višestruki čvorovi

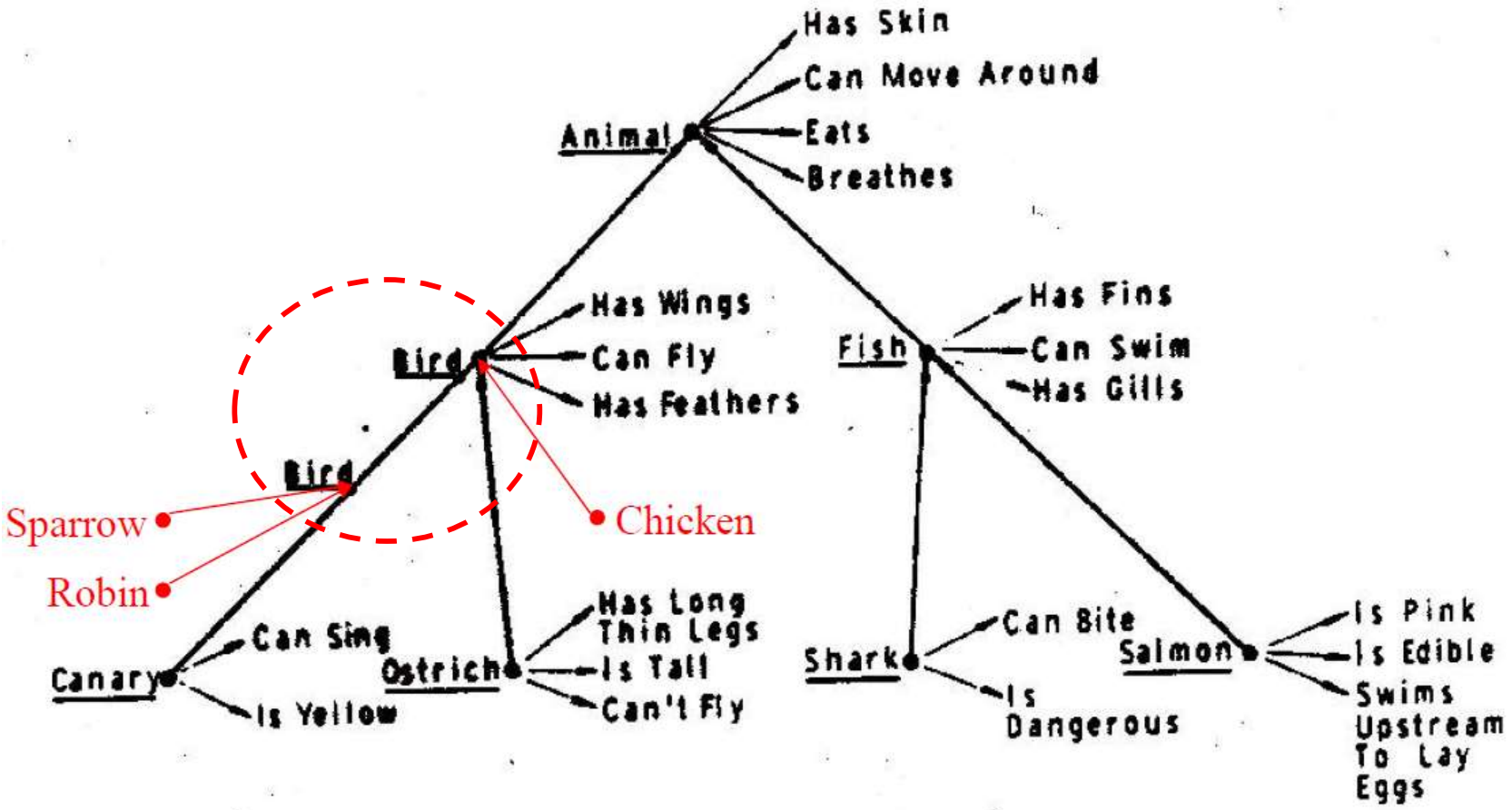
- Mapiranje jednog objekta ili klase na više čvorova
- Reč “bird” se mapira na dva čvora, jedan koji se odnosi na sve ptice i drugi koji se odnosi *samo na tipične* ptice.
- Ideja: reč “bird” se mapira na:
  - *Generalnu definiciju* – klasa toplokrvnih kičmenjaka koje karakteriše perje, kljun bez zuba, polaganje jaja, brz metalbolizam, srce sa četiri komore i lak ali snažan skelet
  - *Prototip ptice* –



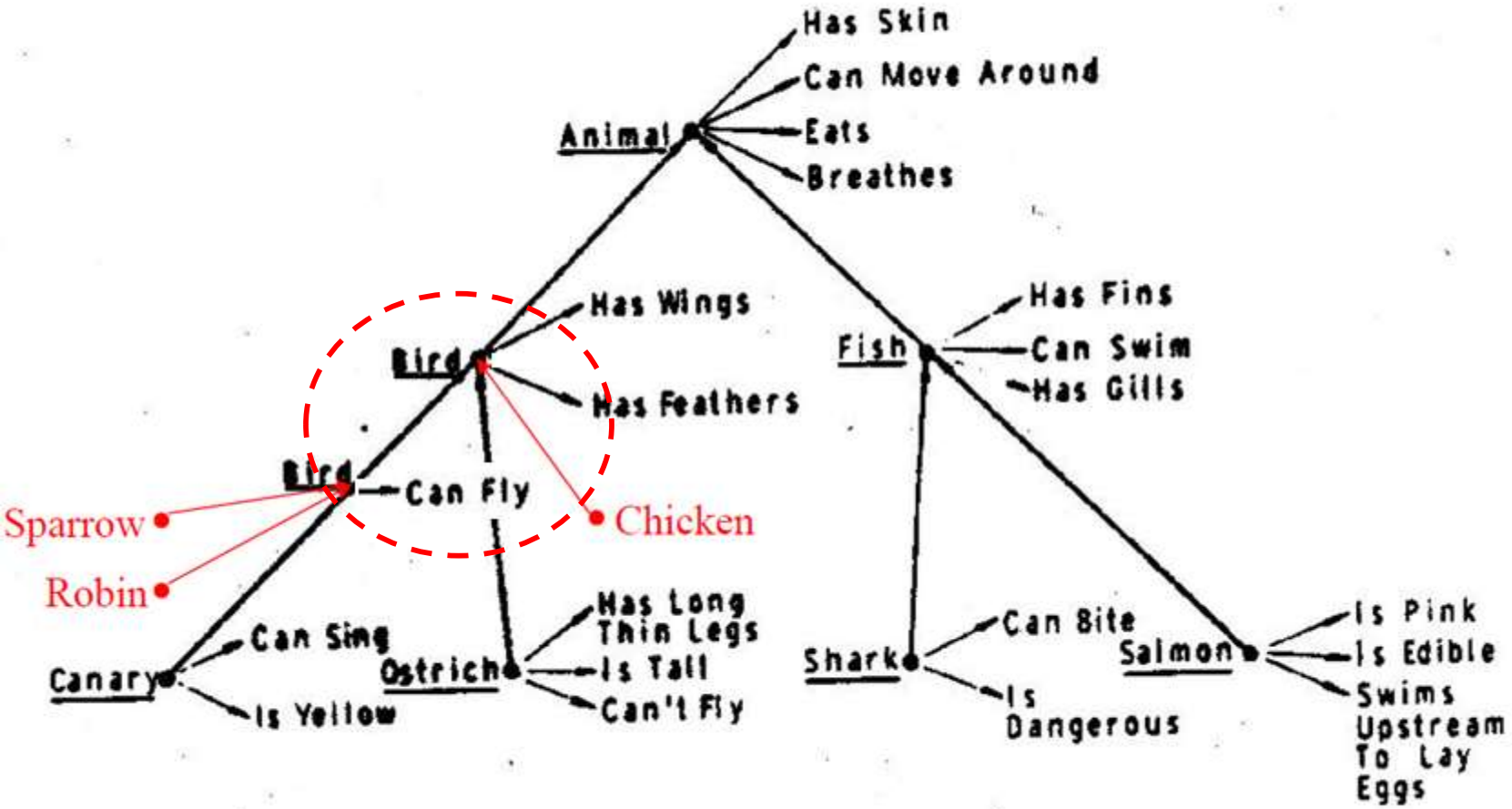
Višestruki čvorovi



Višestruki čvorovi



Višestruki čvorovi



Evidencijalno rezonovanje

- Pristup zvani **evidencijalno rezonovanje** je metod iz **teorije odlučivanja** koji može da pomogne u rešavanju problema višestrukog nasleđivanja i izuzetaka tako što koristi **teoriju verovatnoće za manipulisanje neodređenošću** i **logiku za predstavljanje strukture**
- Evidencijalne informacije se predstavljaju u formi relativnih frekvencija
- Evidencijalnim rezonovanjem se **odabre najverovatniji odgovor**
- Sastoji se od tri koraka:
  - Iznošenje činjenica
  - Izračunavanje vrednosti likelihood funkcije za svaku od konkurentnih hipoteza na osnovu činjenica
  - Izbor najizglednije hipoteze

Likelihood

- Neka je $\boldsymbol{a_1, a_2, \ldots, a_n}$ skup **konkurtentkih hipoteza** i neka je ***K*** **baza znanja**
- **Likelihood $\boldsymbol{l_i}$ hipoteze $\boldsymbol{a_i}$** definiše se kao **broj interpretacija koje zadovoljavaju $\boldsymbol{a_i}$ i koje su konzistentne sa *K*** i podeljen sa **ukupnim brojem interpretacija koje su konzistentne sa *K***
- Znači, najizglednija je ona hipoteza koja je konzistentna sa najvećim brojem interpretacija koje zadovoljavaju *K*

Evidencijalno rezonovanje - primer

- K:
  - ima ukupno 100 slonova
  - njih 40 su kraljevski slonovi
  - Jack je slon
- a_1: Jack je kraljevski slon – l_1 = (Jeck je slon)*(broj kraljevskih slonova)/(ukupan broj slonova)=0.4
- a_2: Jack nije kraljevski slon – l_2 = (Jeck je slon)*(broj slonova koji nisu kraljevski)/(ukupan broj slonova)=0.6

Evidencijalno rezonovanje - primer

- K:
  - ima ukupno 100 slonova
  - 70 slonova su sivi
  - Jack je kraljevski slon
- a1: Jack je siv – l_1 = (Jeck je kraljevski slon)*(broj sivih slonova)/(ukupan broj slonova)=0.7
- a_2: Jack nije siv – l_2 = (Jeck je kraljevski slon)*(broj slonova koji nisu sivi)/(ukupan broj slonova)=0.3
- Zaključak: Jack je siv

Evidencijalno rezonovanje - primer

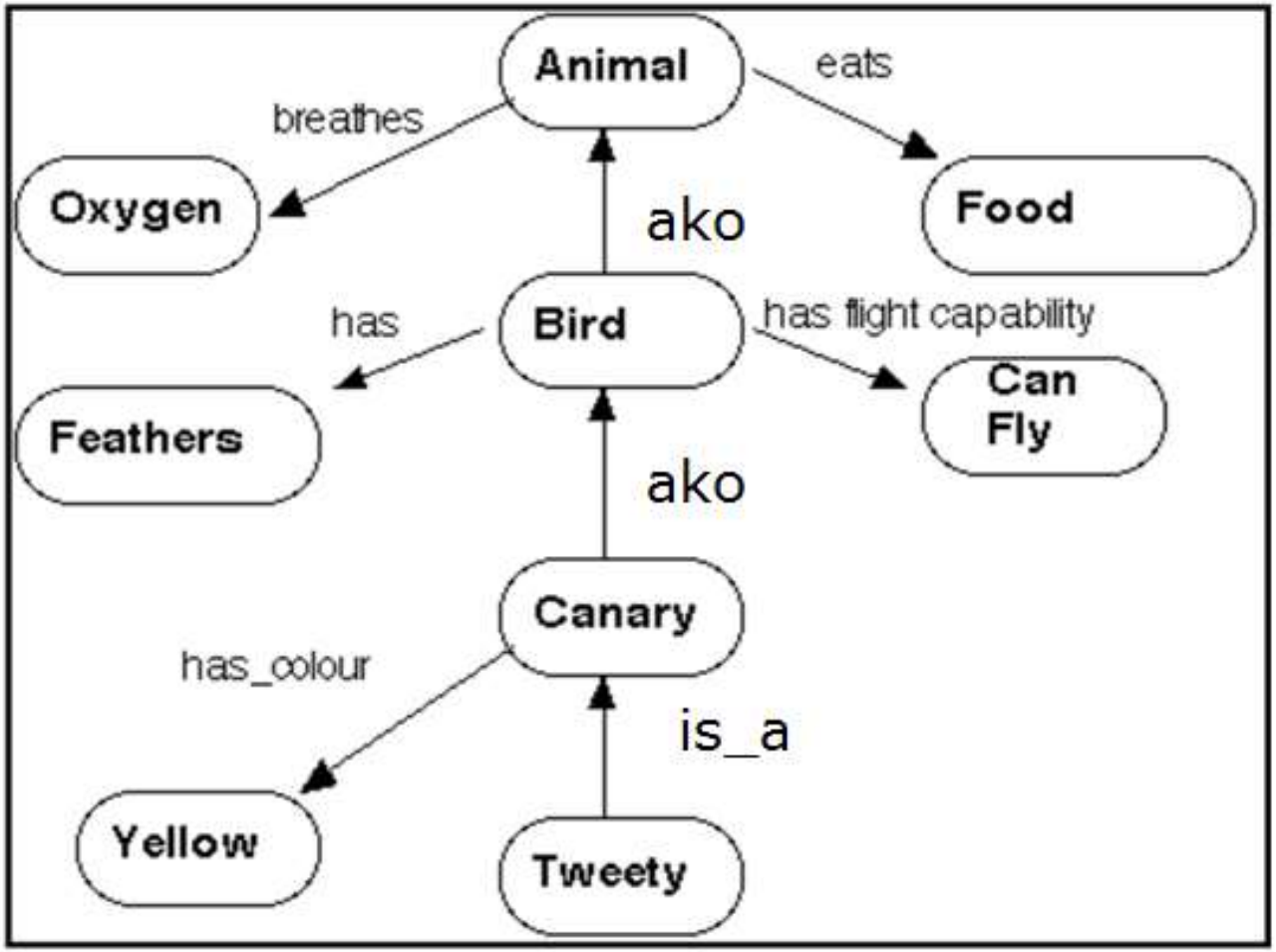
- K:
  - ima ukupno 100 slonova
  - njih 40 su kraljevski slonovi
  - 70 slonova su sivi
  - 30 slonova nisu sivi
- a_1: kraljevki slonovi su sivi – l_1 = (broj kraljevskih slonova)*(broj sivih slonova)/(broj slonova)= 40*70/100=28
- a_2: kraljevki slonovi nisu sivi – l_1 = (broj kraljevskih slonova)*(broj slonova koji nisu sivi)/(broj slonova)= 40*30/100=12
- Zaključak: a_1 hipoteza je verovatnija nego a_2

Evidencijalno rezonovanje - primer

- K:
  - ima ukupno 100 slonova
  - 70 slonova su sivi
  - 30 slonova nisu sivi
  - Nijedan kraljevski slon nije siv
  - Kraljevski slonovi su slonovi
  - Jack je kraljevski slon
- a_1: Jack je siv – više nije konzistentna sa k
- a_2: Jack nije siv – jedina konzistentna hipoteza (l_2 = 100%)
- Zaključak: Jack nije siv
- Šta je potrebno da uvedemo u semantičku mrežu da bismo mogli da imamo evidencijalno rezonovanje?

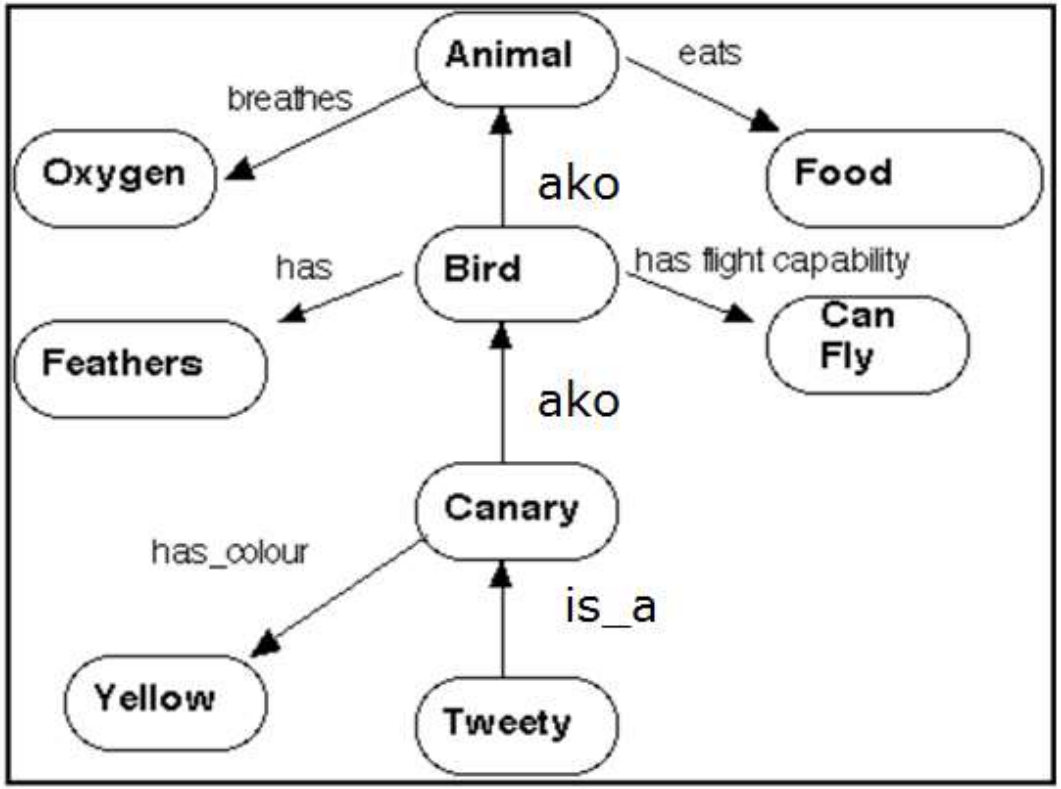
Zaključivanje u semantičkim mrežama

primer: Tweety je kanarinac



Zaključivanje u semantičkim mrežama

- Upiti:
  - "Koje je boje Tweety?"
  - "Može li Tweety da leti?"
  - "da li Tweety udiše kiseonik?"



Zaključivanje u semantičkim mrežama: Algoritam za nasleđivanje vrednosti

- Neka je **F zadati čvor** (“Tweety”) a **S zadati atribut** (n.pr. “breathes”)
- Formiramo **red** koji se sastoji od **čvora F** i **svih klasnih čvorova koji su sa čvorom F** povezani relacijom ***is_a*** (instance of). Čvor F je na vrhu reda.
- Naredni koraci se ponovljaju više puta, sve dok se red **ne isprazni** ili **ne nađe vrednost**.
  - Preuzeti element sa početka reda i videti da li **ima vrednost u svom atributu S**.
  - Ako elemenat ima vrednost u svom atributu S, vrednost je nađena (terminira algoritam).
  - U narodnom koraku dodati na kraj reda čvorove koji su sa prvim elementom povezani relacijom *ako* i ukloniti prvi elemenat iz reda.
  - Ponoviti korak
- Ako je vrednost nađena, tada ta vrednost je vrednost atributa S za čvor F. U protivnom, vrednost nije nađena.

Zaključivanje u semantičkim mrežama: Algoritam za nasleđivanje vrednosti

Neka je **F zadati čvor** (“Tweety”) a **S zadati atribut** (n.pr. “breathes”)

1. (**Tweety**, **Canary**) – inicijalno popunjen red, Tweety **is_a** Canary
2. Preuzet **Tweety**, nema atributa **breathes**, nema ako relacije, ukoni **Tweety**, izgled reda (**Canary**)
3. Preuzet **Canary**, nema atributa **breathes**, postoji ako relacija **Canary** **ako** **Bird**, ubaci **Bird** na kraj reda (**Canary**, **Bird**) , ukoni **Canary**, izgled reda (**Bird**)
4. Preuzet **Bird**, nema atributa **breathes**, postoji ako relacija **Bird** **ako** **Animal**, ubaci **Animal** na kraj reda (**Bird**, **Animal**), ukoni **Bird**, izgled reda (**Animal**)
5. Preuzet **Animal**, ima atribut **breathes**, preuzeti vrednost za atribut **breathes=Oxygen**
6. **ZAKLJUČAK:** Čvor **Tweety** za atribut **breathes** ima vrednost **Oxygen**.

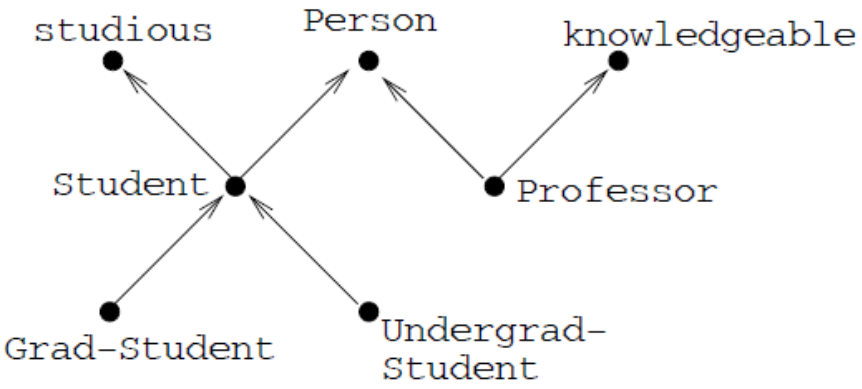
Koji je ovo algoritam?

Zaključivanje u semantičkim mrežama

- Kako se može odgovoriti na pitanja oblika:
  - Da li je **koncept A specijalizacija koncepta B**?
  - Da li je mačka životinja?
  - Da li je BMW auto?
- Za ilustrovanje mehanizma zaključivanja posmatraćemo ograničen oblik semantičke mreže
  - **Mreža sa striktnim nasleđivanjem**

INTUICIJA

- **Mreža sa striktnim nasleđivanjem** sadrži **čvorove** (koncepti, svojstva) i **usmerene linkove** (generalizacija/IS_A relacija).



- **Zadatak rezonovanja**: da li je koncept *B* **specijalizacija** (podkoncept) drugog koncepta *B'*?
- **Pitanje**: Može li se taj zadatak rešiti efikasno?

Mreže kao skupovi formula

Mreža sa striktnim nasleđivanjem je skup Θ formula oblika

$$C_1 \textbf{ is_a } C_2.$$

Primer :

Student	**is_a**	Person
Student	**is_a**	studious
Professor	**is_a**	Person
Professor	**is_a**	knowledgeable
Grad-Student	**is_a**	Student
Undergrad-Student	**is_a**	Student

Zadatak rezonovanja (nasleđivanje): $\Theta \models C_1$ **is_a** C_2.

Logička semantika

Sledeća logička semantika se dodeljuje **is_a** formulama: **Svaka instanca sub-koncepta je ujedno i instanca super-koncepta**.

$$C_1 \textbf{ isa } C_2 \rightarrow \forall x: C_1(x) \rightarrow C_2(x).$$

Sada se **problem nasleđivanja** može **redukovati** na predikatsku logiku na sledeći način.

Neka je $\pi(\Theta)$ translacija. Tada želimo da znamo:

$$\pi(\Theta) \models \forall x: C_1(x) \rightarrow C_2(x).$$

Koja je računska složenost problema?

Polinomijalni algoritam rezonovanja

Neka je G_Θ "graf koji je odgovarajući za Θ". Tada važi:

$$\pi(\Theta) \models \forall x: C_1(x) \rightarrow C_2(x)$$

Iff – if and only if (akko – ako i samo ako)

Postoji putanja u G_Θ od C_1 do C_2.

- Ovim je rezonovanje u mrežama sa striktnim nasleđivanjem svedeno na problem dohvatljivosti grafa, što je rešivo u polinomijalnom vremenu.

Napomena:

- Rezonovanje NIJE JEDNOSTAVNO jer smo koristili graf za reprezentaciju znanja (neki od problema na grafovima su u stvari veoma teški).
- Rezonovanje JESTE JEDNOSTAVNO jer je ekspresivnost, u poređenju sa logikom prvog reda, veoma ograničena.

Semantičke mreže omogućuju tranzitivno rezonovanje, ali?

- Tweety je slon, slon je sisar: Tweety je sisar.
- Predsednik SAD bira se savke 4 godine. Buš je predsednik SAD: Buš se bira svake 4 godine!!!!
- Moj auto je Ford, Ford je fabrika automobila: Moj auto je fabrika automobila!!!!

, ali se može doći do besmislenih zaključaka

Karakteristike svojstava u semantičkim mrežama

- Potrebno omogućiti da se za konkrentno svojstvo navede da li je tranizitivno
- Ali ne samo to – OWL ima sledeće karakterizacije svojstava:
  - SymmetricObjectProperty(:hasSpouse)
  - AsymmetricObjectProperty(:hasChild)
  - DisjointObjectProperties(:hasParent :hasSpouse)
  - ReflexiveObjectProperty(:hasRelative)
  - IrreflexiveObjectProperty(:parentOf)
  - FunctionalObjectProperty(:hasHusband)
  - InverseFunctionalObjectProperty(:hasHusband)
  - TransitiveObjectProperty(:hasAncestor)
  - FunctionalDataProperty(:hasAge)

Implementacija semantičkih mreža

- Dva generalna načina
  - **Direktan pristup**
    - Kao grafovska struktura
    - Zaključuje se prelaskom grafa
  - **Indirektan pristup**
    - Logičko/funkcioanlno programiranje
      - Mehanizam zaključivanja iz Prolog-a može se direktno koristiti
    - Objektno orijentisano programiranje
      - Algoritmi za zaključivanje se moraju implementirati
    - Hibridni pristup
      - CLOS (Common Lisp Object System) – standrad za objektno orjentisano programiranje u LISPu. Implementacija semantičkih mreža je predložena u https://www.cs.unm.edu/~luger/ai-final2/LISP/CH%2018_Semantic%20Networks,%20Inheritance,%20and%20CLOS.pdf

Pitanja implementacije

- Postoje jednostavne i efikasne reprezentacione šeme za semantičke mreže
  - Tabele koje sadrže sve objekte i njihova svojstva
  - Tabele ili povezane liste za relacije
- Konverzija u različite reprezentacione metode
  - Predikatska logika
    - Čvorovi odgovaraju varijablama ili konstantama
    - Linkovi odgovaraju predikatima
  - Propoziciona logika
    - Čvorovi i linkovi se prevode u propozicione varijable i na pogodan način se kombinuju logičkim veznicima

Prednosti semantičkih mreža

- Pogodne su za predstavljanje strukturiranog činjeničnog znanja
- Fleksibilne za predstavljanje konceptnih zavisnosti.
- Mogu da predstave ideje kauzalnosti.
- Relativno lako obuhvataju i predstavljaju rečenice prirodnog jezika.

Nedostaci semantičkih mreža

- Izražajnost
  - Čvor nema internu strukturu
  - Problem predstavljanja nebinarnih relacija (relacije između više čvorova)
    - Reifikacije umeju da „zakomplikuju“ mrežu
  - Nije lako reprezentovati heurističke informacije
  - Proširenje je moguće, ali ne baš na elegantan način
- Efikasnost
  - Može da dovede do velikih skupova čvorova i linkova
  - Pretraga može da dovede do kombinatorne eksplozije jer sama mreža ne sadrži informacije koje bi pomogle pri pretrazi
    - Posebno za upite sa negativnim rezultatom – treba proći kroz celu mrežu
- Upotrebljivost
  - Nedostatak standarda
    - Za tipove čvorova (razlikovanje instance od klasa)
    - Za tipove linkova (likonvi definišu proizvoljno)
  - Imenovanje čvorova
    - Klase, instance

Primer: Semantix Net







Abraracourcix

Astérix

is-boss-of

is-boss-of

Cétautomatix

is-a

is-friend-of

is-a

is-a

buys-from

fights-with

Obélix

Gaul

is-a

Panoramix

is-a



AKO

Dog

sells-to

is-a

Human

lives-with

is-a

takes-care-of



barks-at

Idéfix



Ordralfabetix



[http://www.asterix.tm.fr]

Semantix Net nije baš najpreciznija

- boje
  - Trebale bi da budu pravilno kodirane kao **posebni čvorovi** sa relacijama ka odgovarajućim objektima
- Tipovi fontova
  - Impliciraju različite **tipove relacija**
  - Da nije tako nacrtano, trebali bi dodatni čvorovi i relacije
- Relacije među klasama
  - Ne žive svi psi sa Ljudima
  - AKO (a-kind-of) relacija je specijalna (nasleđivanje)
- Instance
  - Strelice od individuala koji su ljudi ka klasi Human su izostavljenje
    - Pretpostavljeno da AKO dozvoljava nasleđivanje
- Usmerenosti
  - Smer strelica je važan, ne smer teksta

Nedostaci semantičkih mreža

- Imenovanje čvorova i linkova
- Kombinatorna eksplozija pri pretrazi linkova
  - Heuristika??
  - Kontrolne strategije???

Metode za reprezentaciju znanja (ZR metode)

- Produkciona pravila
- Stabla, grafovi (n.pr. Semantičke mreže, konceptne mape, konceptni grafovi, ontologija)
- **Šeme i frejmovi**
- Logika

=> Rezonovanje

Plitko i duboko znanje

- Plitko znanje:
  - Znanje bazirano na činjenicama
  - Bez razumevanja **uslova koji dovode do tih činjenica**
- Semantičke mreže su primer formalizma za predstavljanje plitkog znanja
  - Svo znanje je u čvorovima i vezama
- Duboko znanje:
  - Znanje zasnovano na formalnoj teoriji domena
  - Objašnjava semantiku zašto se nešto dešava
  - Na primer, zaključivanje o bolesti na osnovu znanja patofiziološkog procesa
- Šeme su primer formalizma za predstavljanje dubokog znanja
  - Objašnjavaju semantiku zašto se nešto dešava

Plitko i duboko znanje

- Posmatrajmo plitki medicinski SBZ izgrađen sa pravilima poput sledećeg
  - IF osoba ima temperaturu
    THEN uzeti aspirin
    - Nema objašnjenja zašto će aspirin pomoći da se smanji temperatura
- Znanje je **bazirano na sintaksi**, **ne na semantici**
- Domenski ekspert (lekar) ima duboko znanje o problemu (u većini slučajeva…)
  - Zašto pacijent ima temperaturu? (patofiziologija)
  - Kako deluje aspirin? (farmakologija)

Plitko i duboko znanje

- U SBZ sa plitkim znanjem poptpuno su prihvatljiva pravila kao što je sledeće
  - IF kiša pada u Kaću
    THEN biće zemljotres na Aljasci
- Sve dok činjenice same nisu kontradiktorne:
  - IF kiša pada u Kaću
    THEN ne pada kiša u Kaću
- Plitko znanje je dovoljno za mnoge realne probleme, ali složeni problemi zahtevaju duboko znanje.

Šeme

- U psihologiji **šema** se tumači kao **mreža koncepata** kojom se upravlja proces razmišljanja. Predstavlja aktivnu **organizaciju prošlih iskustava i reakcija** koji se koriste pri razmišljanju i percepciji.
  - Npr. Kada bi prisustvovali **Bečkom** novogodišnjem koncertu očekujete da vidite lepo obučene ljude koji sviraju klasičnu muziku, retko bi čekivali da tamo vidite ljude u staroj pocepanoj i kožnoj odeći koji sviraju metal ili punk.
- U AI **šema** se tumači kao mreža bazičnih gadivnih blokova za organizovanje, čuvanje i pronalaženje znanja.

Šeme

- Pogodne za reprezentovanje složenijih znanja
  - **Kauzalne (uzročne) veze između opažaja ili akcije i odgovarajućeg ishoda**
    - Na primer, hodanje je jedna senzomotorna shema koja uključuje koordinaciju čulnih opažaja sa motornim radnjama
  - "dublje" znanje nego semantičke mreže
    - **Čvorovi mogu da imaju internu strukturu!**
  - Za ljude često **tacitsko znanje**
    - Semantičke mreže omogućuju predstavljanje deklarativnog znanja
- Posmatraćemo šemu kao mrežu koncepata kojom se opisuje neki drugi koncept

Konceptna šema

- **Apstrakcija** koja obuhvata **opšte/tipične karakteristike konkretnih objekta**
  - **Sadrži najvažnija svojstva** koja se obično pridružuju objektu tog tipa
    - Može biti zavisna od zadatka, konteksta, sposobnosti korisnika, …
  - Slično stereotipovima
  - Na primer, **konceptualna shema jabuke** će uključiti opšte **karakteristike jabuka** kao što su **veličina, boja, ukus, ...**
- Pojednostavljuje rasuđivanje koncentrisanjem na **suštinske aspekte**
- Ipak može da zahteva metode zaključivanja specifične za konkretne veze
  - Kao što smo imali da se rezonovanje u semantičkim mrežama baziralo na tipovima veza, na primer AKO veze

Šeme

- Šema može da se linkuje sa drugim šemama
- Šeme mogu da se hijerahijski organizuju
- Frejmovi su najbolji primeri šema

Primeri šeme

- Najčešće korišćene **instance šema** su
  - frejmovi [Minsky 1975]
  - skriptovi [Schank 1977]
- **Frejmovi** su **grupe slotova** pomoću kojih se definišu **stereotipni objekti**
- **Skriptovi** su **vremenski uređene sekvence frejmova**

više naredni čas

Frejm

- Predstavlja znanje o objektu
  - Ima pretpostavljene (dafault) vrednosti za većinu slotova
- Frejmovi mogu se koristiti za predstavu:
  - Deklarativnog znanja (objekti i propertiji)
  - Proceduralnog zananja (preuzimanje informacija i dostizanje cilja)
  - Nasleđivanja
- Frejmovi su organizovani hijerarhijski
  - Kao i semantičke mreže, podržavaju **nasleđivanje**
  - Zajedno sa svim problemima koje to donosi
- Znanje se najčešće organizuje po vezama uzroka i efekta (slično kao u rule based sistemima)

Primer jednostavnog frejma

Slot Name	Filler value
name	Astérix
height	small
weight	low
profession	warrior
armor	helmet
intelligence	very high
marital status	presumed single

Slotovi frejma

- Slotovi mogu da sadrže razne podatke tipa:
  - Pravila, činjenice, slike, video, komentari, informacije za debagovanje, pitanja, hipoteze, ali i druge frejmove
- Slotovi mogu da sadrže proceduralne dodatke
  - Procedure koje se pozivaju u specifičnim situacijama u koje je uključen određeni slot
    - Pri kreiranju, modifikaciji, uklanjanju vrednosti slota

Struktura frejma

- Dva osnovna elementa strukture frejma: **slotovi** i **faceti**.
- Svaki slot sadrži jedan ili više faceta.
- Postoje slotovi kojima se definišu **preci i potomci**
  - Korišćene za uspostavljanje hijerarhije nasleđivanja svojstava (n.pr., specialization-of)
- Postoje deskritpivni slotovi
  - Sadrže deklarativne informacije ili podatke (statičko znanje)
- Proceduralni dodaci za slotove
  - Sadrže funkcije koje mogu da usmeravaju proces rezonovanje (dinamičko znanje) (n.pr., „aktiviraj određeno pravilo ako vrednost pređe neki nivo")
- Pokazivači na povezane frejmove/skriptove – mogu se koristiti za transfer upravljanja na pogodniji frejm ili pokazivač na drugi slot u frejmu.

Struktura frejma

- Frejm je lista imenovanih **slotova**. Svaki slot može da sadrži **činjenicu**, **pokazivač** na drugi frejm ili drugi slot u frejmu, **pravilo** kojim se izvodi vrednost slota, ili **proceduru** kojom se izračunava vrednost slota.

- Pokazivači kao vrednosti slotova omogućavaju frejmovima da podrže nasleđivanje.

Frejm

- Postoje dve vrste frejmova
  - Frejmovi koji reprezentuju **klase** (Class Frames)
  - Frejmovi koji reprezentuju **individuale** ili **instance** (Instance Fremes)
- Obe vrste su identične u pogledu strukture i konstrukcije stime što:
  - **frejmovi klasa** se koriste za opis čitavih klasa npr. frejm koji bi predstavljao ***finansiski izveštaj*** ili aktivnost ***ručanje u restoranu***
  - **frejmovi instanci** se koriste da opišu individualne objekte ili individualne događaje npr. frejm ***finansijski izveštaj br. 5 za 2018 godinu*** ili ***petar petrović je ručao u restoranu***

Rezonovanje

- Postoje dve vrste rezonovanja nad frejmovima:
  1. bottom-up rezonovanje – vođeno podacima, vođeno događajima
  2. top-down rezonovanje – viđeno očekivanjima

Slotovi

- Faceti u slotovima mogu da budu sledećeg oblika:
  - Eksplicitno zadata vrednosti (values)
  - Pretpostavljena vrednost (default)
    - Koristi se ako nije zadata druga vrednost
  - Opseg (range)
    - Koja vrsta informacija može da se pojavi u slotu
  - if-added
    - Proceduralni dodatak koji specificira akciju koja će biti preduzeta ako se vrednost slota doda ili modifikuje (data-driven, event-driven, bottom-up rezonovanje)
  - if-needed
    - Proceduralni dodatak koji aktivira proceduru koja spolja pribavlja informacije koje slot nema (expectation-driven; top-down rezonovanje)
  - if-removed
    - Procedura koja će se izvršiti kada se vrednost ukloni iz slot
  - Ostalo
    - Može da sadrži frejmove, pravila, semantičke mreže ili druge tipove zananja.

Pretpostavljena vrednost slotova

- Pretpostavljena vrednost su od primarne važnosti u frejmovima
- Kako su frejmovi instance šema, a šemama se ostvaruje aktivna organizacija prošlih iskustava i reakcija koji se koriste pri razmišljanju i percepciji, tada pretpostavljene vrednosti
  - Odgovararaju očekivanjima koja su se formirala na osnovu iskustva iz određenih situacija
  - Kada se dogodi nova situacija, najrelevantniji frejm (najbliži) se modifikuje da bi se prilagodili novonastaloj situaciji
  - Koriste se da predstave common sense znanje

Korišćenje frejmova

- Popunjavanje slotova u frejmu
  - Vrednost se može direktno naslediti
  - Može se uzeti pretpostavljena vrednost
    - Ovo su relativno jeftine stvari
  - Informacija se može izvesti proceduralnim dodatcima (ili metodama) koje koriste prednost tekućeg konteksta (slot-specifična heuristika)
  - Popunjavanje slota takođe potvrđuje da je frejm ili skript podesan za tu određenu situaciju

Primer frejma za restoran

- Generički šablon za restorane
  - Različiti tipovi
  - Predefinisane vrednosti
- Skript za tipične sekvence aktivnosti u restoranu

Generički frejm za restoran

Generic RESTAURANT Frame

Specialization-of: Business-Establishment
Types:
- **range**: (Cafeteria, Fast-Food, Seat-Yourself, Wait-To-Be-Seated)
- **default**: Seat-Yourself
- **if-needed**: IF plastic-orange-counter THEN Fast-Food,
  IF stack-of-trays THEN Cafeteria,
  IF wait-for-waitress-sign or reservations-made THEN Wait-To-Be-Seated,
  OTHERWISE Seat-Yourself.

Location:
- **range**: an ADDRESS
- **if-needed**: (Look at the MENU)

Name:
- **if-needed**: (Look at the MENU)

Food-Style:
- **range**: (Burgers, Chinese, American, Seafood, French)
- **default**: American
- **if-added**: (Update Alternatives of Restaurant)

Times-of-Operation:
- **range**: a Time-of-Day
- **default**: open evenings except Mondays

Payment-Form:
- **range**: (Cash, CreditCard, Check, Washing-Dishes-Script)

Event-Sequence:
- **default**: Eat-at-Restaurant Script

Alternatives:
- **range**: all restaurants with same Foodstyle
- **if-needed**: (Find all Restaurants with the same Foodstyle)

Restaurant skript

EAT-AT-RESTAURANT Script

Props: (Restaurant, Money, Food, Menu, Tables, Chairs)

Roles: (Hungry-Persons, Wait-Persons, Chef-Persons)

Point-of-View: Hungry-Persons

Time-of-Occurrence: (Times-of-Operation of Restaurant)

Place-of-Occurrence: (Location of Restaurant)

Event-Sequence:

first: Enter-Restaurant Script

then: if (Wait-To-Be-Seated-Sign or Reservations)
then Get-Maitre-d's-Attention Script

then: Please-Be-Seated Script

then: Order-Food-Script

then: Eat-Food-Script unless (Long-Wait) when Exit-Restaurant-Angry Script

then: if (Food-Quality was better than Palatable)
then Compliments-To-The-Chef Script

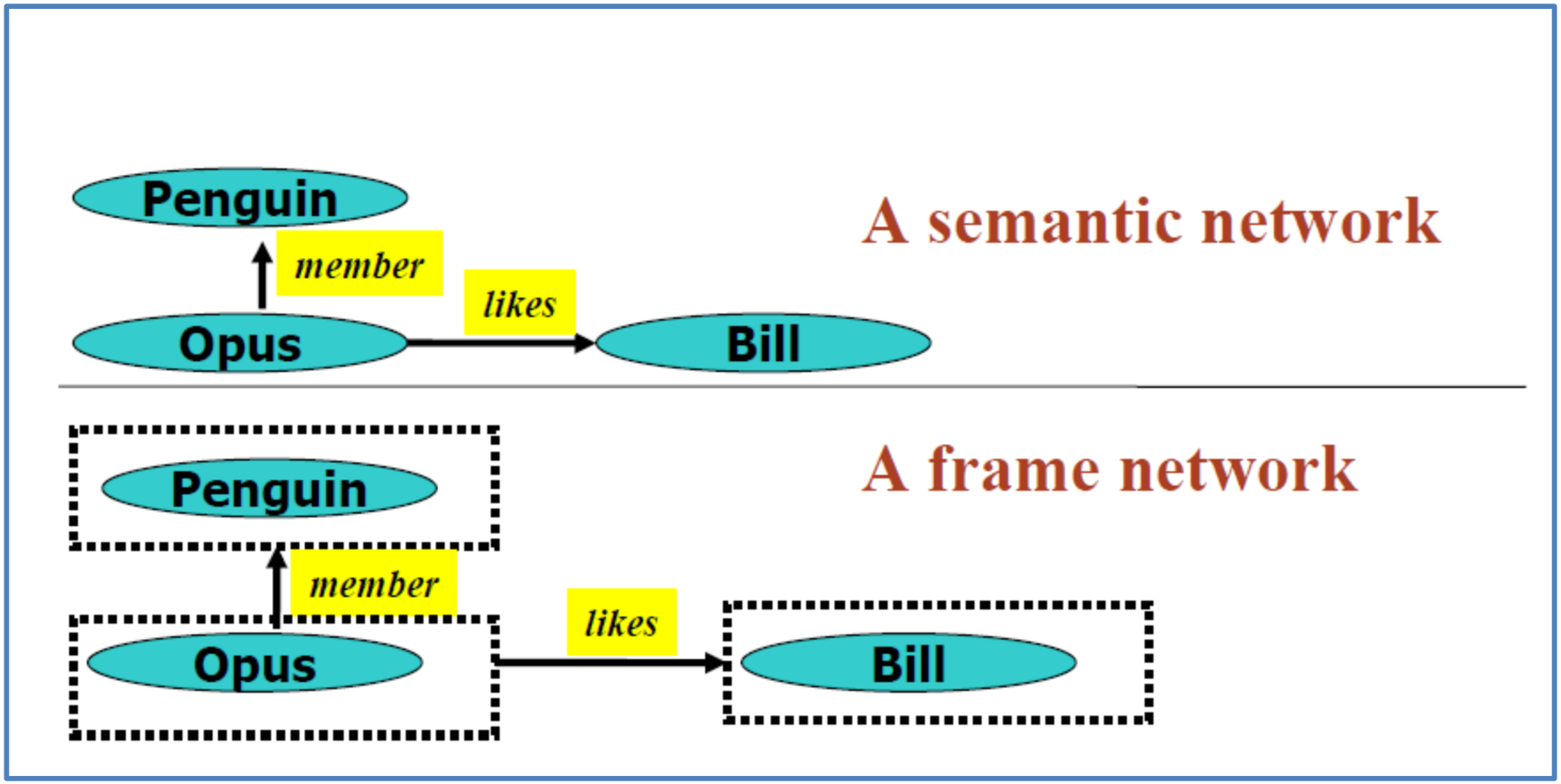
then: Pay-For-It-Script

finally: Leave-Restaurant Script

Frejmovi i Smentičke mreže

- Frejmovi (i šeme generalo) imaju internu strukturu
- Semantičke mreže nemaju internu strukturu
  - labela koje se dodeljuje čvoru predstavlja jedino znanje o tom čvoru
- Semantička mreža je kao struktura koja se sastoji od delova podatka
- Frejmovi su kao struktura podataka u kojoj čvorovi sadrže slogove.

Frejmovi i Smentičke mreže



Frejmovi

- Frames ne bi trebali da se mešaju sa pojomom zapisa (record)
  - Frejmovi ne moraju biti slični jedni drugima
  - Frejovi se organizuju u hijerahijske strukture

Prednosti frejma

- Prilično intuitivan za manipulaciju
  - Sličan orgnizaciji znanja kod ljudi
  - Podesan za kauzalna znanja
  - Lakši za razumevanje od logike i pravila
- Vrlo fleksibilan

Nedostaci frejma

- Sklonost da se frejmovi koriste za definicije koncepata
  - Nije prikladno jer **može da postoji validna instanca koncepta koja se ne uklapa u stereotip**
  - Za prevazilaženje problema mogu se koristiti izuzeci (exceptions)
    - Može da proizvede priličnu zbrku
- Nasleđivanje
  - Nije uvek potrebno sva svojstva klasnih stereotipova propagirati na podklase
  - Promena slotova može da proizvede ne nemerne posledice na podklasama

Metode za reprezentaciju znanja (ZR metode)

- Produkciona pravila
- Stabla, grafovi (n.pr. Semantičke mreže, konceptne mape, konceptni grafovi, ontologija)
- Šeme i frejmovi
- **Logika**

=> Rezonovanje

Logika

- Matematička logika uvodi tehnike reprezentacije znanja koje su razvijene davno pre rule based sistema i semantičkih mreža
- Inicijalno su bile namenje za predstavljanje znanja u matematici
- Našle su veliku primenu u AI
- Mnogi SBZ koriste neku vrstu logike za reprezentaciju znanja (posebno SBZ starijih generacija)
  - Kod mnogih se mehanizam zaključivanja bazira na pravilima Popozicione logike ili Logike prvog reda.

Logika

- Izražava znanje u određenoj matematičkoj notaciji

```
All birds have wings --> ∀x Bird(x) -> HasWings(x)
```

- Pravila za zaključivanje
  - Ako su **tačne** prethostavke ili činjenice, tada su tačne i izvedene pretpostavke ili činjenice koje se dobijaju na osnovu pravila

```
All robins are birds --> ∀x Robin(x) -> Bird(x)
```

- Moglo bi se rezonovati

```
∀x Robin(x) -> HasWings(x)
```

Pregled logičkih jezika

- **propositional logic**
  - **facts**
  - **true/false/unknown**
- first-order logic
  - facts, objects, relations
  - true/false/unknown
- temporal logic
  - facts, objects, relations, times
  - true/false/unknown
- probability theory
  - facts
  - degree of belief [0..1]
- fuzzy logic
  - degree of truth
  - degree of belief [0..1]

Propoziciona logika

- Izveo je iz bogatije predikatske logike Emil Post
- Podskup predikatske logike

Propoziciona logika

- Određuje tačnost ili netačnost izjava (propozicija) kao što su:
  - `ventil je zatvoren`
  - `pet plus četiri je jednako devet`
- Oslanjajući se na njihovu matematičku predstavu
- Kombinujuči ih sa operatorima/veznicima

and	$\wedge$
or	$\vee$
not	$\neg$
implies	$\rightarrow$
equivalent	$\leftrightarrow$

$(X \rightarrow (Y \wedge Z)) \leftrightarrow ((X \rightarrow Y) \wedge (X \rightarrow Z))$

X implicira Y i Z što je isto kao i X implicira Y i X implicira Z

Propoziciona logika

- Osobine
  - **Deklarativna**, delovi sintakse korespondiraju određenim činjenicama
  - **Kompozitna**, značenje celine se određuje na osnovu značenja njenih delova
    - Značenje $A \wedge B$ određeno je značenjem simbola A i B i simbola veznika $\wedge$
  - **Konteksno nezavisna**

Propoziciona logika

- **Logičke konstante**: true, false
- **Propozicioni simboli**: P, Q, S, ...
- **Zagrade**: (…)
- Rečenice/hipoteze/formule nastaju kombinacijom prethodnih sa veznicima:
  - $\wedge$ i
  - $\vee$ ili
  - $\rightarrow$ implikacija
  - $\leftrightarrow$ ekvivalencija
  - $\neg$ negacija

Propozicina signatura, atomi i formule

- *Propoziciona signatura σ*
  - Neprazan skup simbola koji se zovu *atomi*
- *Formule* propozicione signature *σ* formiraju se od atoma i veznika nultog reda (⊥ i T) korišćenjem unarnih veznika (¬) i binarnih veznika (∧, ∨, → i ↔)

Propoziciona logika

- Jednostavni jezik koristan za predstavu ključnih ideja i definicija
- Korisnik definiše set propozicionih simbola (propoziciona signatura), kao što bi bili P, Q i R
- Korisnik definiše semantiku/intrepetaciju svakog simbola
- Primer:
  - P znači “Vrelo je”
  - Q znači “Vlažno je”
  - R znači “Pada kiša”

Propoziciona logika

- Primer:
  - P znači “Vrelo je”
  - Q znači “Vlažno je”
  - R znači “Pada kiša”
- Korisnik definiše rečenice (formule) kombinacijom propozicionih simbola, zagrada i veznika.
- Primer:
  - “Ako je vrelo i vlažno, tada pada kiša”
  - “Ako je vlažno, tada je vrelo”
  - “Vlažno je”

Propoziciona logika

- Primer:
  - P znači “Vrelo je”
  - Q znači “Vlažno je”
  - R znači “Pada kiša”
- Korisnik definiše rečenice (formule) kombinacijom propozicionih simbola, zagrada i veznika.
- Primer:
  - $(P \wedge Q) \rightarrow R$

    “Ako je vrelo i vlažno, tada pada kiša”
  - $Q \rightarrow P$

    “Ako je vlažno, tada je vrelo”
  - Q

    “Vlažno je”

Intepretacija simbola i konstanti

- Simboli mogu biti proizvoljne činjenice
- Konstante True i False imaju fiksnu interpretaciju
  - True vrednost indicira da je svet onakav kako se tvrdi
  - False vrednost indicira svet nije onakav kako se tvrdi

Istinitosne vrednosti

- Simboli FALSE i TRUE su *istinitosne vrednosti*.
- *Intrepretacija* za propozicionu signaturu je dodela vrednosti (ili TRUE ili FALSE) svakom simbolu iz signature i dodela uobičajenih istinitosno-funkcijskih značenja veznicima (tabele istinitosti).
- *Interpretacija* formule je funkcija koji rečenicu/formulu od signature σ mapira na *{TRUE, FALSE}*
- Semantika propozicionih formula određuje koje istinitosne vrednosti se dodeljuju simbolima u furmuli *F* u interpretaciji *I*
- Znači, u zavisnosti od interpretacije formula može da bude TRUE ili FALSE

Semantika veznika

x	¬(x)
FALSE	TRUE
TRUE	FALSE

x	y	x ∧ y	x ∨ y	x → y	x ↔ y
FALSE	FALSE	FALSE	FALSE	TRUE	TRUE
FALSE	TRUE	FALSE	TRUE	TRUE	FALSE
TRUE	FALSE	FALSE	TRUE	FALSE	FALSE
TRUE	TRUE	TRUE	TRUE	TRUE	TRUE

Interpretacija formula

- U propozicionoj logici za svaku formulu *F* i svaku interpretaciju *I* postoji istinitosna vrednost F^I koju interpretacija *I* dodeljuje formuli *F* i definiše se rekurzivno kao:
  - Za svaki atom *F*, $F^I = I(F)$,
  - $\bot^I$ = FALSE, T^I = TRUE,
  - $(\neg F)^I = \neg(F^I)$,
  - $(F\ b\ G)^I = b(F^I, G^I)$ za svaki b binarni veznik.
- Ako je signatura (skup simbola) konačna, onda je i skup interpretacija konačan skup, a vrednosti F^I za sve interpretacije *I* mogu se predstaviti konačnom tabelom, koja se zove ***istinitosna tabela*** formule *F*.

Interpretacija formula

- Za formulu *F* koja je istinita u interpretaciji *I*, $F^I=TRUE$, kažemo da intepretacija *I* ***zadovoljava*** *F*, odnosno da je *I* model *F*

$$I \models F$$

- Za rečenicu/formulu *F* tvrdimo da je ***zadovoljiva*** akko postoji barem jedna njena intrepretacija I za koju važi da je $F^I=TRUE$
- Za rečenicu/formulu *F* tvrdimo da ***nije zadovoljiva*** akko ne postoji ni jedna njena intrepretacija za koju vazi da je $F^I=TRUE$ *(FALSE za sve interpretacija)*
  - Kontradiktorna rečenica/formula *F* ***nije zadovoljiva***
  - Svet nije nikada takav kakav se opisuje
    npr. *"Kiša pada i kiša ne pada"* bi predstavili $P \wedge \neg P$

Tautologija i ekvivalencija

- Za rečenicu/formulu *F* tvrdimo da je ***validna*** ili ***uvek istinita*** *ili* ***tautologija*** *ili* ***uvek zadovoljiva,*** akko je F^I=*TRUE* tačno za svaku moguću interpretaciju *I*
- Formula *F* je *tautologija* ako svaka moguća interpretacija *I* zadovoljava *F*
- *Tautologija* je rečenica/formula koja je *istinita u svakoj interpretaciji*
  - *bez obzira na realno stanje ili semantiku sveta koji se predstavlja propozicionom logikom*
    npr. "Kiša pada ili kiša ne pada" bi predstvaili $P \vee \neg P$
- Dve formule su ***ekvivalentne*** ako su zadovoljene u istim interpretacijama
- Dve formule *F* i *G* su ekvivalentne akko je $F \leftrightarrow G$ tautologija

Skup formula

- U propozicionoj logici **baza znanja** predstavlja se **skupom formula**
- Za **skup formula *Γ*** kažemo da je ***zadovoljiv*** ukoliko postoji makar **jedna interpreacija** koja **zadovoljava sve formule u *Γ***
- Za formulu *F* kažemo da je ***logička posledica*** skupa formula *Γ* (*Γ* |= *F*), odnosno da *F sledi* iz *Γ* akko svaka interpretacija koja zadovoljava sve formule iz *Γ* , zadovoljava i *F* (objašnjenje u nastavku)

Predstavljanje znanja propozicionom logikom

- Za predstavljanje znanja pomoću propozicionih formula biramo propozicionu signaturu σ koja je takva da **interpretacije propozicione signature σ odgovaraju stanjima sistema** koji hoćemo da opišemo.
  - Svaka formula definisana za σ predstavlja uslove stanja
  - Skup formula možemo da posmatramo kao bazu znanja
  - Ukoliko je formula F logička posledica baze znanja Γ, onda uslovi izraženi sa F slede iz znanja predstavljenog pomoću Γ.

Primer

- Pera, Raša i Steva dele kancelariju
- Atom *p* znači da je Pera u kancelariji, atom *r* znači da je Raša u kancelariji, a *s* znači da je Steva u knacelariji
- Iz baze znanja *{p, r}* ne sledi ni s ni ¬*s*
  - **Semantika propozicione logike ne uključuje pretpostavku zatvorenog sveta!**
- Ukoliko bismo u bazu znanja dodali formulu ¬*p* ∨ ¬*r* ∨ ¬*s* (makar jedna osoba je van kancelarije), onda bi formula ¬*s* sledila iz baze znanja

Pravila rezonovanja

- *Logičko rezonovanje (**prirodna dedukcija**)* je proces koji se koristi kako bi se kreirale nove rečenice/formule koje logički proizilaze iz datog skupa formula G.
- Podsećanje: Za formulu *F* kažemo da je *logička posledica* skupa formula *Γ* (*Γ* |= *F*), odnosno da *F sledi* iz *Γ* akko svaka interpretacija koja zadovoljava *Γ* zadovoljava i *F*

Prirodna dedukcija

- Formalizuje proces uvođena i isključivanja pretpostavki
- Uobičajena je u matematičkim dokazima
- Prirodna dedukcija za propozicinu logiku određuje dozvoljene zaključke oblika $\Gamma \rightarrow F$, gde je Γ skup formula, a F formula
- $\Gamma \rightarrow F$ se zove **sekvent**

Pravila rezonovanja u SBZ

- Rezonovanje u SBZ je ispravno (***sound***) ako svaka nova formula koja je nastala na osnovu zaključivanja je i logička posledica baze znanja (nikada se ne dolazi do zaključaka koji ne slede iz baze znanja).
- Rezonovanje u SBZ je kompletno (***complete***) ako se iz SBZ mogu proizvesti sve formule koje se logički mogu rezonovati iz baze znanja (pronalaze se svi zaključci koji sledi iz baze znanja).

Sound pravila zaključivanja

- Primeri

RULE	PREMISE	CONCLUSION
Modus Ponens	A, A → B	B
And Introduction	A, B	A ^ B
And Elimination	A ^ B	A, B
Double Negation Elimination	~~A	A
Unit Resolution	A v B, ~B	A
Resolution	A v B, ~B v C	A v C
Hypothetical Syllogism	A → B, B → C	A → C

- Na osnovu istinitosne tabele svako se pravilo može dokazati da je sound.
- Or Introduction, Or Elimination, Implication Introduction, Implication Elimination, Negation Introduction, Negation Elimination

Modus Ponens

- Eliminiše →
- X → Y, X

_____

Y

  - Ako padne kiša, tada će ulice biti mokre.
  - Pada kiša.
  - Dedukcijom se dolazi do zaključka: Ulice će biti mokre.
- Potvrđuje se očekivanje

Modus Tollens

- Eliminiše →
- X → Y, ¬Y

_____

¬ X

  - Ako padne kiša, tada će ulice biti mokre.
  - Ulice nisu mokre.
  - Dedukcijom se dolazi do zaključka: Ne pada kiša.
- Oprez: izbeci propust potvrđivanja zaključka
  - Ako padne kiša, tada će ulice biti mokre.
  - Ulice su mokre.
  - Dedukcijom se ne može doći do zaključka: Pada kiša
    - Ulice mogu biti mokre iz nekog drugog razloga

Hypothetical Syllogism

- Uvezuje premise i pretpostavke da bi dedukovali zaključak
- $X \rightarrow Y, Y \rightarrow Z$

$X \rightarrow Z$

  - Ako se poveća plata, tada ću imati više novca
  - Ako imam više novca, tada ću moći više stvari da kupim
  - Dedukcijom se dolazi do zaključka: Ako se poveća plata, tada ću moći više stvari da kupim

Resolution

- X v Y, ¬Y v Z

_____

X v Z

  - Osnova za mehanizam zaključivanja u Prologu

Soundness of modus ponens

A	B	A → B	OK?
True	True	True	√
True	False	False	√
False	True	True	√
False	False	True	√

Soundness of the resolution inference rule

α	β	γ	$\alpha \vee \beta$	$\neg\beta \vee \gamma$	$\alpha \vee \gamma$
False	False	False	False	True	False
False	False	True	False	True	True
False	True	False	True	False	False
False	True	True	True	True	True
True	False	False	True	True	True
True	False	True	True	True	True
True	True	False	True	False	True
True	True	True	True	True	True

Dokaz

- Signatura:
  - Vr znači “Vrelo je”
  - Vl znači “Vlažno je”
  - Pk znači “Pada kiša”
- Neka sa SBZ sastoji od pravila i fakta(premisa):
  - $(Vr \land Vl) \rightarrow Pk$
    “Ako je vrelo i vlažno, tada pada kiša”
  - $Vl \rightarrow Vr$
    “Ako je vlažno, tada je vrelo”
  - Vl
    “Vlažno je”
- Želimo da dokažemo da sledi **pada kiša**

Dokaz

1. Vl	Premisa	“Vlažno je”
2. Vl → Vr	Premisa	“Ako je vlažno, tada je vrelo”
3. Vr	Modus Ponens(1,2)	“Vrelo je”
4. (Vr ∧ Vl) → Pk	Premise	“Ako je vrelo i vlažno, tada pada kiša”
5. Vr ∧ Vl	And Introduction(1,3)	“Vrelo je i vlažno je”
6. Pk	Modus Ponens(4,5)	“Pada kiša”

Entailment i derivation

- Entailment: SBZ |= Q
  - Formula Q je posledica (**entailed**) SBZ tj. njegovog skupa premisa i pretpostavki, akko ne postoji situacija u kojoj je Q netačno dok su sve ostale premise i pretpostavke u SBZ tačne.
  - Formula Q je posledica (**entailed**) SBZ tj. njegovog skupa premisa i pretpostavki, akko je zaključak Q tačan u svakoj situaciji u kojoj su sve premise i pretpostavke u SBZ tačne.
- Derivation: SBZ |- Q
  - Q se izvodi (**derive**) iz SBZ ako postoji dokaz koji se sastoji od sekvence validnih (u svakoj interpretaciji tačnih) koraka rezonovanja koji kreću od premisa i pretpostavki a rezultuju sa formulom Q

Soundness i Completeness

- Soundness (ispravnost): ako je SBZ |- Q tada SBZ |= Q
  - Ako je Q izvedeno iz SBZ korišćenjem rezonovanja nad skupom premisa i pretpostavki, tada se Q prosledica SBZ
  - Pocedura izvođenja Q iz SBZ se naziva dokaz
- Completeness: ako je SBZ |= Q tada SBZ |- Q
  - Ako je Q posledica SBZ tj. njegovog skupa premisa i pretpostavki, tada Q se može izvesti iz SBZ korišćenjem pravila razonovanja.
  - Rezonovanje je kompletno ako može da se nađe dokaz za svaki zaključak koji je posledica SBZ

Eksplicitne definicije

- Neka je *Γ* baza znanja za propozicionu signaturu *σ (sadrži simbole npr. a,b,c)*
- Proširiti *Γ eksplicitnom definicijom* znači u *σ* uvesti novi atom (simbol npr. *d)*, a u *Γ* dodati formulu ekvalencije oblika $d \leftrightarrow F$, gde je *F* formula za signaturu *σ* bez novog simbola
- Ovim smo uveli novi atom i pomoću već postojećih atoma u *σ* definisali taj atom

Eksplicitna definicija – primer

- Pretpostavimo da imamo bazu znanja iz prošlog primera

$\sigma = \{p, r, s\}$, $\Gamma = \{p, r\}$

- Eksplicitnu definiciju atoma *d* koji znači „Raša i Steva su u kancelariji“ možemo uvesti kao:

$\sigma' = \{p, r, s, \boldsymbol{d}\}$, $\Gamma' = \{p, r, \boldsymbol{d \leftrightarrow (r \wedge s)}\}$.

- U eksplicitnoj definiciji, atom je skraćeni oblik formule kojom se atom definiše

Pravila rezonovanja u propozicionoj logici

$$(\wedge I)\ \frac{\Gamma \Rightarrow F \quad \Delta \Rightarrow G}{\Gamma, \Delta \Rightarrow F \wedge G} \qquad (\wedge E)\ \frac{\Gamma \Rightarrow F \wedge G}{\Gamma \Rightarrow F} \quad \frac{\Gamma \Rightarrow F \wedge G}{\Gamma \Rightarrow G}$$

$$(\vee I)\ \frac{\Gamma \Rightarrow F}{\Gamma \Rightarrow F \vee G} \quad \frac{\Gamma \Rightarrow G}{\Gamma \Rightarrow F \vee G} \qquad (\vee E)\ \frac{\Gamma \Rightarrow F \vee G \quad \Delta_1, F \Rightarrow H \quad \Delta_2, G \Rightarrow H}{\Gamma, \Delta_1, \Delta_2 \Rightarrow H}$$

$$(\rightarrow I)\ \frac{\Gamma, F \Rightarrow G}{\Gamma \Rightarrow F \rightarrow G} \qquad (\rightarrow E)\ \frac{\Gamma \Rightarrow F \quad \Delta \Rightarrow F \rightarrow G}{\Gamma, \Delta \Rightarrow G}$$

$$(\neg I)\ \frac{\Gamma, F \Rightarrow \bot}{\Gamma \Rightarrow \neg F} \qquad (\neg E)\ \frac{\Gamma \Rightarrow F \quad \Delta \Rightarrow \neg F}{\Gamma, \Delta \Rightarrow \bot}$$

$$(C)\ \frac{\Gamma \Rightarrow \bot}{\Gamma \Rightarrow F}$$

$$(W)\ \frac{\Gamma \Rightarrow \Sigma}{\Gamma, \Delta \Rightarrow \Sigma}$$

Pravila rezonovanja

- Pravila se uglavnom mogu podeliti u dve grupe:
  - Pravila uvođenja (leva kolona) – definišu kako da se *izvede* formula zadate sintaktičke forme
    - Na primer, uvođenje konjunkcije ($\wedge I$) pokazuje kako možemo da izvedemo konjunkciju, ako su zadovoljene oba uslova konjunkcije (formule u bazama znanja Γ i Δ)
  - Pravila eliminacije (desna kolona) – pokazuju kako se *koriste* formule određene sintaktičke forme
    - Na primer, eliminacija konjunkcije ($\wedge E$) pokazuje da konjunkcija može da se koristi da bi se zaključio bilo koji termin konjunkcije
- Poslednja dva pravila su pravlo kontradikcije (*C*) i pravilo slabljenja (*W*)

Primer

- $\Gamma = \{\neg p, q \rightarrow r, p \vee q\}$
- $\Gamma' = \{\neg p, q \rightarrow r, p \vee q, \boldsymbol{p}\}$
  - Uvođenje formule *p* je u kontradikciji sa $\neg p$, jer bi onda na osnovu uvođenja konjunkcije bilo moguće reći $\neg p \wedge p$ što nikada nije TAČNO
  - Unit Resolution bi za $\neg p, p \vee q$ zaključilo *q*
  - U svetlu formule *q*, $q \rightarrow r$ *bi* na osnovu Modus Ponens zaključili da sledi *r*

Dokaz

1.		$\neg p$	$\Rightarrow$ $\neg p$	— axiom
2.		$q \to r$	$\Rightarrow$ $q \to r$	— axiom
3.		$p \vee q$	$\Rightarrow$ $p \vee q$	— axiom
4.		p	$\Rightarrow$ p	— axiom
5.		$p, \neg p$	$\Rightarrow$ $\bot$	— by $(\neg E)$ from 4, 1
6.		$p, \neg p$	$\Rightarrow$ r	— by (C) from 5
7.		q	$\Rightarrow$ q	— axiom
8.		$q, q \to r$	$\Rightarrow$ r	— by $(\to E)$ from 7, 2
9.	$p \vee q, \neg p, q \to r$		$\Rightarrow$ r	— by $(\vee E)$ from 3, 6, 8
10.		$\neg p, q \to r$	$\Rightarrow$ $(p \vee q) \to r$	— by $(\to I)$ from 9

Osnovne i izvedene forme argumanata

- Pravila rezonovanja u propozicionoj logici srodna su formama argumenata

Name	Sequent
Modus Ponens	$((p \to q) \land p) \vdash q$
Modus Tollens	$((p \to q) \land \neg q) \vdash \neg p$
Hypothetical Syllogism	$((p \to q) \land (q \to r)) \vdash (p \to r)$
Disjunctive Syllogism	$((p \lor q) \land \neg p) \vdash q$
Constructive Dilemma	$((p \to q) \land (r \to s) \land (p \lor r)) \vdash (q \lor s)$
Destructive Dilemma	$((p \to q) \land (r \to s) \land (\neg q \lor \neg s)) \vdash (\neg p \lor \neg r)$
Bidirectional Dilemma	$((p \to q) \land (r \to s) \land (p \lor \neg s)) \vdash (q \lor \neg r)$
Simplification	$(p \land q) \vdash p$
Conjunction	$p, q \vdash (p \land q)$
Addition	$p \vdash (p \lor q)$
Composition	$((p \to q) \land (p \to r)) \vdash (p \to (q \land r))$
De Morgan's Theorem (1)	$\neg(p \land q) \vdash (\neg p \lor \neg q)$
De Morgan's Theorem (2)	$\neg(p \lor q) \vdash (\neg p \land \neg q)$
Commutation (1)	$(p \lor q) \vdash (q \lor p)$
Commutation (2)	$(p \land q) \vdash (q \land p)$
Commutation (3)	$(p \leftrightarrow q) \vdash (q \leftrightarrow p)$
Association (1)	$(p \lor (q \lor r)) \vdash ((p \lor q) \lor r)$
Association (2)	$(p \land (q \land r)) \vdash ((p \land q) \land r)$
Distribution (1)	$(p \land (q \lor r)) \vdash ((p \land q) \lor (p \land r))$
Distribution (2)	$(p \lor (q \land r)) \vdash ((p \lor q) \land (p \lor r))$
Double Negation	$p \vdash \neg\neg p$
Transposition	$(p \to q) \vdash (\neg q \to \neg p)$
Material Implication	$(p \to q) \vdash (\neg p \lor q)$
Material Equivalence (1)	$(p \leftrightarrow q) \vdash ((p \to q) \land (q \to p))$
Material Equivalence (2)	$(p \leftrightarrow q) \vdash ((p \land q) \lor (\neg p \land \neg q))$
Material Equivalence (3)	$(p \leftrightarrow q) \vdash ((p \lor \neg q) \land (\neg p \lor q))$
Exportation	$((p \land q) \to r) \vdash (p \to (q \to r))$
Importation	$(p \to (q \to r)) \vdash ((p \land q) \to r)$
Tautology (1)	$p \vdash (p \lor p)$
Tautology (2)	$p \vdash (p \land p)$
Tertium non datur (Law of Excluded Middle)	$\vdash (p \lor \neg p)$
Law of Non-Contradiction	$\vdash \neg(p \land \neg p)$

Prirodna dedukcija

- Opisani sistem dedukcije je
  - Ispravan (sound)– nikada se ne dolazi do zaključaka koji ne slede iz baze znanja
  - Kompletan (complete)– pronalazi se svaki zaključak koji sledi iz baze znanja

Metode zaključivanja

- Dedukcija – sound
  - zaključci slede iz premisa
- Indukcija – unsound
  - na osnovu specifičnih slučajeva (primera) donose se generalni zaključci

Mane propozicione logike

- Teško je identifikovati individuale npr. Mako, Jovan,... obično se simbolima predstavljaju koncepti
- Još teže je definisati svojstva individuala ili relacije između individuala npr. Marko je visok 175cm, Mako i Jovan su najbolji prijatelji,..
- Generalizacija, definisanje paterna ne može lako da se predstavi npr. svi slonovi su sivi
- First-Order-Logic (logika prvog reda) je dovoljno ekspesivna da predstavi pomenute situacije. Dozvoljava definisanje varijabli, relacija, funkcija, vrednosti
  - "Svi slonovi su sivi": $\forall$ x (elephant(x) $\rightarrow$ gray(x))
  - "Postoje beli lavovi": $\exists$ x (lion(x) ^ white(x))

Mane propozicione logike

- Propozicionom logikom se samo ustanovljava postojanje ili nepostojanje fakta iz relanog sveta
- Ima jednostavnu sintaksu i jednostavnu semnatiku.
- Veoma brzo postaje nepraktična, ako je potrebno opisati složenije situacije iz realnog sveta

Pregled logičkih jezika

- propositional logic
  - facts
  - true/false/unknown
- **first-order logic**
  - **facts, objects, relations**
  - **true/false/unknown**
- temporal logic
  - facts, objects, relations, times
  - true/false/unknown
- probability theory
  - facts
  - degree of belief [0..1]
- fuzzy logic
  - degree of truth
  - degree of belief [0..1]

Logika prvog reda

- Dok Propoziciona logika pretpostavlja da se svet sastoji od činjenica, Logika prvog reda predpostavlja da se svet sastoji od:
  - Objekata: ljudi, kuće, utakmice,..
  - Relacija: brat, sestra, živi_u, veći_nego, deo,..
  - Funkcija: boja_kože, tip_frizure, broj_točkova,...

- **First-order logic** je još naziva i **first-order predicate calculus** i **predicate logic**

Predicate Calculus

- Poširuje Popozicionu logiku sa predikatima (**predicates**) i kvanifikatorima (**quantifiers**).
  - Predikati su **izjave o samim objektima** ili **izjave kojima se definišu relacije između objekata**
    - je_ptica
    - ima_krila
    - teži_od_prosečne_težine
    - teži_nego
    - žive_u_istom_gradu
  - Kvanitfikatori se postavljaju za predikate
    - $\forall$ za svako (univerzalni quantifier)
    - $\exists$ postoji (egzistencijalni quantifier)
- Predicate Calculus može definisati izjave kao:
  - $\forall$ XYZ: žive_u_istom_gradu(X,Y) $\wedge$ žive_u_istom_gradu(Y,Z) $\rightarrow$ žive_u_istom_gradu(X,Z)
  - XYZ su **varijable kojima se označavaju objekti (individuali)**
  - Izjava žive_u_istom_gradu(X,Y) vraća vrednost TRUE ili FALSE

Predicate Calculus

- Predicate Calculus je uopštena ali ne dovoljno moćna
- Može se **proširiti** sa **funkcijama**
  - broj_točkova
  - boja
  - visina
- Funkcije ne vraćaju TRUE ili FALSE, već vraćaju objekat
- Može se **proširiti** sa **predikatskom jednakošću**
  - Dva objekata su jednaka akko se ne mogu razlikovati primenom svih predikata i funkcija

Logika prvog reda

- Predicate calculus sa svim pomenutim proširenjima je varijanta logike prvog reda
- Logika prvog reda dozvoljava definisanje kvanifikatora za varijable, ali ne i za predikate i funkcije

Logika prvog reda

- Dodatni koncepti u odnosu na Propozicionu logiku
  - Kompleksni objekti
    - termovi
  - Relacije
    - Predikatske (izjava o objektu ili relacija između objekata)
    - Funkcijske
  - Sintaksa
  - Semantika
  - Pravila rezonovanja
  - Korišćenje

Objekti

- Entiteti iz realnog sveta
  - ljudi, automobili, kompjuteri, ...
- Često korišćeni koncepti
  - boje, svetlo, novac, težina, zavist,...
- Propertiji
  - Opisuju specifične aspekte objekata
    - zelen, težak, vidljiv,...
  - koriste se da bi se napravila razlika između objekata

Relacije

- Ostvarivaju konekciju između objekata
- Njihova semantika zavisi od objekta nad kojima se ona primenjuje

- Funkcije su specijalni tip relacija
  - Nedovsmislene: jedan izlaz za dati ulaz

Sintaksa

- Bazira se na rečenicama/izjavama/formulama, koje su dosta kompleksnije nego u Propozicionoj Logici
  - rečenica sadrži termove koji predstavljaju objekte
  - Postoje različiti tipovi simbola
- Konstantni simboli: A, B, C, Marko, 3,...
  - Opisuju vrednosti
  - Predstavljaju jedinstvene objekte (u određenom kontekstu)
- Predikatski simboli: Mlađi_Nego, Komšije,...
  - Opisuju relaciju između objekata
- Funkcijski simboli: Broj_točkova, Visina, Roditelj_Od
  - Dati objekat je u relaciji sa tačno određenim drugim objektom

Semantika

- Pruža interpretaciju za bazične konstrukte
  - obično se određuje značenjem imena simbola
- Konstantni simboli:
  - njihova intrepretacija ukazuje na objekte iz realnog sveta
- Predikatski simboli:
  - njihova intrepretacija ukazuje na specifičnu relaciju u modelu između objekata
- Funkcijski simboli:
  - Identifikuju objekat koji se odnosi na ntorku drugih objekata

Termovi

- Logički izrazi koji definišu objekte
- Objektne konstante i objektne varijable su termovi
- Kompleksni termovi se konstruišu od funkcijskih simbola i jednostvanih termova, koji se uokviruju zagradama

Logika prvog reda

- U logici prvog reda signatura se sastoji od dva skupa simbola:
  - *Funkcijskih simbola* npr. **broj_zajedničkih_prijatelja**(x,y,z) - f^n_0, f^n_1, f^n_2, ...
  - *Predikatskih simbola* npr. **je_ptica**(x) - P^n_0, P^n_1, P^n_2, ...
- Svaki od ovih simbola ima dodeljenu *arnost* (arity) – relacija između n elemenata
  - Predikstaski simboli arnosti 0 su *propozicione konstante*
  - Funkcijski simboli arnosti 0 su *objektne konstante*

Logika prvog reda

- *Objektne varijable* su elementi koje čine nizovi objektnih simbola
  - Na primer *x, y, z, x*1*, y*1*, z*1
- *Termovi* signature σ su sačinjeni od ***objektnih varijabli, objektnih konstanti*** i ***funkcijskih simbola*** za σ
- ***Atomična formula* signature σ** je izraz oblika $P(t_1, \ldots, t_n)$, gde je P predikat arnosti n, a t_i term signature σ
- ***Formule* logike prvog reda formiraju se od atomičnih formula upotrebom *propozicionih veznika*** ($\wedge$, $\vee$, $\rightarrow$,..) i ***kvantifikatora*** $\forall$ i $\exists$.

Jednakost termova

- $term_1 = term_2$ su jednaka za datu intrepretaciju akko $term_1$ i $term_2$ predstavljaju isti objekat

- Npr., definicija *Sibling* u odnosu na termove *Parent*:

$$\forall x,y\ Sibling(x,y) \Leftrightarrow [\neg(x = y) \wedge \exists m,f \neg (m = f) \wedge Parent(m,x) \wedge Parent(f,x) \wedge Parent(m,y) \wedge Parent(f,y)]$$

Logika prvog reda

- Konstante: KingJohn, RichardTheLionHart, 2, TCD, …
- Predikati: Brother, >, …
- Fukcije: Sqrt, Length, LeftLegOf,…
- Objektne varijable: x, y, a, b,…
- Propozicioni veznici: $\wedge$, $\vee$, $\neg$, $\rightarrow$, $\leftrightarrow$
- Veznik jednakosti: =
- Kvantifikatori: $\exists$, $\forall$

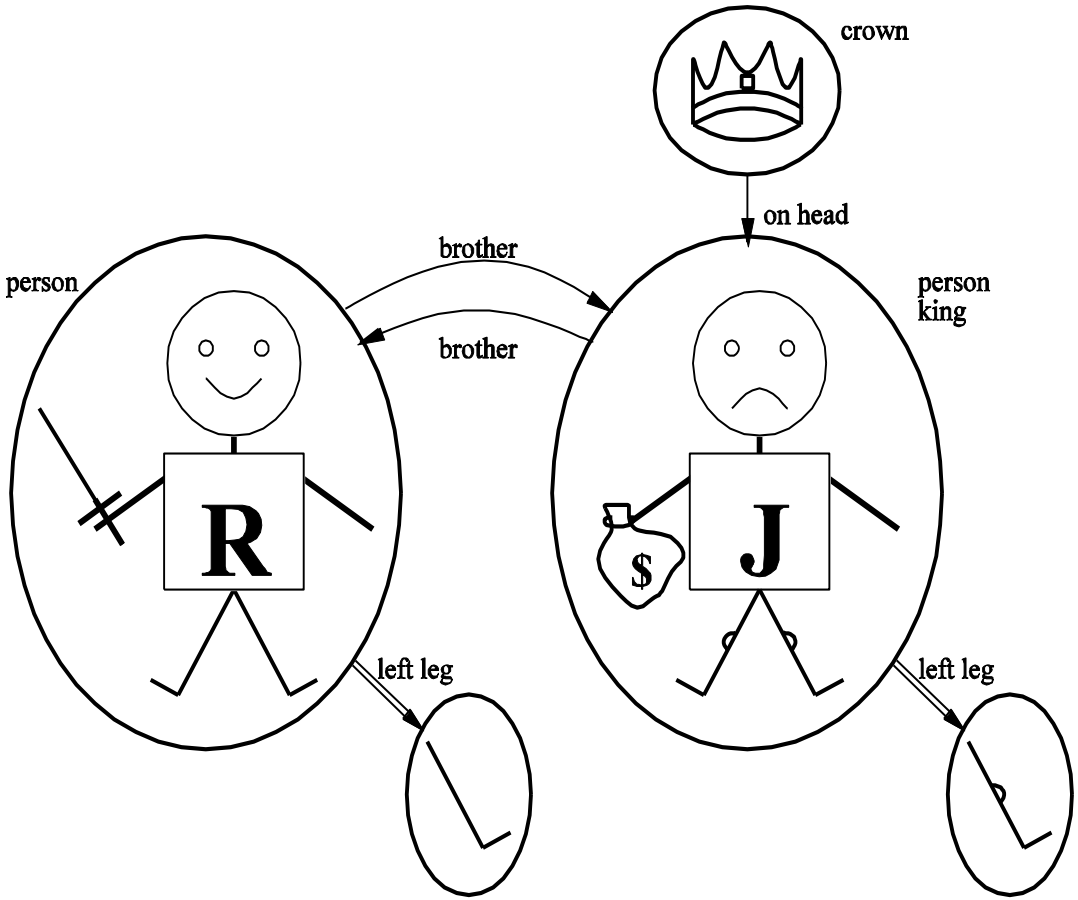
Primer

Formule

Brother(KingJohn, RichardTheLionHart)

> (Length (LeftLegOf(RichardThe…)
(Length (LeftLegOf(KingJohn)

Sibling(KingJohn, Richard…) ⇒
Sibling(Richard…, KingJohn)



Kvantifikatori

- Univerzalni kvanitifikator
  - $\forall x$ *At(x, Hogwarts)* $\Rightarrow$ *Smart(x)*
    - "Everyone at *Hogwarts* is Smart"
    - *Being at Hogwarts implies you are smart*

- $\forall$x *P* se interpretira kao $\forall$x važi da *P* tačno u modelu m akko je P tačno za savko x koje predstavlja sve moguče objekte u modelu m

- Grubo posmatrano, prethodno je ekvivalentno konjukciji svih mogućih instanci za P za sve moguće vrednosti objektnih varijabli x

$$
\begin{aligned}
& \text{At(KingJohn, } \textit{Hogwarts}\text{)} \Rightarrow \text{Smart(KingJohn)} \\
\wedge\ & \text{At(Richard, } \textit{Hogwarts}\text{)} \Rightarrow \text{Smart(Richard)} \\
\wedge\ & \text{At(Robin, } \textit{Hogwarts}\text{)} \Rightarrow \text{Smart(Robin)} \\
\wedge\ & \ldots
\end{aligned}
$$

Česta greška

- Uobičajeno se $\Rightarrow$ koristi za definisanje relacije u kojoj se koristi kvantifikator $\forall$
- Česta greška: korištenje $\wedge$ za definisanje relacije u kojoj se koristi kvantifikator $\forall$:

  $\forall x$ At(x, *Hogwarts*) $\wedge$ Smart(x)

  "Everyone is at *Hogwarts* and everyone is smart"

Kvanitfikator

- Egzistencijalni kvantifikator
  - $\exists x$ *At(x, Hogwarts)* $\wedge$ *Smart(x)*
    - "Someone at *Hogwarts* is Smart"
    - *There exists someone who is both at Hogwarts and Smart*

- $\exists x$ *P* se interpretira kao $\exists x$ tako da je *P* tačno u modelu m akko je *P* tačno za neko x koje predstavlja neki moguči objekat u modelu

- Grubbo posmatrano, prethodno je ekvivalentno disjunkciji svih mogućih instanci za P

  At(KingJohn, *Hogwarts*) $\wedge$ Smart(KingJohn)
  $\vee$ At(Richard, *Hogwarts*) $\wedge$ Smart(Richard)
  $\vee$ At(Robin, *Hogwarts*) $\wedge$ Smart(Aegean)
  $\vee$ ...

Česta greška

- Uobičajeno se $\wedge$ koristi za definisanje relacije u kojoj se koristi kvantifikator $\exists$
- Česta greška: korištenje $\Rightarrow$ za definisanje relacije u kojoj se koristi kvantifikator $\exists$:

  $\exists x$ At(x, *Hogwarts*) $\Rightarrow$ Smart(x)

  "Someone in *Hogwarts* is smart"
  is true even for the case if there is anyone who is not at *Hogwarts*!

  zbog veze implikacije

Osobine kvantifikatora

- $\forall x\ \forall y$ **je isto kao** i $\forall y\ \forall x$
- $\exists x\ \exists y$ **je isto kao i** $\exists y\ \exists x$

- $\exists x\ \forall y$ **nije isto kao** i $\forall y\ \exists x$
- $\exists x\ \forall y$ Loves(x,y)
  - "There is a person who loves everyone in the world"
- $\forall y\ \exists x$ Loves(x,y)
  - "Everyone in the world is loved by at least one person"

- **Dualnost kvantifikatora**: jedan može da se izrazi pomoću drugog
- $\forall x$ Likes(x,IceCream) $\neg\exists x\ \neg$Likes(x,IceCream)
- $\exists x$ Likes(x,Broccoli) $\neg\forall x\ \neg$Likes(x,Broccoli)

Korišćenje logike prvog reda

Domen problema rodbinske veze, definisanje formula/rečenica

- Brothers are siblings

  $\forall x,y\ Brother(x,y) \Leftrightarrow Sibling(x,y)$

- One's mother is one's female parent

  $\forall m,c\ Mother(c) = m \Leftrightarrow (Female(m) \wedge Parent(m,c))$

- “Sibling” is symmetric

  $\forall x,y\ Sibling(x,y) \Leftrightarrow Sibling(y,x)$

Varijable

- Pojava varijable *v* u formuli *F* je ***vezana*** ukoliko pripada podformuli od *F* oblika ∀*v G* ili ∃*v G.*
- Inače je **pojava varijable *slobodna***
- Varijabla je *slobodna* ako nije kvantifikovana sa ∀ ili ∃
- ∀y P(x,y), varijabla *x* je slobodna dok je y vezana
- Ukoliko je makar jedna *pojava varijable* slobodna (nekvatifikovana) onda je i **formula slobodna**

Varijable

- Jedna formula može da ima i slobodne i vezane pojave varijable:

$$P(x) \wedge \exists x Q(x).$$

- Ova formula ekvivalentna je sa:

$$P(x) \wedge \exists x_1 Q(x_1).$$

- I ima značenje:
  - *x* ima svojstvo *P* i postoji neki objekat koji ima svojstvo *Q*

Čemu služe vezane varijable?

- Hoćemo da kažemo:

1. Svi vole picu
2. Svako ko voli picu, voli i sladoled

Zaključak: Svi vole sladoled

Čemu služe vezane varijable?

1. Svi vole picu (dato-premisa)
2. Svako ko voli picu, voli i sladoled (dato-premisa)
3. Uzmimo bilo koju osobu, neka se zove **Pera**. **Pera** voli picu (sledi iz 1)
4. Ako **Pera** voli picu, voli i sladoled (sledi iz 2)
5. Pera voli sladoled (sledi iz 3 i 4 po modus ponensu)
6. Obzirom da je **Pera** „bilo ko“, ono što važi za njega važi i za svakog drugog.

Čemu služe vezane varijable?

- Treba nam simbol (ni objektna konstanta ni varijabla) koji će da znači „bilo koja osoba, na primer neka se zove Pera“ i koja će biti ista u formulama u bazi znanja

1.	$\forall xFx$	Premisa
2.	$\forall x(Fx \rightarrow Gx)$	Premisa
3.	Fa	Iz 1., Univ. Instancijacija
4.	$(Fa \rightarrow Ga)$	Iz 2., Univ. Instancijacija
5.	Ga	Iz 3, 4 po modus ponensu
6.	$\forall xGx$	Iz 5, Univ. Generalizacija

Univerzalno zatvaranje

- **Formula je *zatvorena*** ukoliko nema ni jednu slobodnu varijablu
- *Univerzalno zatvaranje* formule *F* je iskaz $\forall v_1, \ldots, v_n F$ gde su $v_1, \ldots, v_n$ sve moguće varijable formule *F*

Supstitucija

- Rezultat supstitucije varijable *v* termom *t* u formuli *F* je formula koja se dobije kada svaku slobodnu pojavu varijable *v* zamenimo termom *t*
- Tipično, formula *F* u kojoj treba izvršiti supstituciju varijable *v* piše se kao *F(v)*, a formula u kojoj je supstitucija izvršena termom *t* piše se kao *F(t)*

Supstituable

- Term *t* je *supstituabla* varijable *v* u formuli *F* ukoliko, za svaku varijablu *w* koja se pojavljuje u *t*, ne postoji ni jedna podformula u *F* oblika ∀*wG* ili ∃*wG* koja sadrži *v* i koja je slobodna u *F*

∃*x(f (x) = v)*

v=t(w1,w2,w3)

∃*x(f (x) = t(w1,w2, w3))*

- Ukoliko je ovaj uslov narušen, supstitucijom *t* za *v* često dobijamo rezultate koji nemaju očekivano značenje

∃*w2(f (w2) = v)*

v=t(w1,w2,w3)

~~∃*w2(f (w2) = t(w1,w2,w3))*~~

Primer

- $\exists x(f(x) = y)$
  - Postoji neko x za koje je vrednost funkcije f jednaka y
  - [*y* pripada kodomenu funkcije *f*]
- Ispravno - Uvodimo supstituciju $y = g(a, z)$
  - $g(a,z)$ je supstituabla za *y*
- $\exists x(f(x) = g(a, z))$
  - [$g(a, z)$ pripada kodomenu funkcije *f*]
- Neispravno - Uvodimo supstituciju $y = g(a, \mathbf{x})$
  - $g(a,\mathbf{x})$ nije supstituabla za *y*
- $\exists x(f(x) = g(a, x))$
  - [Postoji neko *x* takvo da je vrednost *f* za *x* jednaka vrednosti *g* za *a* i to *x*]
  - Knvatifikator ∃ je „uhvatio“ i x, što nije bio inicijalni plan

više naredni čas

Semantika

- Interpretaciju signature σ čine:
  - Neprazan skup $|I|$, koji se zove *univerzum* (ili *domen*) interpretacije *I*
    - Univerzum je skup objekata o kojima može da se govori
  - Za svaku objektnu konstantu *c* iz σ, element c^I iz $|I|$
    - Objektna konstanta se mapira na objekat iz univerzuma npr. Marko, Jovan,...
  - Za sve funkcijske simbole *f* arnosti *n>0* funkcija f^I iz $|I|^n$ u $|I|$
    - Funkcija n-torki objekata iz univerzuma dodeli objekat iz univerzuma
    - Na primer Monarch(Canada, United Kingdom) = Elizabeth II
  - Za svaku propozicionu konstantu *P* iz σ, element P^I iz skupa {FALSE, TRUE}
  - Za sve predikatske simbole *R* iz σ arnosti $n > 0$, funkcija R^I *iz* $|I|^n$ u {FALSE, TRUE}
    - Predikat dodeli n torki objekata iz univerzuma vrednost tačno ili netačno
    - Na primer President (America, Obama)=TRUE

Semantika

- Semantika logike prvog reda definiše za svaki iskaz/formulu *F* i svaku interpretaciju *I* signature σ istinitosnu vrednost F^I koja je dodeljena iskazu/formuli *F* tom interpretacijom
- **Pažnja!** Ova definicija ne odnosi se na formule koje imaju slobodne varijable!
  - Da li je $\exists x(f(x) = \boldsymbol{y})$ tačno ili netačno ne zavisi isključivo od univerzuma i funkcije *f*. To zaivisi i od **konkretne vrednosti *y*** u univerzumu.

Semantika

- Interpretacija *I zadovoljava* iskaz *F* (I|= *F*) ukoliko je F^I = TRUE
- Iskaz *F* je *logički validan (uvek zadovoljiv, uvek istinit)* ukoliko ga zadovoljava svaka interpretacija
- Dva iskaza su *ekvivalentna* ukoliko su zadovoljeni u istim interpretacijama
- Iskaz koji sadrži slobodne varijable je logički validan ukoliko je njegovo univerzalno zatvaranje logički validno

Semantika

- Baza znanja u logici prvog reda je skup iskaza/rečenica/formula
- Skup iskaza *Γ* (baza znanja) je zadovoljiv ukoliko postoji interpretacija *I* koja zadovoljava sve iskaze iz skupa
- Iskaz *F* je *logička posledica Γ (F* sledi iz *Γ; Γ* |= *F)* ukoliko svaka interpretacija koja zadovoljava *Γ,* zadovoljava i univerzalno zatvaranje *F*

Jedinstveno imenovanje

- Pretpostavimo da hoćemo da prebacimo bazu znanja iz primer iz propozicione logika u logiku prvog reda:

  in(Pera), in(Rasa), in(Steva)

- Postoje interpretacije u kojima se objektne konstante iz skupa {Pera, Rasa, Steva} mapiraju na isti objekat univerzuma
  - Na primer, Pera koristi partizansko ime Rasa
- Kažemo da u **logici prvog reda ne važi jedinstveno imenovanje**
- Ukoliko bismo hteli da naglasimo da ove objektne konstante označavaju različite objekte iz univerzuma morali bismo to eksplicitno da naglasimo

  Pera!=Rasa, Pera!=Steva, Steva!=Rasa

Jedinstveno imenovanje

- Mogli bismo i da zahtevamo da *pretpostavka jedinstvenog imenovanja* važi za signaturu σ:
- $\forall x_1 \ldots x_m y_1 \ldots y_n (f(x_1, \ldots, x_m) \mathrel{!}= g(y_1, \ldots, y_n))$
  - Za sve parove različitih funkcijskih simbola f i g
- $\forall x_1 \ldots x_n y_1 \ldots y_n (f(x_1, \ldots, x_n) = f(y_1, \ldots, y_n) \rightarrow (x_1 = y_1 \wedge \cdots \wedge x_n = y_n))$
  - Za sve funkcijske simbole arnosti veće od 0

Zatvaranje domena

- Posmatrajmo bazu znanja Γ koja odgovara bazi znanja iz primer za propozicionu logiku

$$\neg in(Pera) \vee \neg in(Rasa) \vee \neg in(Steva)$$

- Istu stvar možemo predstaviti i iskazom

$$\exists x \neg in(x)$$

- Da li su ova dva iskaza ekvivalentna?

Zatvaranje domena

- Posmatrajmo bazu znanja Γ koja odgovara bazi znanja iz primer za propozicionu logiku

$$\neg in(Pera) \vee \neg in(Rasa) \vee \neg in(Steva) \quad (1.1)$$

- Istu stvar možemo predstaviti i iskazom

$$\exists x \neg in(x) \quad (1.2)$$

- Ova dva iskaza nisu ekvivalentna
  - Prvi sledi iz drugog
  - Drugi **ne sledi** iz prvog
    - Moguće je da u univerzumu ima još objekata koji nisu imenovani konstantama
    - Moglo bi da se dogodi da postoji još neko ko dolazi u kancelariju

Zatvaranje domena

- Ukoliko hoćemo da „zatvorimo domen“, odnosno da specificiramo da svaki element univerzuma odgovara jednoj od tri navedene objektne konstante (imena) to bismo uradili kao:

$$\forall x(x = Pera \vee x = Rasa \vee x = Steva)$$

Zatvaranje domena

- Navedeno zatvaranje domena garantuje ekvivalenciju *(1.1) i (1.2)*
- Opštije, to zatvaranje domena garantuje ekvivalenciju

$$\forall x F(x) \leftrightarrow F(Pera) \wedge F(Rasa) \wedge F(Steva),$$

$$\exists x F(x) \leftrightarrow F(Pera) \vee F(Rasa) \vee F(Steva)$$

Za svaku formulu *F(x)*

Zatvaranje domena

- Uopšteno, za signaturu σ koja ima konačno mnogo objektnih konstantni $c_1, \ldots, c_n$ i **nema ni jedan funkcijski simbol arnosti veće od 0** *pretpostavka zatvorenog domena* glasi:
- $\forall x(x = c_1 \vee \cdots \vee x = c_n)$

Reifikacija

- Jezik koji smo uveli za opisivanje kancelarija sadrži varijable za osobe, ali ne i varijable za mesta
  - Kažemo da su osobe *reifikovane* u ovom jeziku, a da mesta nisu reifikovana u ovom jeziku
- Mogli bismo proširiti jezik
  - Prevodimo *in* u binarni predikat
  - *in(Pera)* postaje *in(Pera, kancelarija)*

Reifikacija

- Reifikacijom jezik postaje ekspresivniji
  - Možemo, na primer, da kažemo da se svako nalazi na tačno jednom mestu

$$\forall x \exists! p \; in(x, p).$$

- ($\exists! p$ znači *Postoji* ***tačno jedno*** *p*)
- Bez reifikacije mesta to ne bismo mogli da kažemo ko se nalazi tačno u kojoj kancelariji
- Prilikom reifikacije oslonili smo se na činjenicu da **logika prvog reda ne ograničava arnost predikata**, odnosno da možemo da uvedemo novi parametar u predikat.
- Reifikacija omogućuje da logika prvog reda bude proširiva

Eksplicitna definicija predikatskog simbola

- Neka je Γ baza znanja za signaturu σ.
- Proširiti Γ eksplicitnom definicijom *predikatskog simbola P* znači u σ uvesti predikatski simbol P arnosti n i u Γ uvesti iskaz oblika

$$\forall v_1 \ldots v_n (P(v_1, \ldots, v_n) \leftrightarrow F)$$

- Pri tome su $v_1,\ldots,v_n$ različite varijable, a F je formula signature σ
- U stvari, eksplicitna definicija *predikatskog simbola* uvodi novi predikatski simbol za univerzalno kvantifikovane varijable i daje formulu kojoj je ovaj predikatski simbol ekvivalentan
- Uvedeni predikatski simbol je skraćeni zapis za formulu F

Eksplicitna definicija funkcijskog simbola

- Uvođenje novog funkcijskog simbola f u σ
- Proširivanje baze znanja Γ iskazom oblika

$$\forall v_1 \ldots v_n v \, (f(v_1, \ldots, v_n) = v \leftrightarrow F)$$

- Pri čemu su $v_1, \ldots, v_n, v$ različite varijable, a F je iskaz signature σ takava da iz Γ sledi

$$\forall v_1 \ldots v_n \exists ! v F$$

- Ukoliko ne bi važio ovaj uslov, uvođenje nove funkcijske konstante moglo bi da ukloni neke od interpretacija koje zadovoljavaju Γ
  - Nova znanja mogla bi da ranija znanja učine nevažećim
- U stvari, eksplicitnom definicijom funkcijskog simbola uvodi se funkcijski simbol za univerzalno knvatifikovane varijable i iskaz ekvivalentan tom funkcijskom simbolu
  - Za univerzalno kvantifikovane varijable funkcijskog simbola postoji **tačno jedna** vrednost koju funkcija „vraća“

Primer

- U primer kancelarije mogli bismo da uvedemo eksplicitnu definiciju funkcijskog simbola *in*

$$\forall xp(location(x) = p \leftrightarrow in(x, p))$$

Prirodna dedukcija u logici prvog reda

- Važe sva pravila prirodne dedukcije propozicione logike
- Pored toga, uvedena su i pravila za uvođenje i eliminaciju kvantifikatora i rad sa jednakošću (sledeći slajd)
- Kao i propoziciona logika i ovaj sistem je ispravan i kompletan

Prirodna dedukcija u logici prvog reda

$$(\forall I)\ \frac{\Gamma \Rightarrow F(v)}{\Gamma \Rightarrow \forall v F(v)}$$

where v is not a free variable of any formula in Γ

$$(\forall E)\ \frac{\Gamma \Rightarrow \forall v F(v)}{\Gamma \Rightarrow F(t)}$$

where t is substitutable for v in $F(v)$

$$(\exists I)\ \frac{\Gamma \Rightarrow F(t)}{\Gamma \Rightarrow \exists v F(v)}$$

where t is substitutable for v in $F(v)$

$$(\exists E)\ \frac{\Gamma \Rightarrow \exists v F(v) \quad \Delta, F(v) \Rightarrow G}{\Gamma, \Delta \Rightarrow G}$$

where v is not a free variable of any formula in Δ, G

$$\Rightarrow t = t$$

$$(Repl)\ \frac{\Gamma \Rightarrow t_1 = t_2 \quad \Delta \Rightarrow F(t_1)}{\Gamma, \Delta \Rightarrow F(t_2)} \qquad \frac{\Gamma \Rightarrow t_1 = t_2 \quad \Delta \Rightarrow F(t_2)}{\Gamma, \Delta \Rightarrow F(t_1)}$$

Primer 1

1. Svi bernardinci su psi
2. Svi psi su životinje

Zaključak: svi bernardinici su životinje

B(x) – x je bernardinac

P(x) – x je pas

Z(x) – x je životinja

Primer 1

1.	$\forall x\big(B(x) \to P(x)\big)$	Premisa
2.	$\forall x\big(P(x) \to \mathrm{Z}(x)\big)$	Premisa
3.	$B(a) \to P(a)$	Univerzalna instancijacija (1)
4.	$P(a) \to Z(a)$	Univerzalna instancijacija (2)
5.	$B(a) \to Z(a)$	Hipotetski silogizam (3,4)
6.	$\forall x\big(B(x) \to \mathrm{Z}(x)\big)$	Univerzalna generalizacija (5)

Primer 2

1. Svi bernardinici su psi
2. Ako je Reks pas, onda je Reks životinja

- Zaključak: svi bernardinci su životinje

Primer 2

1.	$\forall x\big(B(x) \to P(x)\big)$	Premisa
2.	$P(r) \to Z(r)$	Premisa
3.	$B(r) \to P(r)$	Univerzalna instancijacija (1)
4.	$B(r) \to Z(r)$	Hipotetski silogizam (2,3)
5.	$\forall x\big(B(x) \to \mathrm{Z}(x)\big)$	Univerzalna generalizacija (4)

- **Gde je greška?**

Primer 2

1.	$\forall x\big(B(x) \to P(x)\big)$	Premisa
2.	$P(r) \to Z(r)$	Premisa
3.	$B(r) \to P(r)$	Univerzalna instancijacija (1)
4.	$B(r) \to Z(r)$	Hipotetski silogizam (2,3)
5.	$\forall x\big(B(x) \to \mathrm{Z}(x)\big)$	Univerzalna generalizacija (4)

- **Gde je greška?**
  - Slobodna varijabla r ne sme da se javi u premisama da bi mogla da se upotrebi u univerzalnoj generalizaciji!

Primer 3

- Kako bismo logikom prvog reda predstavili da je funkcija *P* inverzna u odnosu na *Q*?

Primer 3

- Kako bismo logikom prvog reda predstavili da je funkcija *P* inverzna u odnosu na *Q*?

$$\forall xy(Q(x, y) \leftrightarrow P(y, x))$$

Logika drugog reda

- Definicija signature je ista kao i u logici prvog reda
- Sintaksa je bogatija:
  - Pored *objektnih varijabli* imamo i
  - *Funkcijske varijable* arnosti *n>0*
  - *Predikatske varijable* arnosti *n>=0*

Logika drugog reda

- Funkcijske varijable mogu se koristiti za formiranje termova na isti način na koji se koriste i objektne varijable
  - Na primer, ako je α unarna funkcijska varijabla, a c objektna konstanta, onda je $\alpha(c)$ term
- Predikatske varijable se koriste za formiranje atomičnih formula na isti način na koji se koriste i predikatski simboli
- U neatomičnim formulama, funkcijske i predikatske varijable vezuju se kvantifikatorima na isti način kao i objektne varijable
- *To znači da se u logici 2 reda kvanifikatori koriste za objektne varijable, funkcijske varijable i predikatske varijable*

Primer

- Iskaz:

$$\forall\alpha\beta\exists\gamma\ \forall x(\gamma\ (x) = \alpha(\beta(x)))$$

- Znači: za svake dve funkcije α i β postoji neka funkcija γ koja je kompozicija funkcija α i β
- Napomena: U iskazu/formul za loguku drugog reda sve varijable (objektne, funkcijske i predikatske) **su** ograničene/vezane

PL vs FOL vs DL

- Logički sistem je
  - **Sound** – ako rezonovanjem nad netačnim iskazima nikada ne dolazimo tačnih zaključaka
  - **Complete** – svi tačni iskazi mogu se i dokazati
  - **Odlučiv** (**decidable**) – ukoliko postoji efikasna metoda koja (moguće je napisati program koji) određuje da li je formula teorema za zadatu bazu znanja.
- Propoziciona logika je sound, complete, decidable, ali nije dovoljno ekspresivna za predstavljanje složenijeg znanja
- FOL – ekspresvina, sound, complete, ali nije odlučiva
- DL – ekspresivnija od PL, sound, complete, decidable

Deskriptivna logika (DL)

- Familija jezika za predstavljanje terminološkog znanja iz aplikacionog domena na **struktuiran** i **formalno** jasan/razuman način
- Naziv deskriptivna (opisna) logika se koristi jer se domenski pojmovi predstavljaju **pomoću *opisa*** *(deskripcija)* **koncepata**.
  - Deskripcije su izrazi sačinjeni od ***atomičnih koncepata*** (unarnih predikata) i ***atomičnih rola*** (binarnih predikata) korišćenjem konstrukta za povezivanje koncepata i rola.
  - Konstrukti koji se koriste za pravljenje deskripcija zavise od konkretne logike (iz familije deskriptivnih logika)
- Razlikuju se od semantičkih mreža i frejmova po tome što imaju striktno definisanu, formalnu, *logičku semantiku*

Primer

- Ako bi želeli da formalno definišemo kocept:
- “A man that is married to a doctor, and all of whose children are either doctors or professors.”
- Koncept bi mogao da se opiše (deskripcija koncepta):

$$\text{Human} \sqcap \neg\text{Female} \sqcap (\exists \text{married}.\text{Doctor}) \sqcap (\forall \text{hasChild}.(\text{Doctor} \sqcup \text{Professor}))$$

Primer

- Ovaj primer uključuje **bulovske konstrukte**:
  - *Konjunkciju* $\sqcap$ koja se interpretira kao presek skupova
  - *Disjunkciju* $\sqcup$ koja se interpretira kao unija skupova
  - *Negaciju* $\neg$ koja se interpretira kao komplement skupa
- I **konstrukti ograničenja**
  - Egzistencijalno ograničenje $\exists r.C$
  - Ograničenje vrednosti $\forall r.C$

Primer

- Individual (na primer Bob) pripada klasi $\exists married.Doctor$ ako postoji makar jedan indiviual koji je u braku sa Bob(tj. povezan sa Bob kroz ulogu u braku) i koji je doktor (tj. pripada konceptu Doktor).
- Slično, individual (na primer Bob) pripada klasi $\forall hasChild.(Doctor \sqcup Professor)$ ako sva njegova deca (tj. svi individuali povezani sa Bob putem uloge hasChild) su individuali klase Doctor ili klase Profesor (tj. pripadju konceptu Doktor ili Profesor).

Baza znanja

- Opisi/deskripcije koncepta koriste za kreiranje iskaza koje čine **bazu znanja u DL**
- Baza znanja najčešće ima dva dela:
  - TBox – terminološki deo (opisi deskripcija)
  - ABox – činjenični deo

TBox

- Terminološki deo baze znanja (šema)
- Opis relevantnih pojmova/koncepata iz aplikacionog domena tj. domena primene navođenjem
  - Svojstva (propertija) koncepata i rola, i veza među njima
  - Odgovara analogno shemi baze podataka

TBox

- Najjednostvnija forma izraza u TBox bi bio iskaz sa kojom se uvodi naziv (skraćenica) koji se koristi za prestavu kompleksnih deskripcija koncepta
- **Definicije koncepata**
- Uvođenje naziva HappyMan kao skraćenice za deskripciju koncepta za primer od ranije

$$\text{HappyMan} \equiv \text{Human} \sqcap \neg\text{Female} \sqcap (\exists \text{married}.\text{Doctor}) \sqcap (\forall \text{hasChild}.(\text{Doctor} \sqcup \text{Professor}))$$

TBox

- Izražajniji izrazi u TBox dozvoljavaju da u njima navode iskazi opštijih aksioma kao što je:

$$\forall \text{hasChild.Human} \sqsubseteq \text{Human}$$

(jedino ljudi mogu da imaju decu koja su ljudi)

- Za razliku od prošlog primera, ovaj primer **ne definiše koncept** (pojam)
  - Samo ograničava način na koji koncepti i role (Human i hasChild) mogu da se interpretiraju
- TBox imaju izražajnost/ekspesivnost (expressiveness)

TBox

- Znanje u prethodnom primeru koje se opisalo sa DL, očigledno da može da se predstaviti i pomoću formula logike prvog reda
- Zašto bismo koristili deskriptivnu logiku, a ne logiku prvog reda?

TBox

- Zašto bismo koristili deskriptivnu logiku, a ne logiku prvog reda?
  - Odsustvo varijabli u opisima koncepata čini iskaze TBoxa čitljivijim od odgovarajućih iskaza logike prvog reda
  - Glavni razlog, je taj što je DL pažljivo skrojena tako da spaja ekspresivnost (expressiveness) i odlučivost (decidability) za rešavanje važnih problema rezonovanja (jasno će biti kada se objasni ABox)

ABox

- Činjenice (asertacioni deo baze znanja) zovu se Abox (podaci)
- Opisuju konkretne situacije iz aplikacionog domena navođenjem *vrednosti individuala i njihovih svojstva (propertija)*
- Odgovara analogno podacima u bazi podataka

ABox primer

- Činjenice:
- HappyMan*(*BOB*)*, hasChild*(*BOB,MARY*)*, ¬Doctor*(*MARY*)*
- Individual Bob pripada konceptu HappyMan
- Individual Bob je u relaciji hasChild sa Mary
- Individual Mary ne pripada konceptu Doctor

Rezonovanje u DL sistemima

- Sistemi bazirani na znanju koji koriste DL omogućuju automatsku dedukciju *implicitnog znanja* iz *eksplicitno predstavljenog znanja*
  - Garantuju **tačan odgovor** u **konačnom vremenu**
- Za razliku od (većine) baza podataka, prilikom rezonovanja u obzir se uzimaju i terminološki iskazi (šema, TBox) i asertacioni iskazi (podaci, činjenice, ABox)

Rezonovanje u DL sistemima

- Algoritam obuhvatanja
- Algoritam instanciranja
- Algoritam konzistentnosti

Algoritam obuhvatanja

- Algoritam obuhvatanja (*subsumption*) određuje relacije podkoncept/nadkoncept
- $C \sqsubseteq D$
- Ako koncept *D* (nadkoncept) obuhvata koncept *C* (podkoncept), onda su svi instance koncepta *C* ujedno i instance koncepta *D*
- Deskripcija koncepta *C* je podskup deskripcije koncepta *D*
- Ekspicitno navodimo $Doctor \sqsubseteq Human$

Primer

$$\text{HappyMan} \equiv \text{Human} \sqcap$$
$$\neg\text{Female} \sqcap$$
$$(\exists \text{married}.\text{Doctor}) \sqcap$$
$$(\forall \text{hasChild}.(\text{Doctor} \sqcup \text{Professor})) \sqcap$$
$$\textbf{Doctor} \sqsubseteq \textbf{Human}$$

- U deskripciju koncepta *HappyMan* dodali smo novi terminološki aksiom (Doctor $\sqsubseteq$ *Human*) koji kaže da su svi doktori u suštini ljudi
- **Zaključak:** $\exists \text{married}.\text{Human}$
- Rezonujemo algoritmom obuhvatanja da je koncept *HappyMan* obuhvaćen konceptom $\exists \text{married}.\text{Human}$
  - Sve instance koncepta *HappyMan* na relaciji *married* nekom instancom koncepta *Doctor,* a instance koncepta *Doctor su takođe* i instance koncepta *Human*
- **Rezonuje nad TBox**

Algoritam instanciranja

- Određuje relacije instanciranja
- Individual *i* je instanca deskripcije koncepta *C* ako je *i* uvek interpretiran kao element interpretacije koncepta *C*

Primer

TBox:

$$\text{HappyMan} \equiv \text{Human} \sqcap \neg\text{Female} \sqcap (\exists \text{married}.\text{Doctor}) \sqcap (\forall \text{hasChild}.(\text{Doctor} \sqcup \text{Professor}))$$

ABox:

$$\text{HappyMan}(\text{BOB}),\ \text{hasChild}(\text{BOB},\text{MARY}),\ \neg\text{Doctor}(\text{MARY})$$

Zaključak: *MARY* je instanca koncepta *Professor*

Rezonujemo algoritmom instanciranja

Objašnjenje:

- BOB je instanca koncepta HappyMan; **ABox**
- Svi individuali sa kojima je BOB u relaciji hasChild su instance ili koncepta Doctor ili koncepta Professor; **TBox**
- MARY je dete od BOB; **ABox**
- MARY nije instanca koncepta Doctor; **ABox**
- znači rezonujemo algoritmom instanciranja ***MARY* je instanca koncepta *Professor***

Rezonuje koristeći TBox i ABox

Algoritam konzistentnosti

- Proverava da li je baza znanja (ABox i TBox) u konzistentnom stanju (nije kontradiktorna)

Primer

TBox:

$$\text{HappyMan} \equiv \text{Human} \sqcap \neg\text{Female} \sqcap (\exists \text{married}.\text{Doctor}) \sqcap (\forall \text{hasChild}.(\text{Doctor} \sqcup \text{Professor}))$$

ABox:

HappyMan*(*BOB*)*, hasChild*(*BOB*,*MARY*)*, ¬Doctor*(*MARY*)*, **¬Professor*(*MARY*)***

U asetacioni deo Abox smo dodeli novu činjenicu ¬Professor*(*MARY*)*

Rezonujemo algoritmom konzistentnosti

Zaključak: Baza znanja koja sadrži 4 činjenice i opis koncepta HappyMan je nekonzistentna, proveravamo konzistentnos ABox u odnosu na TBox

Rezonuje koristeći TBox i ABox

Zahtevi od DL sistema

- Da bi se osiguralo razumno i prediktivno ponašanje DL sistema, svi problemi koji se rešava trebaju da budu **odlučivi (decidable)**
- Da bi DL sistemi bili upotrebljivi u realnim situacijama problemi koji se rešavaju trebaju da budu **niske složenosti**
  - Posledica je da se moraju uvodi ograničenja u ekspresivnost DL sistema
  - Ako su ograničenja prevelika, opisi relevantnih ideja iz domena primene više ne mogu da izraze
- DL sistemi moraju da budu dovoljno ekspresivni da bi mogli da se koriste za opis kompletnog domena primene
- Potrebno je napraviti dobar trade-off između ekspesivnosti i složenosti problema
- Postoje mnoga proširenja osnovnih DL sistema koja nisu iz domena logike prvog reda
  - DL sa Modalnim i temporalnim operatorima
  - Fazi DL
  - Probabilističke DL

$\mathcal{ALC}$

- Skraćeno od $\mathcal{A}$ttributive concept $\mathcal{L}$anguage with $\mathcal{C}$omplements
- $\mathcal{ALC}$ je primer bazičnog DL jezika
- Deskriptivna logka koja podržava:
  - konjunkciju,
  - disjunkciju,
  - negaciju,
  - egzistencijalna ograničenja ($\exists r.C$)
  - ograničenja vrednosti $\forall r.C$

Sintaksa kostrukta $\mathcal{ALC}$

- Def: Neka su N_C skup naziva koncepata i N_R skup naziva rola. Skup $\mathcal{ALC}$ deskripcija koncepata je najmanji skup takav da je:
  1. $\top$, $\bot$, i svaki naziv koncepta $A \in N_C$ je ujedno i $\mathcal{ALC}$ deskripcija koncepta,
  2. Ako su C i D $\mathcal{ALC}$ deskripcije koncepata i $r \in N_R$, onda su konstrukcije $C \sqcap D$, $C \sqcup D$, $\neg C$, $\forall r.C$ i $\exists r.C$ takođe $\mathcal{ALC}$ deskripcije koncepata.

Često se umesto termina $\mathcal{ALC}$ deskripcija koncepta koristi termin $\mathcal{ALC}$ koncept

Aksiomi u TBox-u

- Najopštija forma TBox aksimona predstavlja Generalna inkluzija koncepata (GCI)
- Analgno nasleđivanju u OOP
- Def: GCI je iskaz oblika $C \sqsubseteq D$, gde su C i D $\mathcal{ALC}$ koncepti. Konačni skup GCI čini TBox. Interpretacija I je model GCI $C \sqsubseteq D$ ako $C^I \subseteq D^I$
- Interpretacija I je model TBox-a T ako je model svake GCI iz T
- **Doctor** $\sqsubseteq$ **Human**

Definisanje

- Simetrični par generalnih inkluzija koncepata $C \sqsubseteq D$ i $D \sqsubseteq C$ skraćeno se piše $C \equiv D$ i zove se *definicija*
- TBox *T* zovemo ***definicioni*** ako sadrži samo definicije koncepata:
  - Sadrži najviše jednu definiciju za svaki naziv koncepta
  - Acikličan je (definicija koncepta A u T ne upućuje direktno ili indirektno opet na A)
- U definicionom TBox-u:
  - Koncepti sa leve strane se zovu ***definisani* koncepti**
  - Koncepti sa desne strane se zovu ***primitivni* koncepti**
- Rezonovanje nad definicionim TBox-ovima je veoma efikasno, pa se koriste kad god je to moguće
- HappyMan ≡ Human ⊓ ¬Female⊓ (∃married.Doctor)⊓....

Aksiomi u ABox-u

- Postoje dva tipa aksioma u ABox-u,
  - prvi za asertovanje individuala kao instanci koncepta,
  - a drugi za asertovanje para individuala kao instance rola
- Def: Asertacioni aksiom može da bude oblika $x : C$ ili oblika $(x, y) : r$ pri čemu je C $\mathcal{ALC}$ koncept, r naziv rola, a x i y su nazivi individuala. Konačan skup asertacionih aksioma je ABox.
  - Interpretacija I je model asertacionog aksioma $x : C$ ako $x^I \in C^I$
  - Interpretacija I je model asertacionog aksioma $(x,y) : r$ ako $(x^I,y^I) \in r^I$
  - I je model ABox-a A ako je model svakog asertacionog individuala u A

Baza znanja

- BZ je Par (*T, A*) gde je *T* TBox, a *A* je ABox.
- Interpretacija *I* je model baze znanja *K=(T,A)* ako je *I* model *T* i *I* model od *A,* što se piše I |= K
- Znači, kada pravimo bazu znanja, možemo da definišemo:
  - Koji koncepti su uključeni u koje koncepte (TBox),
  - Koji individuali su instance kojih koncepata (ABox) i
  - Koji individuali su u relaciji sa kojim individualima (ABox)

Baza znanja

- Def: Za bazu znanja *K=(T,A)* kažemo da je *konzistentna* ako *ima model*.
  - Za koncept C kažemo da je zadovoljiv ($K \models C$) u odnosu na K ako postoji interpretacija I baze znanja K takva da $C^I \neq \emptyset$
  - Takva intepretacija C^I zove se *model C* u odnosu na *K*
  - Koncept *D* obuhvata koncept *C* u odnosu na *K* ($K \models C \sqsubseteq D$) ako je $C^I \subseteq D^I$ za svaki *I* modela *K*
  - Dva koncepta *C* i *D* su ekvivalentna u odnosu na bazu znanja *K* ($K \models C \equiv D$) ukoliko i *C* obuhvata *D* i *D* obuhvata *C* u odnosu na *K*
  - Individual *a* je instnca koncepta *C* u odnosu na *K* ($K \models a:C$) ako $a^I \in C^I$ važi za svaki model *I* baze znanja *K*
  - Par individuala *(a,b)* je u relaciji *r* u odnosu na *K* ($K \models (a,b):r$) ako $(a^I, b^I) \in r^I$ za svaki *I* modela *K*

Rezonovanje

- Za deskriptivne logike sa Bulovim operatorima (kao što je $\mathcal{ALC}$) svo rezonovanje se svodi na proveru konzistentnosti baze znanja
- Na primer, individual *a* je instanca koncepta *C* $K \models a:C$ akko pri dodavanju iskaza $a:\neg C$ u ABox baza znanja $(T,A \cup \{a:\neg C\})$ postaje nekonzistentna

Važna proširenja $\mathcal{ALC}$

- DL su našli važnu primenu kao formalni osnov za definisanje jezika za specificiranje ontologija.
  - DL pužaju dobro definisanu semantiku i moćne tehnike rezonovanja
- Primeri DL baziranih jezika za ontologije su OIL, OIL+DAML i OWL (ontology language standard developed by the W3C Web-Ontology Working Group).
- Da bi se DL koristili u ontologojama kreirana su razna proširenja
- Neka od tipičnih prišireња $\mathcal{ALC}$ obuhvataju:
  - Ograničenja brojeva
  - Inverzne role
  - Tranzitivne role
  - Podrole
  - Konkretne domene
  - Nominale konstrukte

Ograničenja brojeva

- Kardinaliteti za koncepte i relacije.
- Za određeni individual moguće je specificirati u koliko instanci relacija određenog tipa on može da učestvuje
- Primeri:
- Ako hoćeno da kažemo da osoba može biti u braku najviše sa 1 individualom

$$\text{Person} \leq 1\ \text{married}$$

- Ako hoćemo da kažemo da je osoba srećna ako ima od 2 do 4 deteta

$$\begin{aligned}\text{HappyMan} \equiv\ & \text{Human} \sqcap \\ & \neg\text{Female} \sqcap \\ & (\exists \text{married}.\text{Doctor}) \sqcap \\ & (\forall \text{hasChild}.(\text{Doctor} \sqcup \text{Professor})) \sqcap \\ & \geq \mathbf{2\ hasChild} \sqcap \\ & \leq \mathbf{4\ hasChild}\end{aligned}$$

http://www.w3.org/2001/sw/WebOnt/

Ograničenja brojeva

- Moguće je specificirati i tip individuala koji se nalazi u relaiciji ograničenog kardinaliteta

$$\text{HappyMan} \equiv \text{Human} \sqcap$$
$$\neg\text{Female} \sqcap$$
$$(\exists \text{married}.\text{Doctor}) \sqcap$$
$$(\forall \text{hasChild}.(\text{Doctor} \sqcup \text{Professor})) \sqcap$$
$$\geq 2\ \text{hasChild}.\mathbf{Doctor} \sqcap$$
$$\leq 4\ \text{hasChild}$$

Inverzne, tranzitivne role i podrole

- Uz rolu hasChild možemo definisiati:
  - Inverzna rolu: hasParent
  - Tranzitivna rola: hasAncestor
  - Podrolu: hasParent je podrola od hasAncestor

Deskriptivne logike i prediktska logika

- Deskriptivne logike su uglavnom podskupovi logike prvog reda
  - Mogu da sadrže i konstrukte logike drugo reda (na primer tranzitivno zatvaranje rola)
  - Role su binarni predikati
  - Koncepti su unarni predikati
- Deskriptivne logike se koriste umesto predikatske logike jer su uglavnom odlučivi fragmenti predikatske logike

više naredni čas

SW i predstavljanje znanja

- Formalno reprezentovanje znanja na Webu
- Rezonovanje nad tim formalno reprezentovanim znanjem
- Jezik za reprezentovanje znanja je OWL (Web Ontology Language)
  - Razvijen od strane W3C https://www.w3.org/OWL/

OWL semantika

- Semantika OWL se definiše prevođenjima u ekspresivnu deskriptivnu logiku
- To nije slučajno – to je bio cilj
  - Omogućeno je da OWL koristi rezultate istraživanja iz oblasti desrkitpivnih logika (na primer odlučivost i složenosti tipičnih problema rezonovanja)
  - Omogućeno je da OWL koristi gotove DL rezonere
    - FaCT,
    - RACER,
    - …
  - Sa ciljem da se obezbedi rezonovanje za OWL aplikacije

OWL ontologija

- Baza znanja OWL ontologija odgovara DL TBox sa hijerarhijom rola, u kome se OWL
  - Klase (Classes) predstavljaju koncepti iz DL
  - Svojstva (Properties) predstavljaju role iz DL
- Ontologija se sastoji od skupa asertovanih *aksioma,* npr. relacija obuhvatanja između klasa ili svojstava
- Kao i u deskriptivnim logikama za koncepte, i u OWL klasa može da bude data ili imenom ili iskazom koji se sastoji od jednostavnijih klasa i propertija i konstrukta

OWL ontologija

- Primer: DL iskaz Human ⊓ Male može se u ontologiji napisati (RDF sintkasa)

```
<owl:Class>
        <owl:intersectionOf rdf:parseType="Collection">
                <owl:Class rdf:about="#Human"/>
                <owl:Class rdf:about="#Male"/>
        </owl:intersectionOf>
</owl:Class>
```

OWL ontologija

- Primer: DL iskaz ≥ 2 hasChild može se u ontologiji napisati (RDF sintkasa)

```
<owl:Restriction>
        <owl:onProperty rdf:resource="#hasChild"/>
        <owl:minCardinality
                rdf:datatype="&xsd;NonNegativeInteger">2
        </owl:minCardinality>
</owl:Restriction>
```

Metode za reprezentaciju znanja (ZR metode)

- **Produkciona pravila**
- Stabla, grafovi (n.pr. Semantičke mreže, konceptne mape, konceptni grafovi, ontologija)
- Šeme i frejmovi
- Logika

=> Rezonovanje

Rezonovanje

- Rekapitulacija
- Proces konstruisanja novih konfiguracija (rečenica/iskaza) iz starih
  - Ispravno rezonovanje osigurava da nove konfiguracije predstavljaju činjenice koje slede iz činjenica koje su predstavljene starim konfiguracijama
  - Ova veza zove se *posledičnost* (*entailment)* što se piše kao
    KB |= alpha
    - Rečenica alpha je posledica baze znanja KB

Metode zaključivanja

- Procedura zaključivanja može da uradi jednu od dve stvari:
  - Ako je data baza znanja KB, reznovanje može da izvede (derivation) nove rečenice α koje su (pretpostavljeno) posledica KB
    KB |- α ==> KB |= α
  - Ako je data baza znanja KB i druga rečenica α, rezonovanje može da utvrdi da li je α posledica KB ili nije
    KB ∧ α ==> KB |= α
- Zaključivanje koja generiše samo posledične rečenice naziva se ispravnim (sound) ili *truth-preserving*
- Zapis operacija procedure zaključivanja zove se dokaz (proof).
- Za proceduru zaključivanja kaže se da je kompletna ako može da nadje svaki dokaz za svaku rečenicu koja je posledična.

Jezici RZ i programski jezici

- Po čemu se jezici za reprezentaciju znanja (RZ) razlikuju od drugih programskih jezika (Java, C++, C#)?
  - Programski jezici mogu da predstave činjenice i iskaze
- Šta je sa situacijom „postoji jama u lokaciji [2,2] ili [3,1] (ali nismo sigurni)" ili „postoji monstrum u nekom kvadratu"
- Programski jezici nisu dovoljno izražajni za situacije sa nekompletnim informacijama

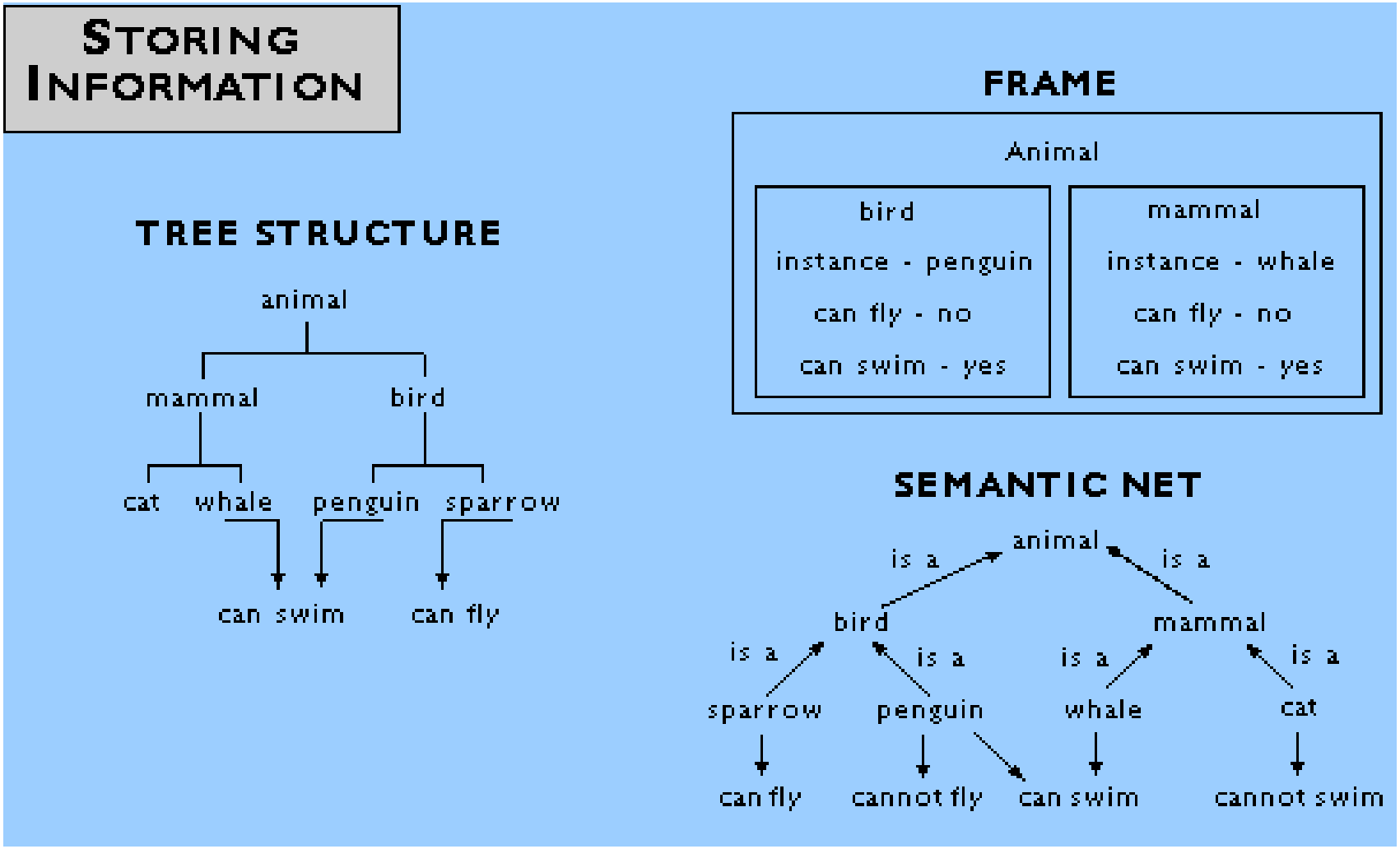
Jezici RZ i prirodni jezik

- Po čemu se jezici za predstavljanje zanja razlikuju od prirodnih jezika
- Prirodni jezici su izražajni oni su se razvijali da zadovolje potrebe komunikacije, a ne reprezentacije. Nama teba reprezentovano znanje.
- Značanje rečenice zavisi od same rečenice i od konteksta u kome je izrečena
  - n.pr. “Look!”
- Razmena znanja se obavlja bez eksplicitne reprezentacije samog znanja
- neodređenosti (n.pr. Mali psi i mačke)

Dobri RZ jezici

- Kombinuju prednosti prirodnih i formalnih jezika:
  - izražajni
  - koncizni
  - nedvosmisleni
  - Nezavisni od konteksta
    - Što se iskaže danas moći će da se interpretira sutra
  - Efikasni
    - Znanje se može predstaviti u mašinski čitljivom obliku
    - Za odabrani format postoje praktične procedure zaključivanja
  - Efektivni
    - Postoji procedura zaključivanja kojom se iz reprezentacije znanja mogu napraviti nove rečenice

Primer: Metodi reprezentacije



314

Šta je Semantički Web?

- Proširenje World Wide Web-a
- Web koji razumeju računari a ne samo ljudi (web podataka, a ne web dokumenata)
  - Namenjen softverskim agentima a ne samo ljudima
  - Definisan strukturiranim dokumentima (recimo, ontologije) koji mogu da referenciraju jedan drugog i da formiraju na taj način ogromne mreže
  - Koristi se za simulaciju znanja u računarskim sistemima
- Dokumenti Semantičkog Web-a mogu da opišu puno toga što ljudi žele da razmenu

Ontologije i Semantički Web

- Ontologije su veliki rečnici
  - Definisani u dokumentima Semantičkog Web-a (OWL)
  - Definišu jezike za druge dokumente (RDF)
  - Resursi mogu da budu instance klasa ontologije
- Upper Ontologije definišu osnovne, apstraktne koncepte
- Lower Ontologije definišu domenski specifične koncepte
- Meta-ontologije definišu druge ontologije

IEEE Standard Upper Ontology(SUO)

- Projekat razvoja standarda za specifikaciju i registrovanje ontologije
- Baziran na doprinosima tri kandidatska SUO projekta
  - IFF
  - OpenCyc/CycL
  - SUMO
- Standard Upper Ontology Working Group (SUO WG), Cumulative Resolutions, 2003, http://suo.ieee.org/SUO/resolutions.html

OpenCyc

- Izvedena iz razvoja Cyc
  - Džinovski sistem baziran na znanju
- Cycorp, The Syntax of CycL, 2002, http://www.cyc.com/cycdoc/ref/cycl-syntax.html http://www.cyc.com/publications/



OpenCYC (1)

- OpenCyc Platform je gejtvej za Cyc, najveću i najkompletniju svetsku bazu znanja i modul za zadravorazumsko rezonovanje.
- OpenCyc sadrži stotine hiljada Cyc termina organizovanih u ontologiju.
- OpenCyc može da se koristi kao osnova za inteligentne aplikacije kao što su:
  - Modelovanje domena
  - Semantička integracija podataka
  - Razumevanje teksta
  - Domenski specifične ekspertske sisteme, itd.

OpenCyc Release 4.0

- **Kor Cyc ontologiju** čiji domen je realitet ukupnog ljudskog konsenzusa o znanju. Tekuće izdanje obuhvata:
  - **~239,000 termina**
  - **~2,093,000 tripleta**
  - Klase:
    - **'place'**: ~19,000
    - **'organization'**: ~26,000
    - **'predicate'**: ~22,000
    - **'business related thing'**: ~28,000
    - **'person'**: ~12,700
  - **~69,000 owl:sameAs links** na eksterne (non-Cyc) prostore imena semantičkih podataka:
    - DBpedia: ~47,000 linkovi, *ukjlučujući 696 linkova na ontotlogiju DBpedia*
    - UMBEL: ~21,000 linkova
    - WordNet: ~11,000 linkova
    - Wikicompany: 1028 linkova
    - CIA World Factbook: 172 linkova
    - RDFAbout SEC identifikatori kompanija: 661 linkova
    - RDFAbout o državama i opštinama: 71 link
    - FOAF: 44 linkova
  - Engleski stringovi (kanonički i alternativni) odgovarajući koncptnim termovima za pomoć u pretrazi i prikazivanju.
- Java-bazirane Cyc Inference Engine i Cyc Knowledge Base Browser
- Dokumentaciju i materijal za učenje.
- Specifikaciju jezika CycL u kome su Cyc i OpenCyc napisani.
- Specifikaciju Cyc API za razvoj aplikacija.

SUMO

- Skraćenica za “Suggested Upper Merged Ontology”
- Niles, Ian, and Adam Pease, Towards a Standard Upper Ontology, 2001
- Standard Upper Ontology Working Group (SUO WG), Cumulative Resolutions, 2003, http://suo.ieee.org/SUO/resolutions.html http://www.adampease.org/OP/



SUMO (http://www.adampease.org/OP/)

- SUMO i prateće domenske ontologije predstavljaju najveću postojeću formalnu javno dostupnu ontotlogiju današnjice
- SUMO je napisana u jeziku SUO-KIF.
- SUMO je vlasništvo IEEE i besplatna je.
- Ima ~25,000 termina i ~80,000 aksioma u svim ontologijama
- Bogato je aksiomatizovana. Svi termini su formalno definisani. Značenja su nezavisna od implementacije zaključivanja. Obezbeđen je sistem zaključivanja i upravljanja ontologijom. Obezbeđen je i sistem za podršku vizualnom editovanju i prikazu ontologija (KSMSA sistem).
- SUMO je jedina formalna ontologija koja je potpuno mapirana na WordNet leksikon.
- Koristi se za istraživanja i aplikacije u pretragama, lingvistici i rezonovanju
- Adam Pease je tehnički urednik SUMO.

WordNet®

- Onlajn leksički referentni sistem
  - Dizajn je inspirisan psiholingvističkim teorijama ljudske leksičke memorije
- Imenice, glagoli, prilozi i predlozi engleskog jezika
  - Organizovani u skupove sinonima koji predstavljaju pojedinačne leksičke koncepte
- Radi se i za druge jezike

Važni koncepti i termini

- Atribut
- Baza znanja
- Epistemologija
- Ekspertski sistem
- facet
- Frejm
- Graf
- Informacija
- Izvođenje
- If-Then pravila
- Koncept
- Link
- Logika
- Ljuska ekspertskog sistema
- Mehanizam zaključivanja
- Meta-znanje
- Nasleđivanje
- Objekat
- Ontologija
- Podatak
- Pravilo
- Produkciona pravila
- Rasuđivanje
- Relacija
- Reprezentacija znanja
- Semantička mreža
- Sistem baziran na znanju (SBZ)
- Slot
- Skript
- Šum
- Šema
- Čvor
- Zaključivanje
- Zdravorazumsko znanje
- Znanje

Sažetak

- Predstavljanje znanja je (a kako i ne bi bilo?) veoma važno sa SBZ
- Popularne reprezentacione šeme su
  - pravila, sementičke mreže, šeme (frejmovi, skriptovi), logika
- Reprezentacione šeme treba da budu podržane odgovarajućim metodama zaključivanja da bi se omogućilo rasuđivanje
- Mora se naći ravnoteža između
  - izražajnosti, računske efikasnosti, razumljivosti